

1. Design and implement processes and communications

└ 1.1 Design and implement traceability and flow of work

└ 1.1.1 Design and implement GitHub Flow

1. What is GitHub Flow and how does it differ from Git Flow?
2. What are the steps of a standard GitHub Flow?
3. How do you implement branch protections in GitHub?
4. How are pull request (PR) reviews enforced in GitHub repositories?
5. What are status checks and how are they configured in GitHub Actions?
6. How can you automate deployments with GitHub Actions during GitHub Flow?
7. How do environment protection rules function in GitHub workflows?
8. How do you manage secrets and credentials in GitHub Actions?
9. What are best practices for committing code in a GitHub Flow process?
10. How do you roll back changes in GitHub Flow?

1. What is GitHub Flow and how does it differ from Git Flow?

GitHub Flow is a lightweight, trunk-based workflow focused on continuous delivery using a main branch and short-lived feature branches. Unlike Git Flow, it avoids long-lived branches like develop or release.

2. What are the steps of a standard GitHub Flow?

1. Create a feature branch from main
2. Make and commit changes locally
3. Push the branch to GitHub
4. Open a pull request (PR)
5. Review, approve, and merge the PR
6. Automatically or manually deploy after merge

3. How do you implement branch protections in GitHub?

In repository settings > Branches:

- Require PR reviews before merging
- Require status checks to pass
- Restrict who can push to the branch

4. How are pull request (PR) reviews enforced in GitHub repositories?

Enable “*Require pull request reviews before merging*” under branch protection rules. You can specify required reviewers and enforce code owner review.

5. What are status checks and how are they configured in GitHub Actions?

Status checks are CI workflows (e.g., build/test) that must pass before a PR can be merged. Define them in `.github/workflows/*.yml` and mark them as required in branch protection settings.

6. How can you automate deployments with GitHub Actions during GitHub Flow?

Use workflows triggered on push or pull_request events. Use jobs and steps to deploy to environments like Azure or AWS. Example:

on: push to main → deploy job runs → production environment

7. How do environment protection rules function in GitHub workflows?

They restrict deployment via GitHub Actions. Configure environments with:

- Required reviewers
 - Wait timer
 - Custom deployment branches
-

8. How do you manage secrets and credentials in GitHub Actions?

Store secrets in *Repository Settings > Secrets and variables > Actions*. Access them using `${{ secrets.SECRET_NAME }}` in workflow YAML.

9. What are best practices for committing code in a GitHub Flow process?

- Use atomic commits (one change per commit)
 - Write clear commit messages
 - Rebase before merging
 - Avoid pushing directly to main
-

10. How do you roll back changes in GitHub Flow?

Use `git revert` to create a new commit that undoes changes. Push the revert commit and merge via PR, triggering redeployment if needed.

1. Design and implement processes and communications

└ 1.1 Design and implement traceability and flow of work

└ 1.1.2 Design and implement feedback cycles

1. What are feedback cycles in DevOps and why are they critical?
 2. How do you configure notifications in Azure DevOps pipelines?
 3. How do you set up GitHub issue automation for feedback tracking?
 4. What integrations are available between GitHub issues and Azure Boards?
 5. How do you configure branch protections with required status checks for feedback?
 6. How do you use GitHub Actions to automate issue creation on failure?
 7. What are the best practices for actionable notification configuration?
 8. How can you integrate feedback from Azure Monitor into GitHub or Azure DevOps?
 9. How do you set up pull request templates to improve developer feedback cycles?
 10. How do service hooks enable automated feedback loops across tools?
-

1. What are feedback cycles in DevOps and why are they critical?

Feedback cycles provide continuous insights (build status, test results, deployment issues) to developers, enabling rapid detection and correction, reducing lead time and improving quality.

2. How do you configure notifications in Azure DevOps pipelines?

Go to *Project Settings* → *Notifications* → *Create Subscription* → *Select event* (e.g., build failure) → *Define filters* (e.g., pipeline name) → *Set recipients* (users or groups).

3. How do you set up GitHub issue automation for feedback tracking?

Use GitHub Actions with *actions/create-issue* or similar to automatically open issues on failures, e.g., test failures or workflow errors, linking to logs or artifacts.

4. What integrations are available between GitHub issues and Azure Boards?

Install the *Azure Boards GitHub app* → *Link repos* → *Use linked commits, PRs, and issues to automatically update Azure Boards work items* for traceability.

5. How do you configure branch protections with required status checks for feedback?

In GitHub → *Settings* → *Branches* → *Add branch protection rule* → *Enable required status checks* (e.g., CI pipeline) → *Ensure passing checks before merging*.

6. How do you use GitHub Actions to automate issue creation on failure?

In a workflow, use a *jobs.<job>.if: failure()* condition → *Add a step* using a GitHub token to call the REST API or a prebuilt action like *peter-evans/create-issue*.

7. What are the best practices for actionable notification configuration?

Notify only relevant users/groups, include clear context (links to builds, commits), avoid noise (e.g., no “success” spam), and prioritize critical failures.

8. How can you integrate feedback from Azure Monitor into GitHub or Azure DevOps?

Use *Azure Monitor Action Groups* → *Webhooks, Logic Apps, or Azure Functions* → *Post alerts as GitHub issues or Azure DevOps work items* automatically.

9. How do you set up pull request templates to improve developer feedback cycles?

In `.github/PULL_REQUEST_TEMPLATE.md`, define required sections (e.g., linked issues, testing done) to standardize information and reduce reviewer cycles.

10. How do service hooks enable automated feedback loops across tools?

Service hooks in Azure DevOps or GitHub connect events (build failures, PR updates) to external systems like Slack, Teams, or custom webhooks, triggering automated feedback.

1. Design and implement processes and communications

└ 1.1 Design and implement traceability and flow of work

└ 1.1.3 Integrate GitHub Projects, Azure Boards, repositories

1. What are the benefits of integrating GitHub Projects with Azure Boards?
 2. How do you connect a GitHub repository to Azure Boards?
 3. What permissions are required to enable GitHub–Azure Boards integration?
 4. How do you automatically link GitHub commits and pull requests to Azure Boards work items?
 5. How can you synchronize work item state between Azure Boards and GitHub Projects?
 6. What are best practices for maintaining traceability between code and work items?
 7. How do you configure automation between GitHub issues and Azure Boards?
 8. What are common troubleshooting steps if integration fails?
 9. How do you use service hooks/webhooks to enhance integration?
 10. What limitations exist when integrating GitHub Projects, Azure Boards, and repositories?
-

1. What are the benefits of integrating GitHub Projects with Azure Boards?

- Enables end-to-end traceability between code, pull requests, and work items.
 - Facilitates agile planning and tracking using *Azure Boards* while developing in *GitHub*.
 - Automatically updates work items from *GitHub* activity, increasing visibility and reducing manual effort.
-

2. How do you connect a GitHub repository to Azure Boards?

- Install the *Azure Boards* app for *GitHub* from the *GitHub Marketplace*.
 - Authorize the app and select the repository.
 - In *Azure Boards*, link the *GitHub* repository using *Project Settings > GitHub Connections*.
-

3. What permissions are required to enable GitHub–Azure Boards integration?

- You need admin permissions on the *GitHub* repository.
 - You must have at least Basic access to the *Azure DevOps* project.
 - The *Azure Boards* app must be authorized with appropriate OAuth permissions to access repos and metadata.
-

4. How do you automatically link GitHub commits and pull requests to Azure Boards work items?

- Use the syntax AB#{work item ID} OR Fixes AB#{work item ID} in commit messages or PR titles/descriptions.
 - Linked work items are updated automatically in *Azure Boards* with commit and PR information.
-

5. How can you synchronize work item state between Azure Boards and GitHub Projects?

- Changes to linked work items in *Azure Boards* are not automatically reflected in *GitHub* Projects.
 - Manual updates or third-party automation (e.g., *GitHub Actions* or *Azure Logic Apps*) are needed for bi-directional state sync.
-

6. What are best practices for maintaining traceability between code and work items?

- Always reference work item IDs in commit messages and PRs.
- Use branch naming conventions (e.g., feature/AB#123-description).
- Enforce PR templates that require work item linkage.
- Periodically audit links between code and work items.

7. How do you configure automation between GitHub issues and Azure Boards?

- Use *Azure Logic Apps* or *GitHub Actions* to create workflows that create or update *Azure Boards* work items based on *GitHub* issue events.
 - *Service hooks* in *Azure Boards* can trigger actions in response to *GitHub* issue activity.
-

8. What are common troubleshooting steps if integration fails?

- Verify *Azure Boards* app is installed and authorized in *GitHub*.
 - Check repository and user permissions in both *GitHub* and *Azure DevOps*.
 - Ensure the repository is correctly linked in *Azure Boards* settings.
 - Review integration logs in *Azure DevOps* for errors.
-

9. How do you use service hooks/webhooks to enhance integration?

- Configure service hooks in *Azure DevOps* to send updates to external systems (like *GitHub* or *Teams*) based on work item changes.
 - Set up *GitHub webhooks* to trigger automation in response to repo or issue events.
-

10. What limitations exist when integrating GitHub Projects, Azure Boards, and repositories?

- Work item state is not bi-directionally synced by default.
- Some features (e.g., advanced queries or cross-org integration) may not be supported.
- Large numbers of repositories may require individual configuration.

1. Design and implement processes and communications

└ 1.1 Design and implement traceability and flow of work

└ 1.1.4 Implement source, bug, and quality traceability

1. What is traceability in DevOps and why is it important?
2. How do you link code changes to work items in Azure DevOps and GitHub?
3. How is bug traceability achieved in modern DevOps workflows?
4. What tools and features enable quality traceability across pipelines?
5. How do you visualize traceability from source code to deployment?
6. What practices ensure end-to-end traceability for compliance and audits?
7. How can automated testing results be linked to work items and requirements?
8. What are common pitfalls in traceability implementation?
9. How do you configure dashboards or reports to show traceability status?
10. What limitations should be considered when implementing traceability?

1. What is traceability in DevOps and why is it important?

- Traceability is the ability to track work items, code changes, bugs, and test results throughout the development lifecycle.
- It ensures transparency, accountability, fast root cause analysis, supporting compliance.

2. How do you link code changes to work items in Azure DevOps and GitHub?

- In *Azure DevOps*, include the work item ID in commit messages, PR titles, or branch names (e.g., #123).
- In *GitHub*, use keywords like Fixes #123 in commits or PRs to link issues.
- Both platforms automatically update the work item or issue with commit and PR references.

3. How is bug traceability achieved in modern DevOps workflows?

- Bugs are tracked as work items/issues in *Azure Boards* or *GitHub Issues*.
- Link code commits and PRs to these bugs using IDs or keywords.
- Automated and manual test results can be associated with bug work items to show resolution status.

4. What tools and features enable quality traceability across pipelines?

- *Azure DevOps* Test Plans, *GitHub Actions*, and pipeline integrations enable linking test runs, results, and code changes to work items.
- Test results, code coverage, and release status can be tracked and visualized against user stories and bugs.

5. How do you visualize traceability from source code to deployment?

- Use *Azure DevOps* traceability views, queries, or built-in dashboards to follow links from work items to code commits, builds, releases, and test results.
- Custom dashboards and reports can be built using *Power BI* or *Azure DevOps Analytics* for full traceability visualization.

6. What practices ensure end-to-end traceability for compliance and audits?

- Enforce linking of every code change and deployment to a tracked work item or requirement.
- Require automated association of test results and bugs to work items in pipelines.
- Maintain detailed audit trails and use approval workflows.

7. How can automated testing results be linked to work items and requirements?

- Configure pipelines to associate test runs with specific work items.
 - Use test case management features in *Azure DevOps* to map test cases to requirements.
 - Automated test tools can update work items with results using REST APIs or extensions.
-

8. What are common pitfalls in traceability implementation?

- Inconsistent work item references in commits/PRs.
 - Lack of enforcement on associating code changes with work items.
 - Poor test case management or missing links between tests and requirements.
-

9. How do you configure dashboards or reports to show traceability status?

- In *Azure DevOps*, use built-in widgets and queries to display links between work items, code, tests, and releases.
 - *Power BI* can connect to *Azure DevOps Analytics* for advanced, custom traceability reporting.
-

10. What limitations should be considered when implementing traceability?

- Full automation may not cover all scenarios; some manual linking may be required.
- Cross-tool traceability (e.g., between *GitHub* and *Azure Boards*) may require custom integration or third-party tools.
- Visualization and reporting capabilities may be limited by platform or data availability.

1. Design and implement processes and communications

└ 1.2 Metrics and queries for DevOps

└ 1.2.1 Dashboards: cycle time, time to recovery, lead time

1. What are cycle time, lead time, and time to recovery in DevOps, and how do they differ?
2. How do you configure dashboards in Azure DevOps to visualize these metrics?
3. Which built-in widgets are used to track cycle time, lead time, and time to recovery in AzDevOps?
4. What is the process for adding custom queries for metric visualization in Azure DevOps dashboards?
5. How can you monitor these metrics using GitHub Insights or GitHub Projects?
6. What actions can reduce cycle time and lead time in a DevOps workflow?
7. How do you configure alerts for abnormal values in these metrics?
8. How is time to recovery (MTTR) tracked in Azure DevOps and GitHub?
9. How should teams interpret trends and anomalies in these metrics for continuous improvement?
10. What are common pitfalls in configuring dashboards for DevOps metric

1. What are cycle time, lead time, and time to recovery in DevOps, and how do they differ?

- Cycle time: Time from starting work on a task to completion (e.g., first commit to deployment).
- Lead time: Time from when a request is made (work item created) to its delivery (deployment).
- Time to recovery (MTTR): Time taken to restore service after a production failure.
- Difference: Lead time includes all waiting; cycle time starts when actual work begins. MTTR focuses on incident recovery only.

2. How do you configure dashboards in Azure DevOps to visualize these metrics?

- Go to *Azure DevOps project > Dashboards > New Dashboard*.
- Add widgets such as Cycle Time, Lead Time, and Query Results.
- Configure each widget to target the correct team/project and backlog level.
- Save and share the dashboard as needed.

3. Which built-in widgets are used to track cycle time, lead time, and time to recovery in Azure DevOps?

- Cycle Time widget: Shows average time from “In Progress” to “Done.”
- Lead Time widget: Displays time from creation to completion.
- Query Results widget: Can be customized to show MTTR based on resolved incidents or bug work items.

4. What is the process for adding custom queries for metric visualization in Azure DevOps dashboards?

- Go to *Boards > Queries > New Query*.
- Define criteria (e.g., State changes, tags for incidents).
- Save and run the query.
- Add the Query Results widget to a dashboard and link it to the saved query for visualization.

5. How can you monitor these metrics using GitHub Insights or GitHub Projects?

- *GitHub Insights* (in *GitHub Enterprise*) provides charts for lead time and cycle time per repo.
- *GitHub Projects* tracks issue and PR workflow states; custom fields and automation can approximate cycle/lead time.
- External tools (e.g., *Azure DevOps*, third-party analytics) can be integrated for MTTR tracking.

6. What actions can reduce cycle time and lead time in a DevOps workflow?

- Automate builds, tests, and deployments.
 - Use small, incremental changes (shorter PRs).
 - Improve code review and approval process speed.
 - Minimize handoffs and blockers (clear ownership).
 - Invest in high test coverage to catch issues early.
-

7. How do you configure alerts for abnormal values in these metrics?

- In *Azure DevOps*, use Analytics views or *Power BI* integration for threshold-based alerts.
 - Configure dashboards with visual cues (color rules) for metric widgets.
 - Use *Azure Monitor* or *Logic Apps* to trigger notifications for breached thresholds.
-

8. How is time to recovery (MTTR) tracked in Azure DevOps and GitHub?

- *Azure DevOps*: Use queries for work items marked as incidents/production bugs; calculate time from “Created” to “Resolved/Closed.”
 - *GitHub*: Track incidents with labeled issues or PRs; use timestamps to calculate recovery duration manually or via automation.
-

9. How should teams interpret trends and anomalies in these metrics for continuous improvement?

- Rising cycle/lead times indicate process bottlenecks or resource issues.
 - Increasing MTTR signals gaps in incident response or recovery process.
 - Regular review in retrospectives supports targeted process improvement.
-

10. What are common pitfalls in configuring dashboards for DevOps metrics?

- Using inconsistent work item states or definitions across teams.
- Not updating queries when workflows change.
- Focusing on vanity metrics without actionable insights.
- Failing to align dashboards to current team processes and priorities.

1. Design and implement processes and communications

└ 1.2 Metrics and queries for DevOps

└ 1.2.2 Metrics/queries for: planning, development, testing, security, delivery, operations

1. What key metrics should be tracked for planning in Azure DevOps?
 2. What queries help monitor development progress and code health?
 3. How do you set up testing metrics and visualize test pass rates?
 4. What security metrics are essential in a DevOps pipeline?
 5. How can delivery performance be measured using Azure DevOps queries?
 6. What operational metrics should teams monitor post-deployment?
 7. How do you build cross-area queries combining multiple DevOps stages?
 8. What tools integrate with Azure DevOps to enhance metric reporting?
 9. How can you automate metric collection and reporting across stages?
 10. What are best practices for maintaining accurate, actionable queries?
-

1. What key metrics should be tracked for planning in Azure DevOps?

- Work item count (new, active, closed).
 - Sprint/iteration velocity.
 - Burndown/burnup charts.
 - Forecast vs. actual completion.
 - Backlog health (aging, blocked items).
-

2. What queries help monitor development progress and code health?

- Active pull requests and status.
 - Code churn (lines added/removed).
 - Commit frequency per developer/team.
 - Open vs. resolved bugs.
 - Code review completion rates.
-

3. How do you set up testing metrics and visualize test pass rates?

- Use *Test Plans > Runs > Query test results* (passed/failed/skipped).
 - Add Test Results Trend widget to dashboards.
 - Configure queries on test case outcomes over time.
-

4. What security metrics are essential in a DevOps pipeline?

- Number of open security bugs.
 - Dependency vulnerability scan results.
 - Secrets scan findings.
 - Code scanning (e.g., static analysis) pass/fail counts.
 - Compliance policy adherence.
-

5. How can delivery performance be measured using Azure DevOps queries?

- Deployment frequency (successful releases over time).
- Release success/failure rates.
- Average deployment duration.
- Rollback counts.
- Time from build to production deployment.

6. What operational metrics should teams monitor post-deployment?

- Incident counts and severity.
 - Mean Time to Detect (MTTD).
 - Mean Time to Recovery (MTTR).
 - Service uptime and availability.
 - Performance indicators (CPU, memory, latency).
-

7. How do you build cross-area queries combining multiple DevOps stages?

- Use Azure DevOps Analytics Views or custom queries linking work items, builds, and releases.
 - Apply area paths, tags, or shared fields across queries to correlate stages.
 - Combine results in *Power BI* or dashboards.
-

8. What tools integrate with Azure DevOps to enhance metric reporting?

- *Power BI* (advanced visualization).
 - *Azure Monitor* and *Log Analytics* (telemetry integration).
 - *Application Insights* (app-level metrics).
 - Third-party tools like *SonarQube*, *WhiteSource* (code quality, security).
-

9. How can you automate metric collection and reporting across stages?

- Use *Azure Pipelines* tasks to export metrics post-stage.
 - Set up scheduled *Analytics View* exports to *Power BI* or storage.
 - Leverage REST APIs to pull and aggregate data.
-

10. What are best practices for maintaining accurate, actionable queries?

- Regularly review query definitions as processes evolve.
- Use consistent naming, tags, and fields.
- Validate data sources and update filters.
- Focus on actionable, team-relevant metrics.

1. Design and implement processes and communications

└ 1.3 Configure collaboration and communication

└ 1.3.1 Document projects with wikis and diagrams

1. What is the purpose of using wikis in DevOps projects?
 2. How do you create and manage a wiki in Azure DevOps?
 3. What are best practices for structuring project documentation in a DevOps wiki?
 4. How do you use Markdown syntax for documentation in Azure DevOps or GitHub wikis?
 5. What is Mermaid syntax, and how is it used in project documentation?
 6. How do you embed and update diagrams in project wikis?
 7. How do you manage permissions and access control for wikis?
 8. What are best practices for keeping documentation up to date in fast-moving DevOps projects?
 9. How can you automate documentation updates from source control history?
 10. What are the benefits of linking work items and documentation in Azure DevOps?
-

1. What is the purpose of using wikis in DevOps projects?

Wikis provide a central, version-controlled location for project documentation, enabling teams to capture knowledge, share processes, and onboard new members efficiently. They support transparency, collaboration, and traceability across the DevOps lifecycle.

2. How do you create and manage a wiki in Azure DevOps?

In *Azure DevOps*, create a wiki by selecting “Wiki” in the project sidebar, then clicking “*Create Project Wiki*” or “*Publish code as wiki*.” You can add, edit, or organize pages directly in the web interface or by editing markdown files in a code-backed wiki repository.

3. What are best practices for structuring project documentation in a DevOps wiki?

Structure documentation with clear hierarchy and navigation: use a home/landing page, group content by feature or topic, and keep pages concise. Regularly update docs, include diagrams, and ensure links between related pages for easy discoverability.

4. How do you use Markdown syntax for documentation in Azure DevOps or GitHub wikis?

Write documentation in plain text using *Markdown* (.md) files. *Markdown* provides formatting for headings, lists, code blocks, tables, links, and images. Both *Azure DevOps* and *GitHub* wikis support standard *Markdown*, allowing rich, readable content with minimal effort.

5. What is Mermaid syntax, and how is it used in project documentation?

Mermaid is a text-based diagramming syntax supported in *Azure DevOps* and *GitHub* wikis. It allows teams to create flowcharts, sequence diagrams, and other visualizations directly within *Markdown* by embedding code blocks marked with ``mermaid`.

6. How do you embed and update diagrams in project wikis?

Embed diagrams by uploading image files and linking them using *Markdown*, or by using *Mermaid* code blocks for auto-generated diagrams. Update diagrams by editing the image file or updating the *Mermaid* code, ensuring version control and easy collaboration.

7. How do you manage permissions and access control for wikis?

In *Azure DevOps*, manage wiki permissions through *Project Settings > Permissions*. Assign read, edit, or administrative rights to users or groups. For code-based wikis, repository permissions determine access. In *GitHub*, wiki permissions follow repository settings.

8. What are best practices for keeping documentation up to date in fast-moving DevOps projects?

- Integrate documentation updates into the development workflow,
 - review docs as part of pull requests,
 - assign documentation ownership,
 - and automate reminders or checks to prevent outdated content.
-

9. How can you automate documentation updates from source control history?

Use tools or scripts to generate or update release notes, changelogs, and API documentation from Git commit history or pull request descriptions. Automate documentation builds with CI pipelines to publish latest changes to wikis.

10. What are the benefits of linking work items and documentation in Azure DevOps?

Linking work items and documentation ensures traceability, provides context for features or issues, helps onboarding, and enables faster decision-making by centralizing information relevant to code, processes, and changes.

1. Design and implement processes and communications

└ 1.3 Configure collaboration and communication

└ 1.3.2 Configure Release Documentation

1. What is release documentation, and why is it important in DevOps?
 2. How do you generate release notes automatically in Azure DevOps?
 3. What information should be included in release notes?
 4. How can you document API changes as part of a release process?
 5. How do you configure and publish release documentation using CI/CD pipelines?
 6. What are best practices for maintaining accurate release documentation?
 7. How do you manage versioning in release documentation?
 8. How can you automate API documentation generation and integration into release notes?
 9. How do you provide access to release documentation for different stakeholders?
 10. What tools are commonly used for release and API documentation in Azure DevOps and GitHub?
-

1. What is release documentation, and why is it important in DevOps?

Release documentation summarizes

- the changes,
- features,
- fixes, and known issues

included in a deployment. It provides transparency, helps stakeholders understand what's delivered, and supports

- troubleshooting,
 - compliance,
 - and audit requirements.
-

2. How do you generate release notes automatically in Azure DevOps?

Use the *Azure DevOps* Release Notes extension or pipeline tasks to extract information from work items, commits, and pull requests. Configure templates to format notes and publish them as part of the release pipeline.

3. What information should be included in release notes?

Include

- new features,
 - bug fixes,
 - improvements,
 - breaking changes,
 - known issues,
 - deployment steps,
 - and references to related work items or tickets.
-

4. How can you document API changes as part of a release process?

Integrate API documentation tools like *Swagger* or *OpenAPI* in the CI/CD pipeline to generate and update API reference docs. Link or embed updated API docs in the release notes for visibility.

5. How do you configure and publish release documentation using CI/CD pipelines?

Add tasks in the pipeline to generate documentation artifacts, such as markdown or HTML files, then publish them to a wiki, repository, or shared location. Use pipeline variables to version and label documentation with each release.

6. What are best practices for maintaining accurate release documentation?

- Update documentation as part of each deployment
 - Review release notes during pull requests
 - Automate as much as possible
 - Assign ownership for validation
 - Ensure docs reflect actual release content
-

7. How do you manage versioning in release documentation?

Organize documentation by release version (e.g., folders or tags), include version numbers in doc titles, and maintain a changelog or index for easy navigation across versions.

8. How can you automate API documentation generation and integration into release notes?

Use tools like Swagger/OpenAPI with build tasks to generate API docs on each build. Automatically attach or link these docs in release notes through pipeline scripts or templates.

9. How do you provide access to release documentation for different stakeholders?

- Publish docs to accessible platforms such as project wikis, shared repos, or internal portals.
 - Set permissions to control access based on roles (e.g., read-only or writing).
-

10. What tools are commonly used for release and API documentation in Azure DevOps and GitHub?

For release documentation:

- Azure DevOps Wiki
- Release Notes Extensions

For API documentation:

- Swagger
- OpenAPI
- DocFX

And Markdown rendered in wikis or static site generators.

1. Design and implement processes and communications

└ 1.3 Configure collaboration and communication

└ 1.3.3 Automate documentation from Git history

1. What are the benefits of automating documentation from Git history?
 2. Which tools can generate documentation from Git commit history?
 3. How do you use conventional commits to structure auto-generated documentation?
 4. What is the typical process to generate changelogs automatically from Git history?
 5. How do you configure a CI pipeline to automate changelog generation?
 6. What is the role of commit message standards in documentation automation?
 7. How can release notes be generated automatically from pull requests or commits?
 8. Which GitHub Actions are commonly used for automated documentation?
 9. How can you integrate auto-generated documentation with project wikis or Markdown files?
 10. What are best practices for maintaining accuracy in automated documentation?
-

1. What are the benefits of automating documentation from Git history?

- Ensures documentation is always up to date with code changes
 - Reduces manual effort and human error
 - Improves traceability and transparency for releases
 - Provides a reliable audit trail for changes
-

2. Which tools can generate documentation from Git commit history?

- Conventional Changelog
 - *GitHub Actions* (e.g., *release-drafter*, *changelog-generator*)
 - *GitVersion*
 - *GitLab* Release Notes
 - *Azure DevOps* Release Notes Extension
-

3. How do you use conventional commits to structure auto-generated documentation?

- Enforce commit messages to follow the Conventional Commits format (e.g., feat:, fix:, docs:)
 - Tools parse commit messages to categorize and format changelogs automatically
 - Enables automation tools to extract features, fixes, and breaking changes for documentation
-

4. What is the typical process to generate changelogs automatically from Git history?

- Enforce commit message standards (e.g., Conventional Commits)
 - Use a changelog tool (e.g., *conventional-changelog*) in CI or manually
 - Tool scans Git history since last tag/release
 - Generates or updates changelog file with categorized changes
-

5. How do you configure a CI pipeline to automate changelog generation?

- Add a changelog generation step (using tools like *conventional-changelog*, *release-drafter*) in your CI workflow (*GitHub Actions*, *Azure Pipelines*, etc.)
- Trigger the step on events like merges to main or new release tags
- Output or commit the changelog file as an artifact or into the repo

6. What is the role of commit message standards in documentation automation?

- Commit message standards enable automated tools to identify the type and scope of changes
 - Structured messages allow accurate and meaningful release notes and changelogs
 - Inconsistent messages lead to incomplete or misleading documentation
-

7. How can release notes be generated automatically from pull requests or commits?

- Use tools (e.g., release-drafter, *GitHub* release notes) that scan merged pull requests or commits
 - Configure templates to map PR titles or commit messages to release note sections
 - Trigger note generation on release/tag creation events in CI
-

8. Which GitHub Actions are commonly used for automated documentation?

- release-drafter/release-drafter
 - conventional-changelog/standard-version
-

9. How can you integrate auto-generated documentation with project wikis or Markdown files?

- Output changelog or release notes to a *Markdown* file (e.g., CHANGELOG.md)
 - Commit the file to the repo so it appears in project docs
 - Use CI to sync the content into wiki pages or documentation sites
-

10. What are best practices for maintaining accuracy in automated documentation?

- Enforce strict commit and PR message conventions
- Review generated docs as part of the release process
- Automate generation as part of CI for consistency
- Regularly update automation tooling and templates

1. Design and implement processes and communications

└ 1.3 Configure collaboration and communication

└ 1.3.4 Integrate using webhooks, GitHub/Azure Boards, Teams

1. What is a webhook and how is it used in DevOps integrations?
 2. How do you configure a webhook in GitHub or Azure DevOps?
 3. What events can trigger webhooks in GitHub and Azure DevOps?
 4. How do you integrate GitHub with Azure Boards?
 5. What information can be synchronized between GitHub and Azure Boards?
 6. How do you integrate Azure DevOps or GitHub with Microsoft Teams?
 7. What are best practices for managing webhook security?
 8. How can you filter or restrict which events trigger integrations?
 9. How do you monitor or troubleshoot webhook and integration delivery?
 10. What are common use cases for combining these integrations in DevOps workflows?
-

1. What is a webhook and how is it used in DevOps integrations?

A webhook is an HTTP callback that notifies external services about specific events (e.g., pushes, PRs). In DevOps, webhooks automate integrations, trigger workflows, and synchronize data across systems.

2. How do you configure a webhook in GitHub or Azure DevOps?

In *GitHub*: Go to *repo Settings* → *Webhooks* → *Add webhook* → *Enter payload URL*, events, and secret.

In *Azure DevOps*: *Project Settings* → *Service Hooks* → *Create Subscription* → *Select service and events*.

3. What events can trigger webhooks in GitHub and Azure DevOps?

Common triggers: push, pull request, release, issue, build completion, work item updated, code review events.

4. How do you integrate GitHub with Azure Boards?

Install *Azure Boards GitHub App* in repo(s), connect organization, link commits/PRs with work items using syntax (e.g., AB#123), and enable automatic updates.

5. What information can be synchronized between GitHub and Azure Boards?

PRs, commits, and linked issues can update *Azure Boards* work items (state, comments, links). Work items can reflect code changes.

6. How do you integrate Azure DevOps or GitHub with Microsoft Teams?

Use the *Azure DevOps* or *GitHub Teams* app. Configure connectors or subscriptions to post updates on builds, PRs, or work items to Teams channels.

7. What are best practices for managing webhook security?

Use HTTPS URLs, validate payload signatures (e.g., X-Hub-Signature), keep secrets confidential, and limit webhook permissions to only required events.

8. How can you filter or restrict which events trigger integrations?

Configure only specific event types in webhook or service hook settings. Use filters (e.g., branch, file path) and conditions in automation workflows.

9. How do you monitor or troubleshoot webhook and integration delivery?

Check webhook delivery logs in *GitHub/Azure DevOps*.

Use

- Status codes
- Request/response details
- Set up retry policies for failures

What are common use cases for combining these integrations in DevOps workflows?

1. Automated notifications (*Teams*)
1. Linking code changes to work items (*Azure Boards*)
2. Triggering CI/CD on PR/merge
3. Automating release processes
4. Synchronizing incident response

2. Source control strategy

└ 2.1 Branching and workflows

└ 2.1.1 Branch strategies: trunk-based, feature, release

-
1. What are the core differences between trunk-based, feature, and release branch strategies?
 2. When should you use trunk-based development in a DevOps project?
 3. What are the benefits and risks of using long-lived feature branches?
 4. How do you implement a release branch strategy in Azure DevOps or GitHub?
 5. How do branch strategies impact CI/CD pipeline design?
 6. What policies should be applied to branches for quality and security?
 7. How do you manage hotfixes in each branch strategy?
 8. What is the role of pull requests in trunk-based vs. feature branch workflows?
 9. How do you handle versioning and tagging in release branch strategies?
 10. What are best practices for merging and resolving conflicts across branch strategies?
-

1. What are the core differences between trunk-based, feature, and release branch strategies?

- Trunk-based uses a single main branch with frequent merges
 - Feature branching uses short-lived branches for each feature
 - Release branching creates dedicated branches for release preparation and hotfixes
-

2. When should you use trunk-based development in a DevOps project?

Use trunk-based

- when you require rapid delivery,
 - frequent integration,
 - minimal merge conflicts,
 - and a strong focus on CI/CD automation.
-

3. What are the benefits and risks of using long-lived feature branches?

Benefits:

- Isolated development
- Reduced integration risk per feature

Risks:

- Increased merge conflicts
 - Stale code
 - Delayed integration
 - Harder CI/CD automation
-

4. How do you implement a release branch strategy in Azure DevOps or GitHub?

1. Create a release branch from main
2. Stabilize code
3. Apply bug fixes or hotfixes
4. Merge critical changes back to main and other relevant branches
5. Use branch policies for quality gates

5. How do branch strategies impact CI/CD pipeline design?

Branch strategies define triggers for builds, tests, and deployments. For example, *main/trunk* triggers production builds; feature branches may trigger only CI builds; release branches may have custom deployment pipelines.

6. What policies should be applied to branches for quality and security?

Require pull request reviews, enforce status checks, restrict direct pushes, enable branch protection, and mandate code scanning for critical branches.

7. How do you manage hotfixes in each branch strategy?

- Trunk-based: commit hotfix directly to main, deploy, then merge with ongoing work.
 - Feature: Apply hotfix to main, rebase feature branches if needed.
 - Release: Apply hotfix to release branch and main, then merge changes as needed.
-

8. What is the role of pull requests in trunk-based vs. feature branch workflows?

- In trunk-based, PRs are typically short-lived and merged quickly.
 - In feature branch workflows, PRs are used to review isolated features and may be open longer, increasing merge complexity.
-

9. How do you handle versioning and tagging in release branch strategies?

- Use tags for releases (e.g., v1.0.0) on release branches;
 - Increment version numbers with each release or hotfix;
 - Track production deployments via tags.
-

10. What are best practices for merging and resolving conflicts across branch strategies?

- Integrate frequently to main
- Rebase instead of merge where possible
- Resolve conflicts early
- Communicate changes
- Automate conflict detection via CI pipelines

2. Source control strategy

└ 2.1 Branching and workflows

└ 2.1.2 Pull request workflows with policies

-
1. What is a pull request (PR) workflow and why is it important in DevOps?
 2. How do you enforce branch policies for pull requests in Azure DevOps and GitHub?
 3. What types of status checks can be required for a PR?
 4. How can you require code review approvals before merging a PR?
 5. How do you prevent direct pushes to protected branches?
 6. What are best practices for PR size and review process?
 7. How do you automate checks and enforce policy compliance in PRs?
 8. How can you enforce security and compliance checks in PR workflows?
 9. What is auto-complete or auto-merge in pull request workflows?
 10. How do you configure and use code owner policies in PR workflows?
-

1. What is a pull request (PR) workflow and why is it important in DevOps?

A PR workflow is a structured process to review, validate, and merge code changes. It ensures code quality, security, and team collaboration before integration.

2. How do you enforce branch policies for pull requests in Azure DevOps and GitHub?

Configure branch protection rules (*GitHub*) or branch policies (*Azure DevOps*) to require

- PR reviews,
 - status checks,
 - and other controls before merging.
-

3. What types of status checks can be required for a PR?

Common checks include

- successful CI builds,
 - code coverage thresholds,
 - security scans,
 - and passing automated tests.
-

4. How can you require code review approvals before merging a PR?

Set branch protection to

- require one or more reviewer approvals,
 - optionally require review from code owners or specific teams.
-

5. How do you prevent direct pushes to protected branches?

Enable branch protection rules or policies that restrict who can push and require all changes to go through PRs.

6. What are best practices for PR size and review process?

- Keep PRs small and focused,
- include clear descriptions,
- require meaningful reviews,
- and use checklists to ensure standards.

7. How do you automate checks and enforce policy compliance in PRs?

- Integrate CI/CD pipelines,
 - configure required status checks,
 - use automated code analysis,
 - and enforce policies as blocking conditions for merging.
-

8. How can you enforce security and compliance checks in PR workflows?

Add automated security scanners (e.g., for secrets, dependencies),
require results as passing status checks before merge.

9. What is auto-complete or auto-merge in pull request workflows?

It allows PRs to be merged automatically when all policies and checks are satisfied, reducing manual intervention and wait time.

10. How do you configure and use code owner policies in PR workflows?

Define a CODEOWNERS file specifying responsible individuals or teams for files/folders. Require code owner review in branch protection settings for critical code areas.

2. Source control strategy

└ 2.1 Branching and workflows

└ 2.1.3 Branch protections and merge restrictions

-
1. What are branch protection rules and why are they critical in DevOps workflows?
 2. How do you configure branch protection in GitHub and Azure DevOps?
 3. What types of merge restrictions can be enforced using branch protection policies?
 4. How can you require status checks before allowing merges?
 5. How do you restrict who can push or merge to protected branches?
 6. What is the purpose of requiring signed commits or commit signatures?
 7. How do you enforce code owner approvals in branch protection?
 8. How can you enforce linear history and prevent force-pushes?
 9. What is the impact of branch protection on CI/CD automation?
 10. What best practices should be followed for branch protections and merge restrictions?
-

1. What are branch protection rules and why are they critical in DevOps workflows?

Branch protection rules enforce

- required checks,
- reviews,
- and restrictions on branches,

preventing unauthorized or unsafe changes and ensuring code quality and stability.

2. How do you configure branch protection in GitHub and Azure DevOps?

In *GitHub*, go to *Repository Settings > Branches > Add rule*. In *Azure DevOps*, navigate to *Project Settings > Repositories > Branches > Add policy* to the desired branch.

3. What types of merge restrictions can be enforced using branch protection policies?

You can require

- pull request reviews,
 - passing status checks,
 - code owner approval,
 - no unresolved conversations,
 - and restrict merges to certain users or teams.
-

4. How can you require status checks before allowing merges?

Set required status checks in branch protection so PRs can only merge if defined CI jobs (build, test, security scan) succeed.

5. How do you restrict who can push or merge to protected branches?

Configure branch protection or security policies to allow pushes or merges only from specified users, teams, or service accounts.

6. What is the purpose of requiring signed commits or commit signatures?

Requiring signed commits verifies the identity of commit authors, helping prevent unauthorized or untraceable changes.

7. How do you enforce code owner approvals in branch protection?

Define a CODEOWNERS file in the repo and require code owner approval via branch protection settings for changes to specific files/paths.

8. How can you enforce linear history and prevent force-pushes?

Enable "*Require linear history*" and "*Prevent force pushes*" options in branch protection to maintain a clean commit history and prevent data loss.

9. What is the impact of branch protection on CI/CD automation?

Branch protection ensures all code passing through CI/CD meets quality gates, but may require automation accounts or bots to be explicitly permitted for merges or status check reporting.

10. What best practices should be followed for branch protections and merge restrictions?

- Apply protections to main/release branches
- Keep required checks relevant
- Regularly review permissions
- Require at least one reviewer
- Avoid over-restriction that blocks productivity

2. Source control strategy

└ 2.2 Repository Configuration and Maintenance

└ 2.2.1 Manage large Files (GIT LFS, git-fat)

1. What is Git LFS and why is it needed in source control?
 2. How does Git LFS work under the hood?
 3. What types of files should be managed using Git LFS?
 4. How do you install and initialize Git LFS in a repository?
 5. How do you track files with Git LFS and what is the basic workflow?
 6. How does storage and bandwidth quota work with Git LFS?
 7. What are key differences between Git LFS and git-fat?
 8. How do you migrate existing large files in a repository to Git LFS?
 9. What are best practices for managing large files in Azure DevOps or GitHub using LFS?
 10. How do you handle permissions and security for large files stored in Git LFS?
-

1. What is Git LFS and why is it needed in source control?

Git LFS (Large File Storage) is an extension that replaces large files (like binaries, media, datasets) in Git with lightweight pointers, storing the actual files in external storage. It's needed because Git is inefficient with large or frequently changing binary files.

2. How does Git LFS work under the hood?

When you commit a tracked large file, *Git LFS* stores a pointer in the repo and uploads the actual content to a remote LFS storage. During checkout, LFS downloads the actual files on demand.

3. What types of files should be managed using Git LFS?

Large binary files, media assets (e.g., images, videos), archives, datasets, or any files that are large or change frequently, and are not efficiently diffed or compressed by Git.

4. How do you install and initialize Git LFS in a repository?

Install Git LFS with:

```
git lfs install
```

Then track file types:

```
git lfs track "*.psd"
```

Commit the `.gitattributes` file and use Git as normal.

5. How do you track files with Git LFS and what is the basic workflow?

Track file patterns with *git lfs track*, add files, commit, and push as usual. LFS replaces tracked files with pointers in the repo and uploads file contents to LFS storage during push.

6. How does storage and bandwidth quota work with Git LFS?

Git LFS hosting providers (e.g., *GitHub*, *Azure DevOps*) enforce storage and bandwidth limits. Exceeding quota can block uploads or downloads until more space or bandwidth is purchased or usage is reduced.

7. What are key differences between Git LFS and git-fat?

Git LFS is widely supported, natively integrated with *GitHub*/*Azure DevOps*, and stores large files externally. *git-fat* is older, works with custom backends, and is less integrated into cloud providers.

8. How do you migrate existing large files in a repository to Git LFS?

Use `git lfs migrate import --include="*.filetype"` to convert files to LFS tracking, rewrite history as needed, and force-push updated branches. Always back up before migrating.

9. What are best practices for managing large files in Azure DevOps or GitHub using LFS?

- Track only necessary files,
 - use `.gitattributes` to specify file patterns,
 - monitor LFS quota,
 - educate contributors,
 - and avoid committing large files directly to Git.
-

10. How do you handle permissions and security for large files stored in Git LFS?

Permissions follow the repository's access control (*GitHub/Azure DevOps*).

To protect sensitive files,

- restrict repo access,
- use branch protections,
- and regularly audit who can push/pull LFS content.

2. Source control strategy

└ 2.2 Repository Configuration and Maintenance

└ 2.2.2 Optimize scaling

1. What challenges occur as Git repositories grow in size and activity?
 2. What is Scalar and how does it help with large repositories?
 3. How do partial clones and sparse-checkout help optimize scaling?
 4. How can cross-repository sharing reduce duplication and improve efficiency?
 5. What are best practices for structuring large monorepos?
 6. How do you monitor and maintain repository performance at scale?
 7. How do you clean up and archive old branches and tags efficiently?
 8. What strategies exist to optimize clone and fetch performance in large repos?
 9. How does Azure DevOps or GitHub handle scaling for large enterprise repositories?
 10. What tools or commands are used to analyze and reduce repository size?
-

1. What challenges occur as Git repositories grow in size and activity?

Large repos can cause slow clone, fetch, and checkout operations, increased storage, network costs, and degraded performance for CI/CD and contributors.

2. What is Scalar and how does it help with large repositories?

Scalar is a Microsoft tool that optimizes very large monorepos by improving Git's handling of large working trees, accelerating commands like status and checkout, and enabling virtualized file systems.

3. How do partial clones and sparse-checkout help optimize scaling?

Partial clones allow downloading only required objects. Sparse-checkout enables working with a subset of the repo, reducing disk usage and improving performance.

4. How can cross-repository sharing reduce duplication and improve efficiency?

Cross-repo sharing (submodules, subtrees, or package feeds) allows teams to reuse code/assets across projects without duplicating large files or code, minimizing storage and bandwidth usage.

5. What are best practices for structuring large monorepos?

- Organize by logical boundaries
 - Use clear folder structures
 - Minimize large binaries
 - Automate code ownership
 - Use sparse-checkout for focused workflows
-

6. How do you monitor and maintain repository performance at scale?

Regularly review repository size, commit history, and object count. Use monitoring tools and Git analytics to spot performance bottlenecks and take action early.

7. How do you clean up and archive old branches and tags efficiently?

Use `git branch -d/-D` for branches, `git tag -d` for tags, and prune remote references. Archive or delete stale branches and tags periodically to keep the repo clean.

8. What strategies exist to optimize clone and fetch performance in large repos?

- Use shallow clones (`git clone --depth`),
- partial clones,
- sparse-checkout,
- and avoid cloning unnecessary history or content.

9. How does Azure DevOps or GitHub handle scaling for large enterprise repositories?

Both platforms offer high-availability storage, optimized network delivery (e.g., CDN), *LFS support*, *Scalar*, and monitoring for large repos. *GitHub* also supports monorepo scaling features.

10. What tools or commands are used to analyze and reduce repository size?

- *git-sizer*
- `git count-objects`
- `git gc`
- `git filter-repo`

help measure, clean, and reduce repo size. Remove large files or history as needed for optimization.

2. Source control strategy

└ 2.2 Repository Configuration and Maintenance

└ 2.2.3 Configure permissions, tags

1. What are the permission levels available in Azure Repos and GitHub repositories?
 2. How do you assign repository permissions in Azure DevOps?
 3. How are permissions managed in GitHub (users, teams, roles)?
 4. What is the difference between repository, branch, and object-level permissions?
 5. How do you use branch protection rules to restrict actions?
 6. What are best practices for granting repository permissions?
 7. How do you configure repository tags in Git and Azure DevOps?
 8. What are the use cases for tags in source control?
 9. How do you protect or restrict tag creation and deletion?
 10. How can you audit or review permission and tag changes in repositories?
-

1. What are the permission levels available in Azure Repos and GitHub repositories?

Azure Repos:

- Reader
- Contributor
- Branch Creator
- Project Administrator
- Custom

GitHub:

- Read
 - Triage
 - Write
 - Maintain
 - Admin
-

2. How do you assign repository permissions in Azure DevOps?

Go to *Project Settings* → *Repositories* → *Select repository* → *Security* → *Add users/groups* → *Assign roles or explicit permissions*.

3. How are permissions managed in GitHub (users, teams, roles)?

Assign roles at repository or organization level. Use teams to group users. Set roles for users or teams via repository *Settings* → *Manage access*.

4. What is the difference between repository, branch, and object-level permissions?

Repository: Control access to entire repo.

Branch: Restrict actions (push, merge) on specific branches.

Object: Limit access to individual files/paths (rare, typically via code owners).

5. How do you use branch protection rules to restrict actions?

In *Settings* → *Branches*, add branch protection rules (e.g., require PR reviews, restrict push, enforce status checks, prevent force pushes or deletions).

6. What are best practices for granting repository permissions?

Grant least privilege required, use teams/groups, avoid Admin unless necessary, periodically review access.

7. How do you configure repository tags in Git and Azure DevOps?

Create:

`git tag v1.0 (local),`

`git push origin v1.0 (remote).`

Azure DevOps: Use web UI or Git CLI; tags sync with repository.

8. What are the use cases for tags in source control?

Mark

- releases,
 - important milestones,
 - stable builds,
 - deployment points.
-

9. How do you protect or restrict tag creation and deletion?

GitHub: Use branch protection for refs/tags/* with GitHub Enterprise.

Azure DevOps: Restrict who can create/delete tags via repository security settings.

10. How can you audit or review permission and tag changes in repositories?

Azure DevOps: Use Audit Logs (Organization settings).

GitHub: Use Audit Log in Enterprise/Organization settings; review git history for tag changes via `git log --tags`.

2. Source control strategy

└ 2.2 Repository Configuration and Maintenance

└ 2.2.4 Recover/remove data with Git commands

1. How do you recover a deleted branch in Git?
 2. How can you recover a deleted commit?
 3. What command is used to undo local changes in a working directory?
 4. How do you permanently remove a file from a Git repository's history?
 5. How do you use git revert and git reset to undo changes?
 6. How can you recover a file deleted in a previous commit?
 7. What is the difference between git reflog and git log for recovery?
 8. How do you use git cherry-pick to restore changes from another branch?
 9. How do you remove sensitive data (e.g., passwords) from commit history?
 10. How can you prevent accidental data loss when cleaning up branches or commits?
-

1. How do you recover a deleted branch in Git?

Find the branch's last commit hash with git reflog then recreate it:

```
git checkout -b branch_name <commit_hash>
```

2. How can you recover a deleted commit?

Use git reflog to locate the commit hash, then checkout or branch from it:

```
git checkout <commit_hash> OR git branch <new_branch> <commit_hash>
```

3. What command is used to undo local changes in a working directory?

git checkout -- <file> (undo changes in file) or

git restore <file> (recommended in newer Git).

4. How do you permanently remove a file from a Git repository's history?

Use

- git filter-branch (legacy)
- git filter-repo (preferred)

to rewrite history and remove the file, then force-push:

```
git filter-repo --path <file> --invert-paths
```

5. How do you use git revert and git reset to undo changes?

git revert <commit> creates a new commit that undoes changes.

git reset --hard <commit> moves HEAD and branch to a specific commit, discarding later commits (local use).

6. How can you recover a file deleted in a previous commit?

Find the commit where the file existed:

```
git checkout <commit_hash> -- <file>
```

7. What is the difference between git reflog and git log for recovery?

git log shows the commit history;

git reflog records all HEAD movements (including deleted branches/commits), useful for recovery.

8. How do you use git cherry-pick to restore changes from another branch?

git cherry-pick <commit_hash> applies a specific commit from another branch onto the current branch.

9. How do you remove sensitive data (e.g., passwords) from commit history?

Use git filter-repo (or filter-branch) to remove secrets, then force-push to rewrite remote history.

10. How can you prevent accidental data loss when cleaning up branches or commits?

- Always back up with git branch <backup> <ref> or create tags before destructive operations
- Review with git reflog if needed.

2. Source control strategy

└ 2.3 Package Management

└ 2.3.1 Tools: Github Packages, Azure Artifacts

-
1. What are GitHub Packages and Azure Artifacts, and how do they differ?
 2. How do you publish a package to GitHub Packages?
 3. How do you publish a package to Azure Artifacts?
 4. How do you authenticate to GitHub Packages and Azure Artifacts?
 5. What package types are supported by GitHub Packages and Azure Artifacts?
 6. How do you configure a package feed in Azure Artifacts?
 7. How do you use a package from GitHub Packages in a CI/CD pipeline?
 8. How do you control access and permissions for packages in GitHub Packages and Azure Artifacts?
 9. How do you manage package retention and cleanup in Azure Artifacts?
 10. How do you configure upstream sources in Azure Artifacts
-

1. What are GitHub Packages and Azure Artifacts, and how do they differ?

- *GitHub Packages* is a package hosting service fully integrated with *GitHub* repositories, supporting multiple package types and commonly used for GitHub-based projects.
 - *Azure Artifacts* is part of Azure DevOps, offering integrated package feeds for teams, granular permissions, and support for upstream sources.
 - Key difference: Azure Artifacts is tightly integrated with Azure DevOps pipelines and boards, while GitHub Packages is GitHub-centric.
-

2. How do you publish a package to GitHub Packages?

- Authenticate with a personal access token or *GitHub Actions*,
 - configure your package manager with your *GitHub* repository URL,
 - and use the package manager's publish command (e.g., `npm publish`, `dotnet nuget push`, `mvn deploy`).
-

3. How do you publish a package to Azure Artifacts?

- Authenticate with *Azure Artifacts* using a personal access token or *Azure CLI*,
 - add the *Azure Artifacts* feed as a source to your package manager,
 - and use the relevant publish command (e.g., `dotnet nuget push`, `npm publish`, `twine upload` for Python).
-

4. How do you authenticate to GitHub Packages and Azure Artifacts?

- For *GitHub Packages*, use a personal access token with `write:packages` scope or *GitHub Actions*' built-in `GITHUB_TOKEN`.
 - For *Azure Artifacts*, authenticate with a personal access token or *Azure CLI* (`az artifacts universal publish/login`), or with build pipeline credentials.
-

5. What package types are supported by GitHub Packages and Azure Artifacts?

- *GitHub Packages* supports npm, Maven, NuGet, Docker, RubyGems, and Apache Ivy.
 - *Azure Artifacts* supports npm, NuGet, Maven, Python, and Universal Packages.
-

6. How do you configure a package feed in Azure Artifacts?

In *Azure DevOps*, go to Artifacts > New Feed, name the feed, set visibility (organization/private), optionally add upstream sources, and save. Configure your package manager (e.g., `nuget.config`, `.npmrc`) to use the feed's URL.

7. How do you use a package from GitHub Packages in a CI/CD pipeline?

1. Add authentication steps to the pipeline (set up `GITHUB_TOKEN` or PAT)
 2. Configure the package manager with the *GitHub Packages* registry URL
 3. Install the package as usual in your pipeline steps
-

8. How do you control access and permissions for packages in GitHub Packages and Azure Artifacts?

- In *GitHub Packages*, permissions are based on repository access and repository roles.
 - In *Azure Artifacts*, use feed-level permissions to grant/restrict to users, groups, or pipelines.
-

9. How do you manage package retention and cleanup in Azure Artifacts?

Configure retention policies in *Azure Artifacts* to automatically delete older or unlisted package versions. Go to *Artifacts > Feed > Settings > Retention policies*, and define rules for how many versions to keep and for how long.

10. How do you configure upstream sources in Azure Artifacts?

In the feed settings, add upstream sources to connect your *Azure Artifacts* feed to public repositories (e.g., `npmjs.com`, `nuget.org`) or other *Azure Artifacts* feeds. This allows packages from upstream sources to be cached and consumed via your feed.

2. Source control strategy

└ 2.3 Package Management

└ 2.3.2 Configure feeds/views for packages

-
1. What is a feed in Azure Artifacts, and what is its purpose?
 2. How do you create a feed in Azure Artifacts?
 3. What are views in Azure Artifacts, and why are they used?
 4. How do you create and manage views in Azure Artifacts?
 5. How do you promote a package to a different view?
 6. How do you configure a package manager to use a specific feed or view?
 7. How do you control access to feeds and views?
 8. What are the default views in Azure Artifacts and their typical use?
 9. How can you connect external sources (e.g., NuGet.org) to your feed?
 10. How do you clean up or delete feeds and views?
-

1. What is a feed in Azure Artifacts, and what is its purpose?

A feed in *Azure Artifacts* is a repository for storing and sharing packages (e.g., NuGet, npm, Maven) within your organization or projects. It enables versioned, centralized package management for CI/CD.

2. How do you create a feed in Azure Artifacts?

Navigate to *Azure DevOps* > *Artifacts* > *New Feed*, provide a name, set visibility (organization or project), configure permissions, and create the feed.

3. What are views in Azure Artifacts, and why are they used?

Views are logical partitions within a feed used to separate packages by quality or environment (e.g., development, prerelease, release). They help control which package versions are available for different stages of the pipeline.

4. How do you create and manage views in Azure Artifacts?

By default, feeds include predefined views (@local, @prerelease, @release). You can create custom views in the feed settings and manage them by promoting or filtering packages.

5. How do you promote a package to a different view?

In *Azure DevOps*, go to *Artifacts* > *Feed* > *Packages*, select a package version, choose Promote, and select the target view (e.g., promote from @prerelease to @release).

6. How do you configure a package manager to use a specific feed or view?

Update your package manager config (e.g., `nuget.config`, `.npmrc`) with the feed URL and append the view (e.g., `https://pkgs.dev.azure.com/org/project/_packaging/feed/view/npm/registry/`). Authenticate as required.

7. How do you control access to feeds and views?

Manage access using *Azure DevOps* security groups and feed permissions. Grant users or pipelines access at the feed or view level with roles like Reader, Contributor, or Owner.

8. What are the default views in Azure Artifacts and their typical use?

- @local (default, all packages)
- @prerelease (pre-production/testing)
- @release (production-approved)

Use these to stage packages through different lifecycle phases.

9. How can you connect external sources (e.g., NuGet.org) to your feed?

Add upstream sources in feed settings to cache packages from public repositories like NuGet.org or npmjs.com, making them available in your feed.

10. How do you clean up or delete feeds and views?

To delete a feed, go to *Azure DevOps > Artifacts > Manage Feeds*, select the feed, and choose Delete. Views can be managed or deleted from feed settings if not default views; packages can also be deleted or unlisted as needed.

2. Source control strategy

└ 2.3 Package Management

└ 2.3.3 Versioning strategies: SemVer, CalVer

-
1. What is Semantic Versioning (SemVer) and what are its components?
 2. What is Calendar Versioning (CalVer) and how is it structured?
 3. When should you use SemVer versus CalVer?
 4. How do pre-release versions and build metadata work in SemVer?
 5. How can you implement SemVer in CI/CD pipelines?
 6. How do you implement CalVer in package versioning?
 7. How does versioning affect dependency management?
 8. How can you enforce versioning rules in Azure DevOps or GitHub pipelines?
 9. What are best practices for updating package versions?
 10. How can you handle breaking changes in a versioning strategy?
-

1. What is Semantic Versioning (SemVer) and what are its components?

SemVer is a versioning scheme with three numeric components: MAJOR.MINOR.PATCH (e.g., 2.1.3).

- MAJOR: Breaking changes
 - MINOR: Backward-compatible features
 - PATCH: Backward-compatible bug fixes
-

2. What is Calendar Versioning (CalVer) and how is it structured?

CalVer is a date-based versioning scheme, typically using formats like YYYY.MM or YY.MM.DD (e.g., 2024.06 or 24.06.05), representing the release date.

3. When should you use SemVer versus CalVer?

- Use *SemVer* when precise tracking of breaking and backward-compatible changes is needed.
 - Use *CalVer* where the release schedule is more important than feature tracking.
-

4. How do pre-release versions and build metadata work in SemVer?

- Pre-release versions use a hyphen and identifier (e.g., 1.2.0-beta).
 - Build metadata uses a plus sign (e.g., 1.2.0+001).
 - Pre-release versions are lower precedence than final releases.
-

5. How can you implement SemVer in CI/CD pipelines?

Automate version bumping based on commit messages or tags, validate version format, and update package files in the build pipeline before publishing artifacts.

6. How do you implement CalVer in package versioning?

Set the version number in build scripts or pipeline templates to the current date (e.g., using a variable or script to generate YYYY.MM.DD) during the build process.

7. How does versioning affect dependency management?

Correct versioning allows tools to resolve compatible dependencies, prevent breaking changes, and manage upgrades/downgrades reliably across services and packages.

8. How can you enforce versioning rules in Azure DevOps or GitHub pipelines?

- Add validation steps to CI/CD pipelines that check version format
 - Enforce increment rules
 - Reject builds if versioning is not compliant (using custom scripts or tasks)
-

9. What are best practices for updating package versions?

- Update versions before publishing,
 - Follow a consistent versioning policy,
 - Document changes,
 - Communicate breaking changes clearly.
 - Automate version updates in pipelines when possible.
-

10. How can you handle breaking changes in a versioning strategy?

- Increment the MAJOR version in SemVer, document breaking changes in release notes, and communicate impacts to consumers.
- In CalVer, document breaking changes in changelogs since the version number alone doesn't indicate compatibility.

2. Source control strategy

└ 2.3 Package Management

└ 2.3.4 Version Pipeline Artifacts

1. What are pipeline artifacts in Azure DevOps and GitHub Actions?
 2. Why is it important to version pipeline artifacts?
 3. How do you set a version for a pipeline artifact in Azure DevOps?
 4. How do you version artifacts in GitHub Actions?
 5. What strategies can you use for artifact versioning (e.g., SemVer, build number, commit SHA)?
 6. How do you ensure traceability between artifacts and source code versions?
 7. How can you automate artifact versioning in a CI/CD pipeline?
 8. How do you store and retrieve versioned artifacts in Azure Pipelines?
 9. How do you promote or consume specific versions of pipeline artifacts?
 10. What are best practices for artifact retention and cleanup?
-

1. What are pipeline artifacts in Azure DevOps and GitHub Actions?

Pipeline artifacts are files or packages produced by CI/CD builds and stored for use in later stages (e.g., deployment, testing) or for sharing across jobs or pipelines.

2. Why is it important to version pipeline artifacts?

Versioning ensures traceability, reproducibility, and prevents conflicts or accidental overwrites. It allows teams to identify which build or commit produced a given artifact.

3. How do you set a version for a pipeline artifact in Azure DevOps?

Define a variable (e.g., \$(Build.BuildNumber) or custom SemVer/CalVer) and use it as the artifact's name or version in PublishPipelineArtifact Or PublishBuildArtifacts tasks.

4. How do you version artifacts in GitHub Actions?

Set a version tag using

- environment variables,
- Git commit SHA,
- or tags,

and include it in the artifact name when using the actions/upload-artifact action.

5. What strategies can you use for artifact versioning (e.g., SemVer, build number, commit SHA)?

Common strategies:

- SemVer: Aligns with application/package version
 - Build number: Unique CI/CD pipeline run
 - Commit SHA: Direct link to source version
 - CalVer: Date-based versioning
-

6. How do you ensure traceability between artifacts and source code versions?

Include commit SHA, build number, or version info in the artifact metadata or name. Store artifact metadata with references to build and source details.

7. How can you automate artifact versioning in a CI/CD pipeline?

Use

- pipeline variables,
- scripts,
- or tasks

to generate and assign a version to artifacts automatically during build or release.

8. How do you store and retrieve versioned artifacts in Azure Pipelines?

Artifacts are published to *Azure DevOps (Artifacts > Pipeline Artifacts/Build Artifacts)*. Use `DownloadPipelineArtifact` OR `DownloadBuildArtifacts` tasks with the specified version or build number.

9. How do you promote or consume specific versions of pipeline artifacts?

Reference the desired artifact version in pipeline tasks or release stages using

- tags,
- build numbers,
- or metadata.

Promotion may involve moving artifacts to a separate feed or environment.

10. What are best practices for artifact retention and cleanup?

Configure retention policies

- to automatically delete old or unused artifacts,
- keep only recent or released versions,
- and prevent storage bloat.
- Document retention settings for compliance.

3. Build and release pipelines

└ 3.1 Package management and testing strategy

└ 3.1.1 Quality/release gates (security, governance)

1. What are quality gates and release gates in Azure Pipelines?
 2. How do you configure a quality gate in a pipeline?
 3. What types of policies or checks are commonly used as gates?
 4. How do you implement security scanning (e.g., SCA, SAST) as a pipeline gate?
 5. How do you enforce governance and compliance requirements in a release pipeline?
 6. What is an approval gate and how is it set up?
 7. How can you integrate external tools (e.g., SonarCloud, WhiteSource, Checkmarx) as gates?
 8. What happens when a gate fails during a release?
 9. How do you audit and report on gate results for compliance?
 10. What are best practices for designing effective release and quality gates?
-

1. What are quality gates and release gates in Azure Pipelines?

Quality gates and release gates are automated checks within *Azure Pipelines* that block progression of builds or releases until specified conditions are met.

- Quality gates run during the build (e.g., code coverage, test pass rate),
 - while release gates execute before or after deployment stages (e.g., approvals, external service checks).
-

2. How do you configure a quality gate in a pipeline?

Define checks or conditions (e.g., code coverage threshold, static analysis results) in the pipeline YAML or classic editor. In YAML, use the `condition` attribute for steps, or specify required checks in branch policies for PR validation.

3. What types of policies or checks are commonly used as gates?

Common gates include:

- Code coverage thresholds
 - Static code analysis (SAST)
 - Open-source dependency scanning (SCA)
 - Linting
 - Required manual approvals
 - Work item linking
 - Deployment health checks
 - External service integrations (e.g., SonarCloud)
-

4. How do you implement security scanning (e.g., SCA, SAST) as a pipeline gate?

Add tasks for security scanning tools (e.g., SonarCloudPrepare, Trivy, OWASP Dependency-Check) in the pipeline. Configure the tool to fail the build or block the release if vulnerabilities or policy violations are detected.

5. How do you enforce governance and compliance requirements in a release pipeline?

- Use environment checks and approvals,
 - integrate mandatory gate tasks (e.g., approval steps, sign-off requirements),
 - and ensure artifacts and deployments meet compliance criteria before advancing to production.
 - Configure auditing and tracking via Azure Pipelines environments and release history.
-

6. What is an approval gate and how is it set up?

An approval gate requires one or more authorized users to approve deployment before the pipeline can proceed. Configure this in the Azure Pipeline environment or stage by adding manual approval checks and specifying approvers in the pipeline settings.

7. How can you integrate external tools (e.g., SonarCloud, WhiteSource, Checkmarx) as gates?

Add *marketplace* tasks or custom scripts for the tool to your pipeline. Use the tool's exit codes or results to determine pass/fail status. For gated releases, configure an environment check using the tool's REST API or output.

8. What happens when a gate fails during a release?

If a gate fails, the pipeline halts at that stage. No further deployments occur until the issue is resolved and the gate is re-evaluated or manually overridden (if permitted).

9. How do you audit and report on gate results for compliance?

Gate results are logged in the pipeline run history and environment checks. Use

- Azure DevOps audit logs,
- downloadable run logs,
- and built-in reporting features

to demonstrate compliance during audits.

10. What are best practices for designing effective release and quality gates?

Use automated gates for security, quality, and compliance.

- Avoid overloading pipelines with too many manual approvals.
- Regularly review and update gate criteria to match evolving requirements.
- Integrate external scanning tools natively.
- Ensure gate failures produce clear, actionable feedback.

3. Build and release pipelines

└ 3.1 Package management and testing strategy

└ 3.1.2 Testing Strategy: Unit, Integration, Load

-
1. What are the differences between unit, integration, and load tests?
 2. How do you structure pipelines to run unit, integration, and load tests efficiently?
 3. How do you configure test tasks in Azure Pipelines or GitHub Actions?
 4. How are test results collected and visualized in Azure DevOps?
 5. What are best practices for running unit tests in CI/CD pipelines?
 6. How do you isolate and manage dependencies for integration tests?
 7. How do you implement load testing in a pipeline?
 8. What tools are commonly used for each test type in Azure DevOps?
 9. How do you handle test failures in pipelines?
 10. How do you ensure code coverage thresholds are enforced during testing?
-

1. What are the differences between unit, integration, and load tests?

- Unit tests validate isolated code units and run quickly without external dependencies.
 - Integration tests verify interactions between multiple components or services.
 - Load tests simulate real-world traffic or usage to assess performance and stability under stress.
-

2. How do you structure pipelines to run unit, integration, and load tests efficiently?

- Run unit tests early and in parallel on every commit.
 - Integration tests run after builds or deployments to test environments.
 - Load tests typically run in dedicated stages after integration tests or before production deployments to avoid resource conflicts.
-

3. How do you configure test tasks in Azure Pipelines or GitHub Actions?

Use built-in tasks (e.g., `VsTest`, `DotNetCoreCLI@2`, `PublishTestResults@2`) in *Azure Pipelines*, or actions like `actions/setup-dotnet`, `actions/upload-artifact` in *GitHub Actions*. Specify test commands and result formats in YAML steps.

4. How are test results collected and visualized in Azure DevOps?

Publish test results using the `PublishTestResults` task. Results appear in the *Azure Pipelines* Test tab with detailed logs, failure summaries, and trends across runs.

5. What are best practices for running unit tests in CI/CD pipelines?

Run all unit tests on every push and pull request. Ensure tests are isolated and fast. Fail the build if any unit test fails. Use code coverage tools to track and enforce minimum coverage levels.

6. How do you isolate and manage dependencies for integration tests?

- Use dedicated test environments, containers, or service virtualization to replicate production.
- Mock or stub external systems as needed.
- Clean up data after each test run to avoid cross-test interference.

7. How do you implement load testing in a pipeline?

Integrate load testing tools (e.g., *Azure Load Testing*, *JMeter*, *k6*) into dedicated pipeline stages. Configure thresholds for performance metrics, and fail the pipeline if critical metrics (latency, errors) exceed limits.

8. What tools are commonly used for each test type in Azure DevOps?

- Unit: *MSTest*, *NUnit*, *xUnit*, *Jest*, *pytest*
 - Integration: *Postman*, *REST Assured*, *SpecFlow*, *TestContainers*
 - Load: *Azure Load Testing*, *Apache JMeter*, *k6*, *Artillery*
-

9. How do you handle test failures in pipelines?

Fail the pipeline stage when critical tests fail.

- Configure alerts and notifications.
 - Optionally, collect logs and artifacts for debugging.
 - Triage failures quickly to maintain build quality.
-

11. How do you ensure code coverage thresholds are enforced during testing?

- Use code coverage tools (e.g., *Coverlet*, *JaCoCo*, *Istanbul*) in the pipeline.
- Set minimum coverage thresholds in pipeline definitions; fail the build if coverage falls below.

3. Build and release pipelines

└ 3.1 Package management and testing strategy

└ 3.1.3 Tests in pipelines: agents, result integration

- What is the role of build/release agents in running pipeline tests?
 - How do you select and configure agents for test execution?
 - How are test tasks integrated in Azure Pipelines and GitHub Actions?
 - How do you publish and visualize test results in Azure DevOps?
 - How do you handle parallel test execution across agents?
 - What formats are supported for test result files?
 - How do you troubleshoot test execution issues on agents?
 - How do you aggregate test results from multiple jobs or agents?
 - How are flaky tests managed and reported in pipelines?
 - What best practices ensure reliable test result integration in CI/CD?
-

1. What is the role of build/release agents in running pipeline tests?

Agents are the compute resources (VMs, containers, or hosted runners) that execute pipeline tasks, including building code, running tests, and publishing test results.

2. How do you select and configure agents for test execution?

- Choose agents based on OS, toolchain, and resource needs (e.g., MS-hosted vs. self-hosted).
 - Configure agent pools in *Azure DevOps* or runners in *GitHub Actions*.
 - Set demands and capabilities in pipeline YAML or settings.
-

3. How are test tasks integrated in Azure Pipelines and GitHub Actions?

Add built-in or custom test tasks (e.g., *VsTest*, *DotNetCoreCLI*, *PublishTestResults*) in *Azure Pipelines* YAML. In *GitHub Actions*, use setup and test actions, then upload results as artifacts.

4. How do you publish and visualize test results in Azure DevOps?

Use the *PublishTestResults@2* task to upload results in supported formats (*JUnit*, *TRX*). View test summaries, trends, and individual failure logs in the *Azure Pipelines* Test tab.

5. How do you handle parallel test execution across agents?

- Enable parallel jobs and test splitting in pipeline settings.
 - Divide test suites by assemblies or test categories to distribute them across multiple agents, reducing total execution time.
-

6. What formats are supported for test result files?

Common supported formats include *JUnit*, *TRX*, *NUnit*, and *xUnit* XML. Use these formats to ensure results are properly parsed and displayed in *Azure DevOps* or *GitHub Actions*.

7. How do you troubleshoot test execution issues on agents?

- Review pipeline logs, agent diagnostic output, and test result files.
 - Check for missing dependencies, incorrect paths, or environment mismatches.
 - Use agent diagnostic commands as needed.
-

8. How do you aggregate test results from multiple jobs or agents?

Publish all test result files to the pipeline using the `PublishTestResults` task with the `mergeTestResults` option enabled. *Azure DevOps* will consolidate results for reporting.

9. How are flaky tests managed and reported in pipelines?

Flag flaky tests using test frameworks or pipeline annotations. *Azure DevOps* Test tab can highlight inconsistent tests. Regularly review and quarantine or fix unreliable tests.

10. What best practices ensure reliable test result integration in CI/CD?

- Standardize test result formats.
- Always publish results (even on failure).
- Isolate tests for reproducibility.
- Review integration logs after each run.
- Clean up environment between runs to avoid residue affecting results.

3. Build and release pipelines

└ 3.1 Package management and testing strategy

└ 3.1.4 Code coverage analysis

1. What is code coverage and why is it important in DevOps pipelines?
 2. How do you collect code coverage in Azure Pipelines?
 3. What tools are commonly used for code coverage in .NET, Java, and JavaScript projects?
 4. How do you publish and visualize code coverage reports in Azure DevOps?
 5. How do you enforce code coverage thresholds in a pipeline?
 6. How do you integrate code coverage analysis in GitHub Actions?
 7. What are best practices for interpreting code coverage metrics?
 8. How do you handle code coverage for integration and UI tests?
 9. How do you troubleshoot missing or incomplete code coverage data?
 10. How should teams use code coverage data to improve code quality?
-

1. What is code coverage and why is it important in DevOps pipelines?

Code coverage measures the percentage of source code executed by tests. It helps identify untested code, drives higher quality, and ensures changes are properly validated before deployment.

2. How do you collect code coverage in Azure Pipelines?

Add code coverage arguments to test commands (e.g., `/p:CollectCoverage=true` for .NET). Use tasks like `PublishCodeCoverageResults@1` to collect and upload coverage data for reporting.

3. What tools are commonly used for code coverage in .NET, Java, and JavaScript projects?

- .NET: *Coverlet*, *Visual Studio Code Coverage*
 - Java: *JaCoCo*, *Cobertura*
 - JavaScript: *Istanbul (nyc)*, *Jest*
-

4. How do you publish and visualize code coverage reports in Azure DevOps?

Use the `PublishCodeCoverageResults` task with supported formats (*Cobertura*, *JaCoCo*). Results are visualized in the pipeline summary and the *Code Coverage* tab for detailed breakdowns.

5. How do you enforce code coverage thresholds in a pipeline?

Configure coverage tools to fail the build if coverage drops below a set threshold (e.g., using *Coverlet's* `--threshold` option). Add these checks in pipeline YAML to enforce quality gates.

6. How do you integrate code coverage analysis in GitHub Actions?

Add coverage tools to your workflow steps, generate a coverage report, and use actions (like `codecov/codecov-action` OR `coverallsapp/github-action`) to upload results. Display coverage summary as a status check or comment.

7. What are best practices for interpreting code coverage metrics?

- Focus on critical code paths and business logic.
- High coverage does not guarantee quality; also review test depth and effectiveness.
- Investigate and reduce untested or unreachable code.

8. How do you handle code coverage for integration and UI tests?

- Instrument the application for integration/UI tests so coverage is measured across components.
 - Merge results from multiple test types for comprehensive reporting.
-

9. How do you troubleshoot missing or incomplete code coverage data?

- Check test configuration and coverage tool integration.
 - Ensure correct file paths and formats.
 - Confirm that instrumented code is being executed and that reports are properly published.
-

11. How should teams use code coverage data to improve code quality?

- Regularly review coverage trends.
- Address gaps in high-risk or critical areas.
- Use coverage as one input for quality, not as the only gate.
- Prioritize meaningful tests over achieving arbitrary coverage targets.

3. Build and release pipelines

└ 3.2 Design and implement pipelines

└ 3.2.1 Automation tools: GitHub Actions, Azure Pipelines

-
1. What are the primary differences between GitHub Actions and Azure Pipelines?
 2. How do you create and structure a workflow in GitHub Actions?
 3. How do you define and configure pipelines using YAML in Azure Pipelines?
 4. What triggers can start workflows in GitHub Actions and Azure Pipelines?
 5. How do you use runners (GitHub) and agents (Azure) in automation pipelines?
 6. How do you securely manage secrets and variables in both tools?
 7. How do you implement environment deployments and approvals?
 8. How do you integrate testing and code quality gates in automation pipelines?
 9. What are best practices for job dependencies and parallel execution?
 10. How do you monitor and troubleshoot workflow and pipeline executions?
-

1. What are the primary differences between GitHub Actions and Azure Pipelines?

- *GitHub Actions* is natively integrated with GitHub, focused on repository-centric automation using workflow files in the repo.
 - *Azure Pipelines* is part of *Azure DevOps*, supports multi-repo, project-level pipelines, offers both YAML and classic UI, and is more feature-rich for complex enterprise scenarios.
-

2. How do you create and structure a workflow in GitHub Actions?

Create a YAML file in `.github/workflows/`. Define name, on (trigger), jobs, and each job's runs-on and steps. Each step specifies actions or shell commands.

3. How do you define and configure pipelines using YAML in Azure Pipelines?

Create `azure-pipelines.yml` at the repo root. Define trigger, pool, variables, stages, jobs, and steps. YAML is used to declaratively describe the pipeline structure and logic.

4. What triggers can start workflows in GitHub Actions and Azure Pipelines?

- *GitHub Actions*: push, pull_request, schedule, workflow_dispatch (manual), environment events.
 - *Azure Pipelines*: trigger (CI on commit), pr, schedule, pipeline (other pipelines), and manual runs.
-

5. How do you use runners (GitHub) and agents (Azure) in automation pipelines?

- *GitHub Actions* uses hosted or self-hosted runners defined in runs-on.
 - *Azure Pipelines* uses Microsoft-hosted or self-hosted agents set in the pool definition.
-

6. How do you securely manage secrets and variables in both tools?

- *GitHub Actions*: Store secrets via *Repository Settings* > *Secrets* and reference as `${{ secrets.NAME }}`.
 - *Azure Pipelines*: Store secrets in Variable Groups and reference as `$(NAME)` in YAML.
-

7. How do you implement environment deployments and approvals?

- *GitHub Actions*: Define environments in workflows, set protection rules and required reviewers.
- *Azure Pipelines*: Use environments with approvals and checks; configure in the Environments tab and reference in YAML with environment.

8. How do you integrate testing and code quality gates in automation pipelines?

- *GitHub Actions*: Add test jobs/steps using testing actions; configure status checks to enforce passing tests before merging.
 - *Azure Pipelines*:
 - Add test tasks,
 - publish test results,
 - set Quality Gates with policies or extensions (e.g., SonarQube).
-

9. What are best practices for job dependencies and parallel execution?

- *GitHub Actions*: Define job dependencies with `needs`; and run jobs in parallel by default unless dependencies are set.
 - *Azure Pipelines*: Use `dependsOn` for controlling order; jobs and stages run in parallel unless dependencies are specified.
-

10. How do you monitor and troubleshoot workflow and pipeline executions?

- *GitHub Actions*:
 - Review logs in the Actions tab
 - Inspect individual job/step output
 - Use `run: echo` for debugging.
- *Azure Pipelines*:
 - Use the Runs/Logs UI
 - Download logs
 - Enable diagnostics
 - Check task-level error output for troubleshooting

3. Build and release pipelines

└ 3.2 Design and implement pipelines

└ 3.2.2 Runner/agent infra: cost, licenses, connectivity

1. What are the types of runners/agents available in GitHub Actions and Azure Pipelines?
2. What are the main cost considerations when selecting hosted vs. self-hosted runners/agents?
3. How do you select the right agent pool or runner for your workload?
4. How do you configure a self-hosted runner in GitHub Actions?
5. How do you configure a self-hosted agent in Azure Pipelines?
6. What connectivity requirements must self-hosted runners/agents meet?
7. What licensing requirements exist for runners/agents?
8. How do you secure access to self-hosted runners/agents?
9. How can you scale runners/agents for parallel job execution?
10. How do you monitor and maintain runner/agent health?

1. What are the types of runners/agents available in GitHub Actions and Azure Pipelines?

- *GitHub Actions*: *GitHub*-hosted runners and self-hosted runners.
- *Azure Pipelines*: Microsoft-hosted agents and self-hosted agents.

2. What are the main cost considerations when selecting hosted vs. self-hosted runners/agents?

Hosted runners/agents are billed per minute beyond free quotas; self-hosted options reduce per-minute costs but require you to provide and maintain the infrastructure.

3. How do you select the right agent pool or runner for your workload?

Choose hosted for ease and standard workloads; select self-hosted for custom requirements, legacy dependencies, higher performance, or cost control.

4. How do you configure a self-hosted runner in GitHub Actions?

Go to *repo or org settings* → *Actions* → *Runners* → *Add runner* → *Download* and configure agent as instructed, then start the service on the target machine.

5. How do you configure a self-hosted agent in Azure Pipelines?

Go to *Project Settings* → *Agent pools* → *Add pool/agent* → *Download* the agent package, run the config script, and register with a PAT (Personal Access Token).

6. What connectivity requirements must self-hosted runners/agents meet?

- They must have outbound internet access to *GitHub/Azure* endpoints
- Access to required build/test/deploy resources
- Any specific firewall/proxy settings applied.

7. What licensing requirements exist for runners/agents?

- *GitHub*-hosted runners are covered by *GitHub* billing. Self-hosted require appropriate OS and third-party software licenses.
- *Azure*-hosted agents are included in *Azure DevOps* plans; self-hosted agents require you to license the host OS/software.

8. How do you secure access to self-hosted runners/agents?

- Restrict network exposure
 - Run with minimum required permissions
 - Use firewalls
 - Keep systems patched
 - Store credentials/secrets securely
-

9. How can you scale runners/agents for parallel job execution?

- Increase the number of runners/agents
 - Use autoscaling solutions
 - Configure jobs/pipelines to utilize available capacity
-

10. How do you monitor and maintain runner/agent health?

- Monitor via GitHub/Azure DevOps UI
- Use logs and alerts
- Automate updates/patches
- Routinely validate connectivity and system performance

3. Build and release pipelines

└ 3.2 Design and implement pipelines

└ 3.2.3 Integration of GitHub repos with Azure Pipelines

1. What are the prerequisites for integrating GitHub repositories with Azure Pipelines?
 2. How do you connect a GitHub repository to an Azure Pipeline?
 3. What authentication methods are supported for GitHub-Azure Pipelines integration?
 4. How do you configure pipeline triggers for GitHub repository events?
 5. How do you set up build validation for GitHub pull requests in Azure Pipelines?
 6. How can you use YAML pipelines with GitHub repositories?
 7. What permissions are required for Azure Pipelines to access a GitHub repository?
 8. How are service connections managed and secured for GitHub integration?
 9. What are best practices for handling secrets in GitHub-Azure Pipelines workflows?
 10. How do you monitor and troubleshoot integration issues between GitHub and Azure Pipelines?
-

1. What are the prerequisites for integrating GitHub repositories with Azure Pipelines?

- An *Azure DevOps* organization and project
 - A *GitHub* repository (personal or organization)
 - Sufficient permissions on both platforms (repo admin or owner, Azure Pipelines Contributor)
 - *Azure Pipelines* app authorized in *GitHub* (installed as needed)
-

2. How do you connect a GitHub repository to an Azure Pipeline?

- In Azure DevOps, create a new pipeline
 - Choose GitHub as the code source
 - Authorize Azure Pipelines to access your GitHub account
 - Select the repository and configure the pipeline (YAML or classic editor)
-

3. What authentication methods are supported for GitHub-Azure Pipelines integration?

- *OAuth* via Azure Pipelines GitHub App (recommended)
 - Personal Access Token (PAT) for classic connections
 - GitHub Apps for organization-wide integration
-

4. How do you configure pipeline triggers for GitHub repository events?

- In the YAML pipeline file, use the `trigger:` keyword for branch updates and `pr:` for pull requests
 - Example:

```
trigger:
  branches:
    include:
      - main
pr:
  branches:
    include:
      - main
```
-

5. How do you set up build validation for GitHub pull requests in Azure Pipelines?

- Enable branch protection in *GitHub*
 - Add *Azure Pipelines* as a required status check for PRs
 - Configure the pipeline to run on pull requests using the `pr: trigger` in YAML
-

6. How can you use YAML pipelines with GitHub repositories?

- Place an `azure-pipelines.yml` file in the root of the GitHub repo
 - Configure the pipeline in *Azure DevOps* to use this YAML file
 - Updates to the YAML file in *GitHub* automatically update the pipeline definition
-

7. What permissions are required for Azure Pipelines to access a GitHub repository?

- Repo-level access for the *Azure Pipelines* app
 - At minimum, read code and read/write for status checks
 - Admin access may be needed for initial setup and webhook management
-

8. How are service connections managed and secured for GitHub integration?

- Use the *Azure Pipelines GitHub App* for secure, managed integration
 - Service connections are created in *Project Settings > Service connections*
 - Use least privilege and restrict access to pipelines needing the connection
-

9. What are best practices for handling secrets in GitHub-Azure Pipelines workflows?

- Store secrets in *Azure Pipelines* variable groups or *GitHub Secrets*
 - Reference secrets in pipeline YAML using secure variables
 - Never hard-code secrets in repository or pipeline files
-

10. How do you monitor and troubleshoot integration issues between GitHub and Azure Pipelines?

- Review pipeline run logs and error messages in *Azure DevOps*
- Check *GitHub* repository “Actions” and “Security & analysis” tabs
- Validate webhooks and service connection health in *Azure DevOps* project settings
- Check permission scopes and reauthorize the *Azure Pipelines GitHub App* if needed

3. Build and release pipelines

└ 3.2 Design and implement pipelines

└ 3.2.4 Pipeline triggers, YAML usage, job execution

1. What types of pipeline triggers are supported in Azure Pipelines?
 2. How do you define continuous integration (CI) and pull request (PR) triggers in YAML?
 3. How do you disable or override triggers for specific branches or paths?
 4. What is the purpose of paths and paths-ignore in triggers?
 5. How do you set up scheduled (cron) triggers in a YAML pipeline?
 6. How do you structure multi-stage YAML pipelines?
 7. What are jobs and how do you define them in YAML?
 8. How do you configure job dependencies and execution order?
 9. How do you run jobs in parallel or sequentially?
 10. What are key best practices for reliable job execution in Azure Pipelines YAML?
-

1. What types of pipeline triggers are supported in Azure Pipelines?

- Continuous integration (CI) triggers
 - Pull request (PR) triggers
 - Scheduled triggers (cron)
 - Pipeline triggers (triggered by another pipeline)
-

2. How do you define continuous integration (CI) and pull request (PR) triggers in YAML?

Use the trigger and pr keys with properly indented branches:

```
trigger:
  branches:
    include:
      - main
      - develop
pr:
  branches:
    include:
      - main
      - develop
```

3. How do you disable or override triggers for specific branches or paths?

Set trigger: none or use exclude: under branches.

```
trigger: none

trigger:
  branches:
    exclude:
      - experimental
```

4. What is the purpose of paths and paths-ignore in triggers?

- paths: Run pipeline only when files matching the patterns are changed
- paths-ignore: Ignore runs if only files matching these patterns are changed

```
trigger:
  branches:
    include:
      - main
  paths:
    include:
      - src/**
  paths-ignore:
    - docs/**
```

5. How do you set up scheduled (cron) triggers in a YAML pipeline?

Use the schedules key:

```
schedules:
  - cron: "0 0 * * 1"
    displayName: "Weekly build"
    branches:
      include:
        - main
    always: true
```

6. How do you structure multi-stage YAML pipelines?

Use the stages key to define multiple stages, each with its own jobs.

```
stages:
  - stage: Build
    jobs:
      - job: BuildJob
        steps:
          - script: echo Building
  - stage: Deploy
    jobs:
      - job: DeployJob
        steps:
          - script: echo Deploying
```

7. What are jobs and how do you define them in YAML?

Jobs are collections of steps run on agents. Define under jobs.

```
jobs:
  - job: TestJob
    steps:
      - script: echo Running tests
```

8. How do you configure job dependencies and execution order?

Use the dependsOn keyword. By default, jobs run in parallel unless dependencies are set.

```
jobs:
  - job: Build
    steps:
      - script: echo Build
  - job: Test
    dependsOn: Build
    steps:
      - script: echo Test
```

9. How do you run jobs in parallel or sequentially?

- Jobs run in parallel by default
- Use `dependsOn` to run jobs sequentially
- No `dependsOn` means jobs start together

10. What are key best practices for reliable job execution in Azure Pipelines YAML?

- Use clear job/stage names
- Define explicit dependencies with `dependsOn`
- Isolate steps to minimize side effects
- Use variables and templates for reusability
- Monitor pipeline runs for failures and flaky jobs

3. Build and release pipelines

└ 3.3 Advanced pipeline design

└ 3.3.1 Hybrid pipelines, self-hosted agents, VM templates

1. What is a hybrid pipeline and when should it be used?
 2. How do you configure a pipeline to run on both Microsoft-hosted and self-hosted agents?
 3. What are the core considerations when setting up self-hosted agents in Azure Pipelines?
 4. How do you secure and update self-hosted agents?
 5. What are the benefits and drawbacks of self-hosted agents vs. Microsoft-hosted agents?
 6. How do you use VM templates in pipeline agent pools?
 7. What are the main steps to register a new self-hosted agent?
 8. How do you configure hybrid jobs across multiple agent types in a single pipeline YAML?
 9. What are the key troubleshooting steps if a self-hosted agent fails to run jobs?
 10. How do you ensure scalability and availability of self-hosted agents in enterprise scenarios?
-

1. What is a hybrid pipeline and when should it be used?

A hybrid pipeline uses both Microsoft-hosted and self-hosted agents within the same pipeline to leverage cloud scalability and custom environments. Use it when some tasks require custom tooling, dependencies, or access to private resources not available on Microsoft-hosted agents.

2. How do you configure a pipeline to run on both Microsoft-hosted and self-hosted agents?

Define separate jobs or stages in your YAML pipeline, specifying `pool: Default (self-hosted)` or `pool: vmImage: 'ubuntu-latest'` (Microsoft-hosted) for each job. Example:

```
jobs:
- job: Build
  pool:
    vmImage: 'ubuntu-latest'
- job: CustomTask
  pool:
    name: Default
```

3. What are the core considerations when setting up self-hosted agents in Azure Pipelines?

Ensure agents meet system requirements, have required network access, correct permissions, are kept secure, updated, and registered to the right agent pool. Monitor agent health and load for capacity planning.

4. How do you secure and update self-hosted agents?

- Run agents as a non-admin user
- Restrict network access
- Install security updates and agent updates promptly
- Protect agent keys
- Use firewalls/endpoint protection
- Regularly audit and rotate agent authentication tokens

5. What are the benefits and drawbacks of self-hosted agents vs. Microsoft-hosted agents?

Self-hosted agents:

- Full control over software/tools, network, and environment
- Can access private resources
- Require maintenance, security, scaling, and cost management

Microsoft-hosted agents:

- Fully managed, always updated, no maintenance
 - Scales automatically
 - Limited customizations, no persistent state, no access to private on-prem resources
-

6. How do you use VM templates in pipeline agent pools?

Create a preconfigured VM image (template) with required tools and software. Use this template to automatically provision new VMs as agents (e.g., with Azure Scale Set agent pools), ensuring consistent environments and fast scaling.

7. What are the main steps to register a new self-hosted agent?

- Download the agent package from Azure DevOps.
 - Extract the files.
 - Run the config script (e.g., config.cmd or config.sh), provide server URL, authentication token (PAT), and select agent pool/name.
 - Start the agent interactively or as a service/daemon.
-

8. How do you configure hybrid jobs across multiple agent types in a single pipeline YAML?

- In YAML, define multiple jobs, each with its own pool property.
 - Example:

```
jobs:
  - job: linuxBuild
    pool:
      vmImage: 'ubuntu-latest'
  - job: customJob
    pool:
      name: MySelfHostedPool
```
 - This enables one pipeline to use both Microsoft-hosted and self-hosted agents.
-

9. What are the key troubleshooting steps if a self-hosted agent fails to run jobs?

- Confirm agent is online and showing as “Available” in Azure DevOps.
 - Check agent logs for errors.
 - Verify agent machine connectivity to Azure DevOps (outbound HTTPS 443).
 - Ensure authentication tokens are valid and not expired.
 - Confirm permissions for agent pool and job steps.
 - Validate required software/dependencies are installed.
-

10. How do you ensure scalability and availability of self-hosted agents in enterprise scenarios?

- Use Azure Virtual Machine Scale Set (VMSS) agent pools for auto-scaling.
- Deploy agents across multiple regions or availability zones.
- Monitor agent utilization and automate provisioning/de-provisioning.
- Set up health checks and automated recovery for failed agents.
- Maintain backup agents to handle load spikes.

3. Build and release pipelines

└ 3.3 Advanced pipeline design

└ 3.3.2 Reusable elements: YAML templates, task/variable groups

1. What are YAML templates in Azure Pipelines, and why use them?
 2. How do you create and include a template in a pipeline YAML file?
 3. What is a task group, and how is it different from a YAML template?
 4. What are variable groups, and how are they used in Azure Pipelines?
 5. How do you reference variable groups in YAML pipelines?
 6. How can you pass parameters to YAML templates?
 7. What is the best practice for sharing templates across multiple pipelines or repos?
 8. How do you manage secrets with variable groups?
 9. How do you use conditions and expressions within YAML templates?
 10. What are common pitfalls or limitations when using reusable pipeline elements?
-

1. What are YAML templates in Azure Pipelines, and why use them?

YAML templates are reusable pipeline definitions that allow you to define pipeline logic once and include it in multiple pipelines. They promote DRY principles and consistency across builds and releases.

2. How do you create and include a template in a pipeline YAML file?

Define the template in a separate YAML file (e.g., template.yml). Include it in your pipeline using the template keyword:

```
- template: template.yml
```

Parameters can be passed if defined.

3. What is a task group, and how is it different from a YAML template?

A task group is a saved set of tasks from classic (visual designer) pipelines, reusable in multiple pipelines via the UI. YAML templates are code-based and used in YAML pipelines for reusability and version control.

4. What are variable groups, and how are they used in Azure Pipelines?

Variable groups store and manage values (e.g., connection strings, secrets) shared across pipelines. They centralize variables for reuse and easier updates.

5. How do you reference variable groups in YAML pipelines?

Use the variables block and the group keyword:

variables:

```
- group: MyVariableGroup
```

This makes the group's variables available in the pipeline.

6. How can you pass parameters to YAML templates?

Define parameters in the template using the parameters block. Pass values when including the template:

```
- template: template.yml
```

parameters:

```
name: value
```

7. What is the best practice for sharing templates across multiple pipelines or repos?

Store templates in a central repository, reference them via repository resources, and use versioning or branch references for control.

8. How do you manage secrets with variable groups?

Mark variables as “secret” in the variable group. Access secrets in pipelines but they are masked in logs and not exposed to scripts by default.

9. How do you use conditions and expressions within YAML templates?

Templates and steps can use condition and expressions (e.g., `eq(variables['Build.SourceBranch'], 'refs/heads/main')`) to control execution based on pipeline context or parameters.

10. What are common pitfalls or limitations when using reusable pipeline elements?

Common issues:

- Parameter type mismatches
- Circular template references
- Scope/visibility problems with variable groups
- Inability to use task groups in YAML pipelines (YAML and task groups are separate)
- Secret variables not available to all steps by default

3. Build and release pipelines

└ 3.3 Advanced pipeline design

└ 3.3.3 Checks and approvals with YAML environments

1. What are environment checks in Azure Pipelines YAML?
 2. How do you configure approvals for a deployment environment in YAML?
 3. What types of checks can be enforced in an environment?
 4. How do you reference and use environments in YAML pipelines?
 5. How do manual approvals work in multi-stage pipelines?
 6. How can you restrict deployments to specific environments?
 7. How do you require multiple approvers for an environment?
 8. What are best practices for designing approval workflows in YAML?
 9. What limitations or pitfalls exist with environment checks and approvals?
 10. How do you audit or monitor approval history for environments?
-

1. What are environment checks in Azure Pipelines YAML?

Environment checks are policies (such as approvals, service health, or business rules) that must pass before a deployment can proceed to a given environment.

2. How do you configure approvals for a deployment environment in YAML?

Define the environment in the deployment job and set up approvals in the Azure DevOps portal under *Pipelines > Environments*. Approvals are not specified directly in YAML.

3. What types of checks can be enforced in an environment?

Common checks include:

- Manual approvals
 - Azure Function or REST API checks
 - Business rule validations
 - Service health checks
-

4. How do you reference and use environments in YAML pipelines?

Use the `environment:` keyword in a deployment job:

```
environment: 'staging'
```

This triggers checks and approvals configured for that environment.

5. How do manual approvals work in multi-stage pipelines?

The pipeline pauses at the environment check until designated users approve. Only after approval does the deployment job execute.

6. How can you restrict deployments to specific environments?

Limit access using environment security settings in *Azure DevOps*, specifying who can deploy and approve to each environment.

7. How do you require multiple approvers for an environment?

In the *Azure DevOps* portal, add multiple users or groups to the approval list for the environment. You can set whether all or just one must approve.

8. What are best practices for designing approval workflows in YAML?

- Use environment approvals for production/staging
 - Limit approvers to relevant stakeholders
 - Combine automated and manual checks
 - Regularly review and update approver lists
-

9. What limitations or pitfalls exist with environment checks and approvals?

- Approval policies must be managed in the portal, not in YAML
 - Delays if approvers are unavailable
 - Lack of fine-grained conditional logic for approvals
 - Inability to automate some business rules fully
-

10. How do you audit or monitor approval history for environments?

View approval history and audit logs in the *Azure DevOps* portal under the environment's "History" tab. This includes who approved, when, and comments if provided.

3. Build and release pipelines

└ 3.4 Deployment strategy and execution

└ 3.4.1 Blue green, canary, ring, progressive exposure, A/B

-
1. What is blue-green deployment and when should it be used?
 2. What is canary deployment and what problem does it solve?
 3. What is a ring deployment in Azure DevOps?
 4. How does progressive exposure differ from blue-green and canary deployments?
 5. What is A/B testing in deployment strategies?
 6. How do you minimize downtime using blue-green deployment in Azure?
 7. How can Azure DevOps Pipelines implement canary releases?
 8. What are best practices for monitoring during a canary or ring deployment?
 9. How do feature flags support progressive exposure and A/B testing?
 10. What are the rollback strategies for blue-green and canary deployments?
-

1. What is blue-green deployment and when should it be used?

Blue-green deployment runs two identical production environments ("blue" and "green"). Traffic switches from blue to green after validation, minimizing downtime and risk. Use when you require near-zero downtime and easy rollback.

2. What is canary deployment and what problem does it solve?

Canary deployment releases new changes to a small subset of users first. It mitigates risk by exposing only a fraction of traffic to new changes. If issues are detected, rollback is quick and impact is limited.

3. What is a ring deployment in Azure DevOps?

A ring deployment rolls out releases in controlled stages ("rings"), each representing a user or environment group. Inner rings (e.g., internal users) get updates first, followed by broader rings (e.g., all users). This enables staged rollout and feedback at each step.

4. How does progressive exposure differ from blue-green and canary deployments?

Progressive exposure gradually increases the percentage of users exposed to a new release based on real-time feedback. Unlike blue-green (all at once) and canary (fixed groups), exposure is dynamically adjusted, allowing rapid rollback or pause if issues occur.

5. What is A/B testing in deployment strategies?

A/B testing directs user segments to different application versions simultaneously (A vs. B) to compare behavior or feature impact. It's used to evaluate new features or UI changes by analyzing real user metrics before full rollout.

6. How do you minimize downtime using blue-green deployment in Azure?

Deploy to the "green" slot/environment, validate, then use Azure Traffic Manager or App Service swap to instantly redirect production traffic. This ensures seamless cutover with minimal or no downtime.

7. How can Azure DevOps Pipelines implement canary releases?

Use deployment jobs and environment approvals to release to a small subset of instances or users first. Gradually increase scope by adding stages that deploy to more targets as health checks pass.

8. What are best practices for monitoring during a canary or ring deployment?

- Enable detailed telemetry and alerts (e.g., Application Insights).
 - Monitor error rates, latency, and user experience in real-time.
 - Automate rollback if metrics exceed defined thresholds.
-

9. How do feature flags support progressive exposure and A/B testing?

Feature flags allow you to enable or disable features for specific users or groups without redeploying code. They control exposure dynamically, supporting gradual rollout (progressive exposure) or randomized experiments (A/B testing).

10. What are the rollback strategies for blue-green and canary deployments?

For blue-green, switch traffic back to the previous environment. For canary, halt rollout and reroute all users to the stable version, removing problematic instances from the canary group.

3. Build and release pipelines

└ 3.4 Deployment strategy and execution

└ 3.4.2 Ordered dependency deployments

1. What is an ordered dependency deployment?
 2. Why are ordered deployments important in multi-tier applications?
 3. How can you enforce deployment order in Azure DevOps pipelines?
 4. How do pipeline stages and dependencies work in YAML pipelines?
 5. What are artifact dependencies and how do they affect deployment order?
 6. How can environment approvals enforce ordered deployment?
 7. What's the risk of ignoring dependency order in deployments?
 8. How can you handle rollback if a downstream dependency fails?
 9. How do you automate database and application deployment order?
 10. What tools or tasks in Azure Pipelines help manage ordered deployments?
-

1. What is an ordered dependency deployment?

It is a deployment sequence that ensures components are deployed in a specific order based on their dependencies, such as deploying a database before the application that uses it.

2. Why are ordered deployments important in multi-tier applications?

They prevent runtime errors by ensuring lower-level components (e.g., databases, APIs) are ready before dependent components (e.g., web apps) are deployed.

3. How can you enforce deployment order in Azure DevOps pipelines?

Define multiple stages in YAML, using the `dependsOn` keyword to control execution order. Each stage deploys one component; later stages run only after dependencies succeed.

4. How do pipeline stages and dependencies work in YAML pipelines?

Stages can be explicitly ordered using `dependsOn`. A stage will only start after its dependencies have completed successfully, ensuring proper deployment sequence.

5. What are artifact dependencies and how do they affect deployment order?

Artifact dependencies link pipeline outputs (e.g., builds) to deployments. A deployment waits for required artifacts to be published by earlier stages, enforcing correct order.

6. How can environment approvals enforce ordered deployment?

You can require manual or automated approvals on environments. A later environment or stage won't proceed until prior deployments are approved, adding an extra control layer for order.

7. What's the risk of ignoring dependency order in deployments?

If components are deployed out of order, dependencies may be missing or incompatible, causing application errors, failed deployments, or downtime.

8. How can you handle rollback if a downstream dependency fails?

Automate rollback by defining pipeline tasks that revert all affected components to their previous stable versions if any dependent deployment fails.

9. How do you automate database and application deployment order?

In your pipeline YAML, create separate stages for the database and application. Set the application stage to `dependsOn` the database stage, ensuring the database is deployed and verified first.

11. What tools or tasks in Azure Pipelines help manage ordered deployments?

Azure Pipelines provides

- YAML stages,
- `dependsOn`,
- classic Release pipeline environments,
- deployment jobs,
- approvals,
- gates,
- and artifact dependencies

to control and enforce deployment order.

3. Build and release pipelines

└ 3.4 Deployment strategy and execution

└ 3.4.3 Minimize downtime: VIP swap, rolling, slots

-
1. What is a VIP swap and when should it be used in Azure deployments?
 2. How do deployment slots help minimize downtime in Azure App Service?
 3. What is a rolling deployment and how is it executed in Azure?
 4. Compare the benefits and limitations of VIP swap versus rolling deployment.
 5. How do you perform a slot swap in Azure Pipelines?
 6. What considerations are required for zero-downtime deployments in multi-instance environments?
 7. How do health checks affect rolling deployments and slot swaps?
 8. What are common pitfalls to avoid when minimizing downtime during deployment?
 9. How do you roll back safely after a failed slot swap or rolling deployment?
 10. What best practices ensure minimal user impact during production deployments using these strategies?
-

1. What is a VIP swap and when should it be used in Azure deployments?

A VIP (Virtual IP) swap exchanges the public IP addresses of two *Azure* cloud service deployments (usually staging and production) instantly. It should be used to promote a pre-warmed, validated deployment to production with minimal downtime.

2. How do deployment slots help minimize downtime in Azure App Service?

Deployment slots allow you to host multiple app versions (e.g., staging, production) on the same App Service. You deploy to a staging slot, validate, then swap to production, ensuring fast cutover and rollback with minimal disruption.

3. What is a rolling deployment and how is it executed in Azure?

A rolling deployment updates subsets of instances incrementally, removing a portion from the load balancer, updating it, then proceeding to the next subset. In *Azure*, rolling deployments can be managed using *Virtual Machine Scale Sets* or *Kubernetes* for controlled, phased rollouts.

4. Compare the benefits and limitations of VIP swap versus rolling deployment.

- **VIP swap:** Near-instant cutover, easy rollback, but both environments must be identical.
 - **Rolling deployment:** Gradual updates, reduced risk of mass failure, allows monitoring during rollout, but may introduce temporary state inconsistencies and slightly longer total deployment time.
-

5. How do you perform a slot swap in Azure Pipelines?

Add an "Azure App Service deploy" task, specify the target deployment slot (e.g., staging), then use the "Swap Slots" action to exchange staging and production after validation. This can be automated with *Azure CLI* or built-in pipeline tasks.

6. What considerations are required for zero-downtime deployments in multi-instance environments?

Ensure at least one instance is always available by updating instances incrementally, using health probes, and configuring load balancers to direct traffic only to healthy, updated instances. Maintain session state externally to prevent user disruption.

7. How do health checks affect rolling deployments and slot swaps?

Health checks ensure only healthy instances receive traffic during and after deployment. In rolling deployments, unhealthy instances are removed from rotation. For slot swaps, pre-swap validation uses health probes to confirm readiness before traffic is switched.

8. What are common pitfalls to avoid when minimizing downtime during deployment?

- Skipping health checks
 - Deploying breaking schema changes without backward compatibility
 - Not externalizing session state
 - Failing to test slot settings or configuration differences
 - Insufficient monitoring during rollout
-

9. How do you roll back safely after a failed slot swap or rolling deployment?

For slot swaps, immediately swap back to the previous slot. For rolling deployments, stop the rollout and redeploy the previous stable version to affected instances. Always validate state and configuration before rollback.

10. What best practices ensure minimal user impact during production deployments using these strategies?

- Use staging slots and swap only after validation
- Deploy during low-traffic periods
- Implement incremental rollout with monitoring and automated rollback
- Externalize state/session management
- Automate health checks and alerts

3. Build and release pipelines

└ 3.4 Deployment strategy and execution

└ 3.4.4 Hotfix and resiliency planning

1. What is a hotfix and when should it be used in a DevOps context?
 2. How do you implement a hotfix deployment process in Azure DevOps or GitHub Actions?
 3. What strategies enable rapid hotfix deployment with minimal risk?
 4. How do you separate hotfix and feature releases in your pipeline?
 5. What are the best practices for rolling back a failed hotfix?
 6. How do you ensure pipeline and deployment resiliency against failures?
 7. What mechanisms can automate detection and recovery from failed deployments?
 8. How can you design pipelines to support high availability and quick recovery?
 9. What tools and features help with monitoring and alerting for deployment issues?
 10. What are key considerations for maintaining compliance and auditability during hotfixes and recovery actions?
-

1. What is a hotfix and when should it be used in a DevOps context?

A hotfix is an urgent, targeted code or configuration change deployed to resolve critical issues in production. Use a hotfix when immediate correction is needed to restore functionality or security.

2. How do you implement a hotfix deployment process in Azure DevOps or GitHub Actions?

- Create a hotfix branch from the production or main branch.
 - Apply and test the fix.
 - Use your CI/CD pipeline to deploy directly to production or a hotfix slot.
 - Merge the hotfix back into main and other relevant branches.
-

3. What strategies enable rapid hotfix deployment with minimal risk?

- Maintain a streamlined hotfix pipeline with minimal validation steps.
 - Use automated tests focused on affected areas.
 - Deploy via slots or canary/rolling deployments for quick rollback.
 - Isolate hotfix changes from feature development.
-

4. How do you separate hotfix and feature releases in your pipeline?

- Use branching strategies (e.g., dedicated hotfix branches).
 - Tag or label pipeline runs as hotfix or feature.
 - Configure CI/CD to allow independent release of hotfixes to production without waiting for feature branch merges.
-

5. What are the best practices for rolling back a failed hotfix?

- Immediately redeploy the previous stable release using pipeline history, slot swap, or rollback tasks.
- Validate system state and data consistency.
- Communicate status and root cause to stakeholders.

6. How do you ensure pipeline and deployment resiliency against failures?

- Use automated health checks after deployment steps.
 - Implement retry logic for transient errors.
 - Use deployment slots, rolling updates, or blue-green strategies.
 - Monitor pipeline health and enforce gates/approvals.
-

7. What mechanisms can automate detection and recovery from failed deployments?

- Configure alerts and automated rollbacks in CI/CD pipelines.
 - Use *Azure Monitor* or *Application Insights* to detect failures.
 - Employ pipeline conditions to trigger redeployment or revert actions automatically.
-

8. How can you design pipelines to support high availability and quick recovery?

- Use multi-region or multi-zone deployments.
 - Keep standby or backup instances updated.
 - Automate failover and rollback procedures.
 - Test disaster recovery regularly.
-

9. What tools and features help with monitoring and alerting for deployment issues?

- *Azure Monitor*, *Log Analytics*, and *Application Insights* for real-time monitoring and alerts.
 - *Service Health* and *Resource Health* for Azure infrastructure monitoring.
 - *GitHub Actions* and *Azure DevOps* built-in notification and alert integrations.
-

10. What are key considerations for maintaining compliance and auditability during hotfixes and recovery actions?

- Enforce approvals and access controls for hotfix branches and deployments.
- Maintain detailed logs and change history for all hotfixes.
- Document root cause and recovery steps for every incident.

3. Build and release pipelines

└ 3.5 Implement deployment solutions

└ 3.5.1 Feature Flags with Azure App Configuration

1. What are feature flags and why are they used in DevOps?
 2. What is Azure App Configuration and how does it support feature management?
 3. How do you create and manage feature flags in Azure App Configuration?
 4. How do you enable or disable a feature flag at runtime without redeploying code?
 5. What is the Feature Manager library and how is it used in .NET applications?
 6. How can feature flags be targeted for specific users or environments?
 7. How do you implement feature flag evaluation in Azure DevOps pipelines?
 8. What are best practices for rolling out and rolling back features using flags?
 9. How can feature flag usage be audited and monitored?
 10. What are the risks or anti-patterns when using feature flags in production?
-

1. What are feature flags and why are they used in DevOps?

Feature flags are switches in application code to enable or disable features at runtime. They allow teams to release code to production safely, test in production, perform gradual rollouts, and roll back features instantly without redeployment.

2. What is Azure App Configuration and how does it support feature management?

Azure App Configuration is a centralized service for application settings and feature flags. Its Feature Management capability allows dynamic creation, updating, and targeting of feature flags for apps at scale, decoupling feature rollout from code releases.

3. How do you create and manage feature flags in Azure App Configuration?

Feature flags are created and managed in the *Azure* portal under *App Configuration > Feature Manager*. Flags can be enabled, disabled, or targeted, and settings are available via client libraries or REST API.

4. How do you enable or disable a feature flag at runtime without redeploying code?

Change the flag's state in *Azure App Configuration*. The application, using the *Feature Management SDK*, will check for flag changes at runtime (polling or refresh interval) and update behavior instantly, without requiring a redeployment.

5. What is the Feature Manager library and how is it used in .NET applications?

The *Microsoft.FeatureManagement* library is a .NET SDK that integrates with *Azure App Configuration*. Add and configure it in your app to query and evaluate feature flags dynamically using dependency injection, middleware, or attributes.

6. How can feature flags be targeted for specific users or environments?

Azure App Configuration supports filters (e.g., user targeting, time windows, percentage rollout). Configure these filters in *Feature Manager* to enable features for selected users, groups, or environments based on context.

7. How do you implement feature flag evaluation in Azure DevOps pipelines?

Use the *Azure App Configuration* task in *Azure Pipelines* to retrieve flag values as pipeline variables. This allows pipeline stages, jobs, or tasks to conditionally execute based on the flag state in *App Configuration*.

8. What are best practices for rolling out and rolling back features using flags?

- Use gradual exposure (percentage rollout, user targeting),
- monitor impact,
- and have clear rollback procedures.
- Remove obsolete flags quickly to avoid technical debt and keep the configuration clean.

9. How can feature flag usage be audited and monitored?

Audit changes using *Azure Activity Logs* and *App Configuration* revision history. Monitor feature usage and application impact using *Application Insights* or custom telemetry tied to flag evaluation points.

11. What are the risks or anti-patterns when using feature flags in production?

Common risks include

- flag overload,
- leaving dead flags,
- inconsistent flag states across services,
- and insufficient monitoring.

These can lead to increased complexity, technical debt, and production issues if not managed properly.

3. Build and release pipelines

└ 3.5 Implement deployment solutions

└ 3.5.2 Deploy containers, binaries, scripts

-
1. What are the main deployment options for containers in Azure DevOps?
 2. How do you deploy Docker containers to Azure Kubernetes Service (AKS) using a pipeline?
 3. How do you publish and deploy application binaries using Azure Pipelines?
 4. What are the typical steps to deploy a script-based application in Azure DevOps?
 5. How can deployment scripts be securely managed and executed in pipelines?
 6. What tasks are available for deploying to Azure Web Apps from Azure DevOps?
 7. How do you automate image versioning and tagging during container deployment?
 8. What is the difference between classic and YAML-based deployment pipelines for these scenarios?
 9. How do you handle rollback for failed deployments of containers, binaries, or scripts?
 10. What are best practices for monitoring and validating deployments?
-

1. What are the main deployment options for containers in Azure DevOps?

You can deploy containers to

- Azure Kubernetes Service (AKS),
 - Azure Container Instances,
 - Azure App Service for Containers,
 - or custom environments using built-in *Azure DevOps* tasks and service connections.
-

2. How do you deploy Docker containers to Azure Kubernetes Service (AKS) using a pipeline?

Use *Azure Pipelines* tasks: build and push image with *Docker* or *Azure CLI*, then use the *KubernetesManifest* or *kubectl* task to apply *Kubernetes* manifests and deploy to AKS.

3. How do you publish and deploy application binaries using Azure Pipelines?

Build binaries using build tasks, publish them as pipeline artifacts, then deploy using tasks like *AzureWebApp* OR *AzureFileCopy* or custom script tasks to the desired target.

4. What are the typical steps to deploy a script-based application in Azure DevOps?

Store scripts in source control, reference them in pipeline YAML or classic release, use tasks like *Bash* *PowerShell* OR *Command Line* to execute scripts, and manage outputs or errors accordingly.

5. How can deployment scripts be securely managed and executed in pipelines?

Store scripts in versioned source control. Store secrets/credentials in secure pipeline variables or *Azure Key Vault*. Never hard-code secrets in scripts; access them via secure variables or *Key Vault* references.

6. What tasks are available for deploying to Azure Web Apps from Azure DevOps?

- *AzureWebApp* task for code or binaries
- *AzureWebAppContainer* for *Docker* images
- *Azure App Service Deploy* for deployment slots and multi-phase scenarios

7. How do you automate image versioning and tagging during container deployment?

In your pipeline,

- use build IDs,
- commit hashes,
- or semantic versioning in the *Docker* tag step.

Automate tagging in the Docker or Azure CLI tasks before pushing images to a registry.

8. What is the difference between classic and YAML-based deployment pipelines for these scenarios?

- YAML pipelines provide pipeline-as-code, versioning, templates, and better reuse.
 - Classic pipelines are GUI-based, less portable, and harder to manage for complex deployments.
-

9. How do you handle rollback for failed deployments of containers, binaries, or scripts?

Use pipeline stages to deploy to slots or canary environments. For containers, roll back by redeploying a previous image tag. For scripts/binaries, restore prior artifacts or use automated pipeline rollback steps.

11. What are best practices for monitoring and validating deployments?

- Enable deployment logs,
- configure health probes,
- set up *Application Insights*,
- and use release gates or post-deployment tests to validate success before full rollout.

3. Build and release pipelines

└ 3.5 Implement deployment solutions

└ 3.5.3 Database Task Deployments

-
1. What are the main methods for deploying database changes in Azure DevOps pipelines?
 2. How do you configure and use the Azure SQL Database Deployment task?
 3. What are the best practices for database schema migrations in CI/CD?
 4. How do you manage rollback and recovery for failed database deployments?
 5. What security considerations must be addressed for database deployments in pipelines?
 6. How do you automate database deployments using DACPACs?
 7. How can you validate and test database changes before deployment?
 8. What tools are commonly integrated for database deployments in Azure Pipelines?
 9. How do you handle environment-specific database settings during deployment?
 10. How do you use release gates or approvals for database deployment steps?
-

1. What are the main methods for deploying database changes in Azure DevOps pipelines?

- Use built-in *Azure SQL Database Deployment* tasks,
- SQL scripts,
- DACPAC deployment,
- third-party tools (e.g., Redgate),
- or custom scripts.

The most common approaches are DACPAC and SQL script execution.

2. How do you configure and use the Azure SQL Database Deployment task?

Add the "*Azure SQL Database Deployment*" task to your pipeline, specify the Azure service connection, target server/database, authentication, and select deployment type (DACPAC, SQL script). Set necessary parameters like SQL user/password or managed identity.

3. What are the best practices for database schema migrations in CI/CD?

- Version control all database changes,
- use incremental scripts or migration frameworks,
- validate in lower environments,
- run automated tests post-deployment,
- use idempotent scripts,
- and ensure deployment steps are repeatable.

4. How do you manage rollback and recovery for failed database deployments?

- Implement automated database backups before deployment,
- use transaction blocks in scripts,
- maintain rollback scripts,
- and configure point-in-time restore for Azure SQL.

For DACPAC, keep previous versions for re-deployment.

5. What security considerations must be addressed for database deployments in pipelines?

- Store credentials securely (*Azure Key Vault*, pipeline secrets),
- use managed identities or service principals,
- restrict permissions to least privilege,
- and audit all deployment activities.

6. How do you automate database deployments using DACPACs?

Build the DACPAC artifact from source, publish it as a pipeline artifact, and deploy using the "*Azure SQL Database Deployment*" task with the DACPAC file, specifying connection and deployment options.

7. How can you validate and test database changes before deployment?

- Run schema validation tasks,
- execute unit and integration tests against a staging database,
- and use automated smoke tests post-deployment.

Use tools like *tSQLt* or *SQL Server Data Tools (SSDT)* for validation.

8. What tools are commonly integrated for database deployments in Azure Pipelines?

- *Azure SQL Database Deployment* task,
- *SQL Server Data Tools (SSDT)*,
- Redgate SQL Change Automation,
- custom PowerShell or Bash scripts,
- and *Azure Key Vault* for secrets management.

9. How do you handle environment-specific database settings during deployment?

- Use pipeline variables/groups,
- parameterize scripts or DACPAC publish profiles,
- and maintain separate configuration files or ARM templates per environment.

10. How do you use release gates or approvals for database deployment steps?

- Configure pipeline environments with manual approval checks,
- set up release gates based on monitoring/alerts or pre/post-deployment conditions,
- and require authorized personnel to approve before production deployments.

3. Build and release pipelines

└ 3.5 Implement deployment solutions

└ 3.5.4 Configuration Management Tools & Strategies

-
1. What are configuration management tools, and why are they important in DevOps pipelines?
 2. Which configuration management tools are commonly used with Azure DevOps?
 3. How do you integrate tools like Ansible, Chef, or Puppet into Azure Pipelines?
 4. What are best practices for managing application configuration in CI/CD?
 5. How do you secure sensitive configuration data in deployment pipelines?
 6. What is Infrastructure as Code (IaC), and how does it relate to configuration management?
 7. How can you automate configuration drift detection and remediation?
 8. How do you handle configuration for different environments (dev, test, prod)?
 9. What Azure-native configuration management options exist?
 10. How do you monitor and audit configuration changes across environments?
-

1. What are configuration management tools, and why are they important in DevOps pipelines?

Tools that automate

- the deployment,
- configuration,
- and maintenance

of infrastructure and applications. They ensure

- consistency,
- repeatability,
- and compliance

across environments.

2. Which configuration management tools are commonly used with Azure DevOps?

Ansible, Chef, Puppet, PowerShell DSC, and SaltStack. Azure-native tools include *Azure Automation State Configuration* and *Azure Automanage*.

3. How do you integrate tools like Ansible, Chef, or Puppet into Azure Pipelines?

Use marketplace tasks or custom scripts within pipeline steps. Install tool dependencies, authenticate to targets, and invoke playbooks/manifests/recipes as pipeline tasks.

4. What are best practices for managing application configuration in CI/CD?

- Store configuration as code,
 - use source control,
 - parameterize settings,
 - avoid hardcoding secrets,
 - validate changes in lower environments,
 - and document configuration decisions.
-

5. How do you secure sensitive configuration data in deployment pipelines?

- Use *Azure Key Vault* or pipeline secret variables,
 - avoid storing secrets in code or repositories,
 - restrict access to secrets,
 - and audit secret usage in pipeline logs.
-

6. What is Infrastructure as Code (IaC), and how does it relate to configuration management?

- IaC is managing infrastructure (servers, networks, etc.) using code and automation.
 - Configuration management ensures resources are provisioned & configured to desired state.
-

7. How can you automate configuration drift detection and remediation?

Use tools like *Azure Automation State Configuration*, *PowerShell DSC*, or monitoring solutions to regularly check for drift and automatically reapply the desired state if differences are detected.

8. How do you handle configuration for different environments (dev, test, prod)?

Use environment-specific parameter files, variable groups, or templates. Separate configuration data from code, and apply correct settings via pipeline variables at deployment time.

9. What Azure-native configuration management options exist?

Azure Automation State Configuration (DSC), *Azure Automate*, *Azure App Configuration*, and *Azure Policy* for enforcing compliance.

10. How do you monitor and audit configuration changes across environments?

Enable activity logs and change tracking (*Azure Monitor*, *Log Analytics*), use version control for configuration code, and review audit trails for pipeline and resource changes.

3. Build and release pipelines

└ 3.6 Infrastructure as Code (IaC)

└ 3.6.1 Define IaC strategy: Source Control, Automation

1. What is Infrastructure as Code (IaC) and why is it important in DevOps?
 2. What are the key principles of a successful IaC strategy?
 3. Which tools are commonly used for IaC in Azure and GitHub-based environments?
 4. How should IaC code be managed in source control?
 5. What are best practices for organizing IaC repositories?
 6. How can you automate testing and validation of IaC before deployment?
 7. What are the main automation options for deploying IaC in Azure DevOps and GitHub Actions?
 8. How do you manage secrets and sensitive configuration in IaC automation?
 9. What are recommended practices for versioning and tracking changes in IaC?
 10. How can you ensure repeatable and idempotent deployments with IaC?
-

1. What is Infrastructure as Code (IaC) and why is it important in DevOps?

- IaC is the practice of managing and provisioning infrastructure using code files, enabling automation, repeatability, and consistency.
 - It eliminates manual configuration drift and supports rapid, reliable delivery.
-

2. What are the key principles of a successful IaC strategy?

Key principles:

- version everything in source control,
 - automate validation/deployment,
 - use modular and reusable code,
 - ensure idempotency,
 - and follow DRY (Don't Repeat Yourself) practices.
-

3. Which tools are commonly used for IaC in Azure and GitHub-based environments?

Popular IaC tools:

- *ARM Templates*,
- *Bicep*,
- *Terraform*,
- *Azure CLI*,
- *Azure PowerShell*,
- and *Ansible*.

For automation:

- *Azure DevOps Pipelines*
 - and *GitHub Actions*.
-

4. How should IaC code be managed in source control?

- Store all IaC code in a versioned repository (e.g., Git).
- Use branching strategies,
- enforce code reviews,
- enable traceability,
- and restrict direct changes to main branches.

5. What are best practices for organizing IaC repositories?

- Structure repositories by environment, service, or module.
 - Use directories for dev, test, prod.
 - Keep sensitive values out of code and use templates/modules for reusability.
-

6. How can you automate testing and validation of IaC before deployment?

- Set up pipeline stages to lint, syntax check, and validate templates.
 - Use test deployments in isolated environments before production rollout.
-

7. What are the main automation options for deploying IaC in Azure DevOps and GitHub Actions?

- Use *Azure Pipelines* or *GitHub Actions* to trigger IaC deployments on push, PR, or schedule.
 - Integrate approvals, environment checks, and rollback mechanisms.
-

8. How do you manage secrets and sensitive configuration in IaC automation?

- Store secrets in *Azure Key Vault* or *GitHub Secrets*.
 - Reference them securely in pipeline definitions.
 - Never hard-code secrets in source or IaC files.
-

9. What are recommended practices for versioning and tracking changes in IaC?

- Tag releases,
 - use PRs for change review,
 - link commits to work items,
 - and document change history.
 - Use versioned modules
 - and maintain a clear changelog.
-

10. How can you ensure repeatable and idempotent deployments with IaC?

Write templates and scripts to be idempotent, meaning running them multiple times produces the same result. Use declarative syntax (e.g., *Bicep*, *Terraform*) and avoid manual post-deployment steps.

3. Build and release pipelines

└ 3.6 Infrastructure as Code (IaC)

└ 3.6.2 Implement desired state: Bicep, ARM, DSC

1. What does “desired state” mean in Infrastructure as Code?
 2. What are the core differences between Bicep and ARM templates?
 3. How do you declare and deploy resources with Bicep?
 4. How do you declare and deploy resources with ARM templates?
 5. What is Desired State Configuration (DSC) and when should you use it?
 6. How do you apply and monitor DSC on Azure virtual machines?
 7. How can you use Bicep and ARM templates together?
 8. What are best practices for parameterizing templates in Bicep and ARM?
 9. How do you validate and test Bicep or ARM templates before deployment?
 10. What methods can you use for secure secrets management with Bicep, ARM, and DSC?
-

1. What does “desired state” mean in Infrastructure as Code?

It refers to defining the exact configuration of infrastructure resources that should exist. Tools apply this state declaratively, ensuring the environment matches the intended configuration automatically.

2. What are the core differences between Bicep and ARM templates?

Bicep is a simpler, declarative DSL that transpiles to *ARM* JSON templates. *Bicep* improves readability and authoring, while *ARM* is JSON-based, more verbose, and harder to maintain.

3. How do you declare and deploy resources with Bicep?

Write a `.bicep` file with resource blocks. Deploy using:

```
az deployment group create --resource-group <rg> --template-file main.bicep
```

4. How do you declare and deploy resources with ARM templates?

Use a JSON file defining resources under `resources` array. Deploy using:

```
az deployment group create --resource-group <rg> --template-file main.json
```

5. What is Desired State Configuration (DSC) and when should you use it?

DSC is a *PowerShell*-based tool to configure and enforce VM state (e.g., software installed, services running). Use it when managing OS-level configurations, not just *Azure* resources.

6. How do you apply and monitor DSC on Azure virtual machines?

Use *Azure Automation DSC* or *Azure Automanage Machine Configuration*. Onboard VMs, assign DSC configurations, and monitor compliance from the *Azure* portal.

7. How can you use Bicep and ARM templates together?

Bicep transpiles to *ARM* JSON for deployment. You can reference existing *ARM* templates in *Bicep* using module blocks or include *Bicep* files inside *ARM* with linked templates.

8. What are best practices for parameterizing templates in Bicep and ARM?

- Use parameters for values like names, locations, and sizes.
- Provide default values where safe.
- Avoid hardcoding.
- Use `parameters.json` files for reusable input sets.

9. How do you validate and test Bicep or ARM templates before deployment?

- Use bicep build and bicep linter, OR az deployment validate to catch errors early.
 - Run test deployments in non-prod environments to ensure correctness.
-

11. What methods can you use for secure secrets management with Bicep, ARM, and DSC?

- Store secrets in *Azure Key Vault*.
- Reference them using `reference()` or `getSecret()` in templates or variables.
- Never embed secrets directly in the code or parameter files.

3. Build and release pipelines

└ 3.6 Infrastructure as Code (IaC)

└ 3.6.3 Azure Deployment Environments (Self-Deployment)

-
1. What are Azure Deployment Environments and what use case do they solve?
 2. How do Azure Deployment Environments support developer self-service?
 3. What are environment definitions and how are they created?
 4. How do you configure projects and environments for team use?
 5. How is access controlled in Azure Deployment Environments?
 6. What IaC tools are supported in environment definitions?
 7. How do environment templates integrate with source control?
 8. How do you automate cleanup and lifecycle of deployment environments?
 9. How are cost controls and quotas applied to environments?
 10. What are best practices for using Azure Deployment Environments in enterprise DevOps?
-

1. What are Azure Deployment Environments and what use case do they solve?

They provide on-demand, consistent environments (dev/test/staging) for developers using predefined IaC templates, reducing wait times and manual provisioning.

2. How do Azure Deployment Environments support developer self-service?

Developers can deploy approved templates directly from a portal or CLI without needing contributor rights on subscriptions or waiting for central IT provisioning.

3. What are environment definitions and how are they created?

They are IaC templates (*Bicep* or *ARM*) stored in Git repositories, defining the infrastructure for an environment. Created and managed via *Azure* portal or CLI.

4. How do you configure projects and environments for team use?

- Admins create projects, link *GitHub* repos with definitions, and assign users or groups.
 - Developers then select and deploy environments under that project.
-

5. How is access controlled in Azure Deployment Environments?

Using *Azure* RBAC at the project level—roles like Environment User OR Project Admin control who can deploy, manage, or view environments.

6. What IaC tools are supported in environment definitions?

Bicep and ARM templates are supported. Terraform is currently not supported directly within Azure Deployment Environments as of June 2025.

7. How do environment templates integrate with source control?

- Templates are stored in *GitHub* (public or private repos).
 - The system pulls definitions directly from branches specified during environment setup.
-

8. How do you automate cleanup and lifecycle of deployment environments?

Use environment expiration policies and lifecycle hooks (e.g., cleanup scripts) defined in the template or enforced via policy-as-code in *Azure*.

9. How are cost controls and quotas applied to environments?

- Set resource quotas,
- tag policies,
- and auto-expiration on environments.
- *Azure Cost Management* can be used to track and limit spend per environment.

10. What are best practices for using Azure Deployment Environments in enterprise DevOps?

- Use centralized template governance,
- automate validation,
- enforce least-privilege RBAC,
- tag all resources,
- and integrate environment cleanup into CI/CD workflows.

3. Build and release pipelines

└ 3.7 Maintain and optimize pipelines

└ 3.7.1 Monitor pipeline health (failures,duration)

-
1. What tools in Azure DevOps and GitHub Actions are used to monitor pipeline failures and durations?
 2. How do you configure pipeline run analytics and dashboards in Azure DevOps?
 3. What are the key metrics to track for pipeline health?
 4. How can alerts be set up for failed or long-running pipelines?
 5. How do you investigate and troubleshoot failed pipeline runs?
 6. How can you identify flaky tests or unreliable steps in pipelines?
 7. What practices improve visibility of pipeline duration trends?
 8. How do you optimize notification settings for pipeline health?
 9. What integrations support advanced monitoring (e.g., Azure Monitor, Log Analytics)?
 10. How do you audit historical pipeline performance and failure trends?
-

1. What tools in Azure DevOps and GitHub Actions are used to monitor pipeline failures and durations?

- *Azure DevOps* uses built-in Pipeline Run history, Analytics views, and Dashboards.
 - *GitHub Actions* provides Workflow Run history and summary views. Both support integration with *Azure Monitor* and *Log Analytics*.
-

2. How do you configure pipeline run analytics and dashboards in Azure DevOps?

Use *Azure DevOps* Analytics extension and built-in widgets to display pipeline duration, success rate, and failure trends on Dashboards. Configure custom queries in Analytics views for detailed tracking.

3. What are the key metrics to track for pipeline health?

- Track pipeline failure rate,
 - average and maximum run duration,
 - time to fix failures,
 - frequency of runs,
 - and number of flaky tests or steps.
-

4. How can alerts be set up for failed or long-running pipelines?

- In *Azure DevOps*, configure Alerts under *Project Settings > Notifications* for build or release failures and time thresholds.
 - In *GitHub Actions*, use `workflow_dispatch` or third-party integrations to trigger notifications for failures or delays.
-

5. How do you investigate and troubleshoot failed pipeline runs?

Review detailed run logs, error messages, and failed step outputs. Use build and release timeline views to identify where failures occur and correlate with recent changes or configuration updates.

6. How can you identify flaky tests or unreliable steps in pipelines?

Analyze test run history for intermittent failures.

Use *Azure DevOps* Test Analytics or *GitHub Actions* Test Reporter to flag non-deterministic or frequently failing tests and steps.

7. What practices improve visibility of pipeline duration trends?

- Regularly review analytics dashboards, set up charts for historical run times, and compare against baselines.
 - Visualize trends using built-in or Power BI dashboards.
-

8. How do you optimize notification settings for pipeline health?

Configure notifications to target only relevant teams/users and filter on critical events like failures or abnormal durations. Avoid excessive notifications to reduce alert fatigue.

9. What integrations support advanced monitoring (e.g., Azure Monitor, Log Analytics)?

Connect *Azure DevOps* pipelines to *Azure Monitor* and *Log Analytics* using Diagnostic Settings or REST APIs. Export pipeline metrics for advanced querying, alerting, and dashboarding.

10. How do you audit historical pipeline performance and failure trends?

- Use *Azure DevOps* Analytics views or *Power BI* to query and visualize pipeline history, identifying long-term trends in duration, failure rate, and reliability.
- Archive logs for compliance audits if required.

3. Build and release pipelines

└ 3.7 Maintain and optimize pipelines

└ 3.7.2 Optimize: Cost, Time, Reliability, Performance

-
1. What strategies reduce pipeline execution costs in Azure DevOps and GitHub Actions?
 2. How do you minimize pipeline execution time?
 3. What approaches improve pipeline reliability?
 4. How can parallelism and agent management be used to optimize performance?
 5. What best practices reduce resource consumption in pipelines?
 6. How do you cache dependencies to accelerate builds?
 7. How do you monitor and control agent utilization?
 8. What methods reduce wait times for pipeline approvals and manual interventions?
 9. How do you ensure pipelines are robust against transient infrastructure failures?
 10. How can pipeline task reuse and modularization improve efficiency?
-

1. What strategies reduce pipeline execution costs in Azure DevOps and GitHub Actions?

- Use self-hosted agents for long-running jobs,
 - choose lower-cost hosted agents,
 - minimize unnecessary runs,
 - and clean up unused artifacts and resources after builds.
-

2. How do you minimize pipeline execution time?

- Enable parallel jobs,
 - break pipelines into smaller stages,
 - cache dependencies, run only required tests,
 - and avoid redundant steps.
 - Trigger pipelines only on relevant changes.
-

3. What approaches improve pipeline reliability?

- Implement retries for transient failures,
 - use stable base images and tools,
 - version dependencies,
 - and separate critical and optional tasks.
 - Regularly review and update pipeline tasks.
-

4. How can parallelism and agent management be used to optimize performance?

- Increase parallel jobs,
 - use job dependencies to optimize order,
 - distribute workloads across multiple agents,
 - and use agent pools to avoid bottlenecks.
-

5. What best practices reduce resource consumption in pipelines?

- Remove unused tasks, artifacts, and images.
 - Use selective checkout,
 - optimize scripts,
 - and limit scope of triggered builds and tests to affected components.
-

6. How do you cache dependencies to accelerate builds?

Implement build and task caching (e.g., npm, pip, Maven cache tasks in Azure Pipelines or actions/cache in GitHub Actions) to avoid downloading/building unchanged dependencies.

7. How do you monitor and control agent utilization?

- Track agent pool usage via *Azure DevOps* Analytics or *GitHub Actions* Insights.
 - Scale agents up or down as needed, and prioritize high-value jobs.
-

8. What methods reduce wait times for pipeline approvals and manual interventions?

- Automate approvals where possible,
 - use automatic triggers,
 - set up environment checks,
 - and streamline required manual steps to minimize human delays.
-

9. How do you ensure pipelines are robust against transient infrastructure failures?

- Configure retry logic on failed steps,
 - use resilient cloud infrastructure,
 - and implement fallbacks or safe defaults for network- or resource-related errors.
-

11. How can pipeline task reuse and modularization improve efficiency?

- Use YAML templates, task groups, and reusable workflows to standardize common tasks,
- reduce duplication,
- and simplify maintenance across multiple pipelines.

3. Build and release pipelines

└ 3.7 Maintain and optimize pipelines

└ 3.7.3 Artifact retention strategy

1. What is an artifact retention policy in Azure DevOps?
 2. Where do you configure artifact retention in Azure Pipelines?
 3. What is the default retention policy for pipeline artifacts?
 4. How can you prevent a pipeline run's artifacts from being deleted?
 5. How do retention policies differ between classic and YAML pipelines?
 6. How does pipeline retention impact storage costs and performance?
 7. What permissions are needed to modify retention settings?
 8. How do retention settings affect deployment history and auditability?
 9. How can you override global retention policies per pipeline?
 10. What are best practices for implementing artifact retention strategies?
-

1. What is an artifact retention policy in Azure DevOps?

A retention policy defines how long pipeline artifacts, logs, and test results are kept before being automatically deleted to manage storage usage.

2. Where do you configure artifact retention in Azure Pipelines?

Retention is configured under *Project Settings* → *Pipelines* → *Retention*, or directly in the pipeline YAML using retention under the pipeline block.

3. What is the default retention policy for pipeline artifacts?

The default policy retains artifacts for 30 days or 3 successful runs per branch, whichever is met last.

4. How can you prevent a pipeline run's artifacts from being deleted?

Mark the pipeline run as “retained” either manually via the UI or using `az pipelines runs update --id <runId> --retain true`.

5. How do retention policies differ between classic and YAML pipelines?

Classic pipelines use UI-based retention rules; YAML pipelines can define retention directly in code, allowing more granular, version-controlled policies.

6. How does pipeline retention impact storage costs and performance?

Longer retention increases storage usage and cost; excessive artifact history may degrade performance in large-scale environments.

7. What permissions are needed to modify retention settings?

You need at least *Project Administrator* or *Build Administrator* role to modify global retention settings or mark runs for retention.

8. How do retention settings affect deployment history and auditability?

Deleting artifacts or logs too early can break traceability and hinder auditing of what was deployed, when, and from which source.

9. How can you override global retention policies per pipeline?

In YAML, set retention under the pipeline root, e.g.:

```
retention: { daysToKeep: 10, minimumToKeep: 1, deleteBuildRecord: true }
```

11. What are best practices for implementing artifact retention strategies?

- Use shorter retention for high-frequency pipelines.
- Retain longer for production deployments.
- Mark important releases as retained.
- Periodically audit unused runs.

3. Build and release pipelines

└ 3.7 Maintain and optimize pipelines

└ 3.7.4 Migrate Classic to YAML

-
1. What are the key differences between Classic and YAML pipelines?
 2. Why should you migrate from Classic to YAML pipelines?
 3. How do you export a Classic pipeline to YAML?
 4. Which features in Classic pipelines require manual translation when migrating?
 5. How can task groups be converted for YAML pipelines?
 6. What tools assist in converting Classic to YAML?
 7. What common issues arise when migrating triggers?
 8. How do you structure multi-stage YAML pipelines?
 9. What are best practices when migrating pipelines incrementally?
 10. How do you validate that a migrated YAML pipeline matches original behavior?
-

1. What are the key differences between Classic and YAML pipelines?

- Classic pipelines use a GUI for configuration
 - YAML pipelines are defined as code, versioned with source, and support advanced features like templates and multi-stage workflows.
-

2. Why should you migrate from Classic to YAML pipelines?

- YAML enables pipeline-as-code,
 - better version control,
 - reuse with templates,
 - portability,
 - and full CI/CD traceability.
-

3. How do you export a Classic pipeline to YAML?

Go to the *Classic pipeline* → *Edit* → *View YAML (only available for simple jobs)* → *Copy and manually refine*. Complex flows require manual recreation.

4. Which features in Classic pipelines require manual translation when migrating?

Multi-agent jobs, deployment groups, task groups, approvals, and environment strategies often require re-implementation using YAML stages and environments.

5. How can task groups be converted for YAML pipelines?

Recreate task group logic as YAML templates, then reference with `- template: path/to/template.yml` in pipelines.

6. What tools assist in converting Classic to YAML?

There's no full auto-converter; use "View YAML" for individual jobs, then rely on docs, pipeline schema references, and manual refactoring.

7. What common issues arise when migrating triggers?

Classic pipelines may use UI-based schedules or branch filters that must be manually defined in YAML using trigger and schedules.

8. How do you structure multi-stage YAML pipelines?

Use the stages keyword with each stage defining jobs and steps, enabling CI, CD, approvals, and environment separation in one file.

9. What are best practices when migrating pipelines incrementally?

- Start with CI stages, validate output, then add CD stages.
 - Keep Classic as fallback until YAML proves stable.
 - Use templates for reuse.
-

11. How do you validate that a migrated YAML pipeline matches original behavior?

- Compare build logs, test results, deployment targets, and approval flows side-by-side.
- Use test branches to validate before cutting over.

4. Security and compliance

└ 4.1 Authentication and authorization

└ 4.1.1 Choose: Service Principals, Managed Identity

1. What is a Service Principal in Azure and when should it be used?
 2. What is a Managed Identity and what types are available?
 3. What are the main differences between Service Principals and Managed Identities?
 4. In which scenarios is a Service Principal preferred over Managed Identity?
 5. When should you use a Managed Identity instead of a Service Principal?
 6. How are credentials managed for Service Principals versus Managed Identities?
 7. What are the security best practices for Service Principals?
 8. What are the security advantages of Managed Identities?
 9. How do you grant permissions to a Service Principal and a Managed Identity?
 10. What is the process to rotate credentials for Service Principals and Managed Identities?
-

1. What is a Service Principal in Azure and when should it be used?

A Service Principal is an identity created for use

- with applications,
- hosted services,
- and automation tools

to access Azure resources. Use it when you need an application or script to authenticate and operate independently of user context.

2. What is a Managed Identity and what types are available?

An automatically managed identity in Azure AD for Azure resources. Two types exist:

- *system-assigned* (tied to a single resource)
 - and *user-assigned* (independent, reusable across resources).
-

3. What are the main differences between Service Principals and Managed Identities?

- Service Principals require manual credential management (client secrets or certificates) and can be used outside Azure.
 - Managed Identities have automatic credential rotation, no secret exposure, and only work within supported Azure services.
-

4. In which scenarios is a Service Principal preferred over Managed Identity?

Use a Service Principal

- when accessing Azure resources from outside Azure,
 - in cross-cloud or on-premises scenarios,
 - or when granular control over authentication lifecycle is required.
-

5. When should you use a Managed Identity instead of a Service Principal?

Use a Managed Identity when your workload runs entirely on Azure and needs secure, automated authentication to Azure resources—eliminating secret management.

6. How are credentials managed for Service Principals versus Managed Identities?

- Service Principals require explicit secret/certificate creation, storage, and rotation.
 - Managed Identities handle credential mgmt. & rotation automatically, exposing no secrets.
-

7. What are the security best practices for Service Principals?

- Use the least-privilege principle,
 - regularly rotate secrets/certificates,
 - use certificate-based authentication over client secrets,
 - restrict access with conditional access policies,
 - and store secrets in Azure Key Vault.
-

8. What are the security advantages of Managed Identities?

Managed Identities

- remove the need to handle credentials,
 - provide automatic credential rotation,
 - and limit exposure of secrets,
 - reducing the risk of leaks or misuse.
-

9. How do you grant permissions to a Service Principal and a Managed Identity?

Assign the required Azure RBAC role to the Service Principal or Managed Identity at the correct scope (subscription, resource group, or resource level) via the Azure portal, CLI, or PowerShell.

10. What is the process to rotate credentials for Service Principals and Managed Identities?

For Service Principals, manually update secrets or certificates and update applications to use the new credentials. For Managed Identities, Azure handles rotation automatically—no manual action required.

4. Security and compliance

└ 4.1 Authentication and authorization

└ 4.1.2 GitHub auth: Apps, tokens

-
1. What are the primary authentication methods available for GitHub automation?
 2. What is a GitHub App and when should it be used?
 3. What is a personal access token (PAT) and what is it used for?
 4. What is the GITHUB_TOKEN and how is it used in GitHub Actions?
 5. How do you control the permissions of a GitHub App versus a PAT?
 6. When should you use GITHUB_TOKEN instead of a PAT in GitHub Actions?
 7. How do you authenticate to GitHub in CI/CD pipelines securely?
 8. What are the risks associated with PATs and how can you mitigate them?
 9. How do you revoke access for GitHub Apps, PATs, and GITHUB_TOKEN?
 10. What auditing capabilities exist for GitHub authentication methods?
-

1. What are the primary authentication methods available for GitHub automation?

The main methods are

- GitHub Apps,
 - personal access tokens (PATs),
 - and the built-in GITHUB_TOKEN used by GitHub Actions.
-

2. What is a GitHub App and when should it be used?

A GitHub App is an integration with granular permissions, installed directly on repositories or organizations. Use it for automation, CI/CD, and integrations needing controlled, auditable access.

3. What is a personal access token (PAT) and what is it used for?

A PAT is a user-generated token that grants API access on behalf of a user. It is used for scripting, CLI tools, and third-party integrations where user-based access is needed.

4. What is the GITHUB_TOKEN and how is it used in GitHub Actions?

GITHUB_TOKEN is an automatically generated token available to workflows, scoped to the repository and job. It allows secure automation within Actions without manual secret handling.

5. How do you control the permissions of a GitHub App versus a PAT?

- GitHub Apps have fine-grained, configurable permissions per repository and event.
 - PATs inherit the full or partial permissions of the user who created them.
-

6. When should you use GITHUB_TOKEN instead of a PAT in GitHub Actions?

Use GITHUB_TOKEN for most automation in Actions as it is automatically created, limited in scope, and auto-rotated—minimizing risk of credential leakage.

7. How do you authenticate to GitHub in CI/CD pipelines securely?

Use GITHUB_TOKEN for native Actions, GitHub App tokens for granular access, and store any PATs in encrypted secrets—never hardcode credentials.

8. What are the risks associated with PATs and how can you mitigate them?

PATs are long-lived and may have broad access if leaked.

- Limit their scope,
- rotate regularly,
- use only when required,
- and store securely in GitHub Secrets.

9. How do you revoke access for GitHub Apps, PATs, and GITHUB_TOKEN?

- Uninstall the GitHub App,
- delete or regenerate the PAT from user settings,
- and GITHUB_TOKEN is invalidated automatically at job end.

11. What auditing capabilities exist for GitHub authentication methods?

Audit

- logs track PAT usage,
- GitHub App installation and actions,
- and GITHUB_TOKEN workflow invocations.

Review logs for unauthorized or unusual access.

4. Security and compliance

└ 4.1 Authentication and authorization

└ 4.1.3 Azure DevOps service connections, tokens

1. What is a service connection in Azure DevOps?
 2. What types of service connections can you configure?
 3. How do you securely store and manage service connection credentials?
 4. What is the difference between service principals, managed identities, and PATs in Azure DevOps?
 5. How do you configure a new Azure Resource Manager (ARM) service connection?
 6. What permissions are required to create and use service connections?
 7. What are personal access tokens (PATs) and when should they be used?
 8. What are best practices for managing and rotating tokens in Azure DevOps?
 9. How can you audit usage and security of service connections and tokens?
 10. How do you restrict access to service connections in a project or pipeline?
-

1. What is a service connection in Azure DevOps?

A service connection is a secure configuration in *Azure DevOps* that enables pipelines to connect and authenticate with external systems or Azure resources, using credentials or identity.

2. What types of service connections can you configure?

Common types include

- Azure Resource Manager,
- GitHub,
- Docker Registry,
- Generic,
- AWS,
- GCP,
- and Service Fabric.

Each targets a specific external service or cloud.

3. How do you securely store and manage service connection credentials?

Credentials are encrypted and stored securely in *Azure DevOps*. Sensitive details (like secrets or certificates) are not exposed in pipelines and are managed by *Azure DevOps* security controls.

4. What is the difference between service principals, managed identities, and PATs in Azure DevOps?

- **Service principals:** Used for non-interactive authentication to Azure resources; recommended for ARM connections.
 - **Managed identities:** Azure AD identities for Azure services; used for enhanced security and automation within Azure.
 - **PATs (Personal Access Tokens):** User-scoped tokens for accessing *Azure DevOps* REST APIs; not recommended for automation or service connections due to security risks.
-

5. How do you configure a new Azure Resource Manager (ARM) service connection?

Go to *Project Settings* → *Service Connections* → *New service connection* → *Select Azure Resource Manager* → *Authenticate (typically with a service principal)* → *Grant permissions* → *Save*.

6. What permissions are required to create and use service connections?

- Creating: *Project Administrator* or *Service Connection Admin*.
 - Using: Grant access to specific users, groups, or pipelines; restrict to those who need it.
-

7. What are personal access tokens (PATs) and when should they be used?

PATs are user-generated tokens for authenticating to Azure DevOps REST APIs. Use only for short-term, user-scoped automation; avoid for production pipelines or shared services.

8. What are best practices for managing and rotating tokens in Azure DevOps?

- Use service principals or managed identities over PATs.
 - Set expiration dates on tokens.
 - Regularly review and rotate credentials.
 - Revoke unused or suspicious tokens promptly.
-

9. How can you audit usage and security of service connections and tokens?

Use *Azure DevOps* Audit Logs to track creation, modification, and usage of service connections and PATs. Review access and permission changes regularly.

10. How do you restrict access to service connections in a project or pipeline?

Set security on each service connection by configuring user/group permissions or restricting pipeline usage. Use the “Limit job authorization” and “Grant access permission to all pipelines” settings appropriately.

4. Security and compliance

└ 4.1 Authentication and authorization

└ 4.1.4 Permissions/Roles in GitHub

1. What are the main types of roles in a GitHub repository?
 2. What permissions does each role have in GitHub?
 3. How do you assign and manage roles in a GitHub repository?
 4. What is the difference between a repository collaborator and a team member?
 5. How do you restrict branch access and actions in GitHub?
 6. What is the purpose of code owners and how do they affect permissions?
 7. How do organization-level permissions differ from repository-level permissions?
 8. What are outside collaborators, and how are they managed?
 9. What best practices should be followed for managing GitHub permissions?
 10. How do you audit and review repository access in GitHub?
-

1. What are the main types of roles in a GitHub repository?

Owner, Admin, Maintainer, Write (Contributor), Triage, Read, and Outside Collaborator.

2. What permissions does each role have in GitHub?

- **Owner:** Full control over organization and all repositories.
 - **Admin:** Full control over repository settings and content.
 - **Maintainer:** Manages teams, can triage issues/PRs.
 - **Write:** Push code, create branches, manage issues/PRs.
 - **Triage:** Manage issues/PRs (label, assign), but cannot write code.
 - **Read:** View content, clone repo, open issues/PRs.
 - **Outside Collaborator:** Limited to explicit permissions given.
-

3. How do you assign and manage roles in a GitHub repository?

Go to *Repository* → *Settings* → *Manage Access*. Add users/teams and assign appropriate role.

4. What is the difference between a repository collaborator and a team member?

- A collaborator is granted access directly to a repository, typically outside the organization.
 - A team member is part of an organization team, with permissions managed at the team level.
-

5. How do you restrict branch access and actions in GitHub?

Configure branch protection rules in repository settings:

- restrict pushes,
 - require PR reviews,
 - enforce status checks,
 - and limit who can merge.
-

6. What is the purpose of code owners and how do they affect permissions?

Code owners are responsible for specific files or directories. Their review is required before changes to owned code can be merged, enforcing accountability.

7. How do organization-level permissions differ from repository-level permissions?

- *Organization-level permissions* apply across all repos and include actions like inviting members and managing billing.
 - *Repository-level permissions* control access to specific repos only.
-

8. What are outside collaborators, and how are they managed?

Outside collaborators are users not part of the org but given access to specific repos. Managed by adding/removing their access in repo settings.

9. What best practices should be followed for managing GitHub permissions?

- Use least privilege principle.
 - Prefer team-based over individual permissions.
 - Regularly review and audit access.
 - Use branch protections and required reviews.
-

11. How do you audit and review repository access in GitHub?

- Use the "Manage Access" tab to review users and roles.
- For organizations, use GitHub's audit log and periodic access reviews.

4. Security and compliance

└ 4.2 Permissions and access

└ 4.2.1 Azure DevOps permissions, security groups

1. What are the default security groups in Azure DevOps?
 2. How do permissions inheritance and explicit overrides work in Azure DevOps?
 3. How do you assign and manage permissions at organization, project, and object level?
 4. What is the difference between 'Allow', 'Deny', and 'Not set' for permissions?
 5. How do you grant least privilege access in Azure DevOps?
 6. How are access levels different from security group permissions?
 7. What are best practices for configuring permissions in Azure DevOps?
 8. How can you audit and review effective permissions for a user or group?
 9. What steps are required to restrict branch permissions or pipeline execution?
 10. How do you manage external access (e.g., Stakeholders, outside collaborators)?
-

1. What are the default security groups in Azure DevOps?

Azure DevOps default security groups include *Project Administrators*, *Contributors*, *Readers*, *Build Administrators*, and *Project Collection Administrators*. Each group grants a specific set of permissions appropriate for common roles.

2. How do permissions inheritance and explicit overrides work in Azure DevOps?

Permissions are inherited from higher-level containers (organization or project) to lower-level resources (repos, pipelines, etc). Explicitly setting a permission ('Allow' or 'Deny') at a lower level overrides inherited permissions.

3. How do you assign and manage permissions at organization, project, and object level?

Navigate to *Organization/Project settings > Security*. Assign users/groups to built-in or custom security groups, then set permissions at the organization, project, or resource (repo, pipeline) level.

4. What is the difference between 'Allow', 'Deny', and 'Not set' for permissions?

'Allow' grants the permission, 'Deny' blocks it (overrides all 'Allow'), and 'Not set' means the permission is neither granted nor denied, so inheritance applies.

5. How do you grant least privilege access in Azure DevOps?

Add users to the group with the minimal permissions needed. Avoid granting *Project Administrator* unless required, and restrict direct assignments.

6. How are access levels different from security group permissions?

- Access levels (Stakeholder, Basic, Visual Studio, etc.) control which features are available.
 - Security group permissions define what users can do within those features.
-

7. What are best practices for configuring permissions in Azure DevOps?

Use built-in groups for common roles, apply 'Deny' sparingly, review permissions regularly, and use custom groups only when necessary.

8. How can you audit and review effective permissions for a user or group?

Use the Permissions tab under the security settings of a resource. Select a user/group to view their effective permissions, including inherited and explicitly set rights.

9. What steps are required to restrict branch permissions or pipeline execution?

Go to the repository or pipeline, select Security, and set branch or pipeline permissions for specific users/groups (e.g., restrict force push or approve builds).

10. How do you manage external access (e.g., Stakeholders, outside collaborators)?

Assign external users to Stakeholder access or limit to specific security groups. Review permissions to ensure no unnecessary access is granted beyond required scope.

4. Security and compliance

└ 4.2 Permissions and access

└ 4.2.2 Access levels: stakeholders, collaborators

1. What are the main access levels available in Azure DevOps?
 2. What capabilities are included with Stakeholder access?
 3. How do access levels differ from security group permissions?
 4. Who should be assigned the Stakeholder access level?
 5. What are the limitations of Stakeholder access compared to Basic?
 6. How do you manage access levels for users in Azure DevOps?
 7. What is the recommended approach for granting outside collaborator access?
 8. What restrictions apply to outside collaborators in Azure DevOps?
 9. How can you audit and review access levels across an organization?
 10. What are best practices for assigning access levels in Azure DevOps?
-

1. What are the main access levels available in Azure DevOps?

- Stakeholder,
 - Basic,
 - and Visual Studio Subscriber.
-

2. What capabilities are included with Stakeholder access?

- Stakeholders can
 - view & edit work items,
 - create & manage backlogs and boards,
 - and run some queries.
 - They cannot access code, pipelines, or advanced features.
-

3. How do access levels differ from security group permissions?

Access levels determine feature visibility and availability; security groups control specific permissions within those features.

4. Who should be assigned the Stakeholder access level?

Users who need visibility into project work but do not require code or pipeline access, such as business analysts, sponsors, and some external partners.

5. What are the limitations of Stakeholder access compared to Basic?

Stakeholders cannot contribute code, manage pipelines, view test plans, or access advanced reporting and dashboards.

6. How do you manage access levels for users in Azure DevOps?

Go to *Organization Settings > Users*, select the user, and assign the appropriate access level (Stakeholder, Basic, etc.).

7. What is the recommended approach for granting outside collaborator access?

Invite external users with the minimum required access level, typically as Stakeholders or limited Basic users, and restrict security group membership as needed.

8. What restrictions apply to outside collaborators in Azure DevOps?

- They may have limited access to organization-wide resources,
 - cannot be assigned certain licenses,
 - and may be restricted by policies (e.g., limited to specific projects).
-

9. How can you audit and review access levels across an organization?

In *Organization Settings > Users*, filter or export the user list to review assigned access levels and license usage.

11. What are best practices for assigning access levels in Azure DevOps?

- Assign the lowest access level that meets business needs,
- review user assignments regularly,
- and update access as roles or requirements change.

4. Security and compliance

└ 4.2 Permissions and access

└ 4.2.3 Project/team config in Azure DevOps

1. How do you create a new project in Azure DevOps?
 2. What are teams in Azure DevOps and why are they used?
 3. How do you add users to a project or team?
 4. What is the difference between project and team-level security?
 5. How do you configure team backlogs and boards?
 6. How do you manage team areas and iterations?
 7. How can you restrict visibility of repositories or pipelines within a project?
 8. What are the steps to rename or delete a project or team?
 9. How do you set default teams or manage multiple teams within a project?
 10. What are best practices for organizing teams and projects in Azure DevOps?
-

1. How do you create a new project in Azure DevOps?

Go to *Azure DevOps portal* > *New Project* > Enter name, description, visibility, and process, then create.

2. What are teams in Azure DevOps and why are they used?

Teams are subgroups within a project that enable independent backlog, board, and sprint planning. They help scale agile practices for large projects.

3. How do you add users to a project or team?

In *Project Settings* > *Teams*, select a team > Add Members.

For project-level access, use *Project Settings* > *Permissions* or *Organization Settings* > *Users*.

4. What is the difference between project and team-level security?

- Project-level security controls access to resources across the entire project;
 - team-level settings affect only the team's artifacts (backlogs, boards, etc.).
-

5. How do you configure team backlogs and boards?

In *Team Settings*, define areas and iterations for the team, then customize columns, swimlanes, and filters for boards as needed.

6. How do you manage team areas and iterations?

Assign work item areas and sprints (iterations) in *Project Settings* > *Project configuration*, then map them to teams via *Team Settings*.

7. How can you restrict visibility of repositories or pipelines within a project?

Use *Project Settings* > *Repositories/Pipelines* > *Security* to set permissions for teams or users, controlling access per repo or pipeline.

8. What are the steps to rename or delete a project or team?

- For projects: *Organization Settings* > *Projects* > *Select project* > *Rename or Delete*.
 - For teams: *Project Settings* > *Teams* > *Select team* > *Edit or Delete*.
-

9. How do you set default teams or manage multiple teams within a project?

Each project has a default team (created with the project); more teams can be added via *Project Settings* > *Teams*. You can set team defaults for areas and iterations.

11. What are best practices for organizing teams and projects in Azure DevOps?

- Align teams to delivery units (feature, component, cross-functional),
- avoid unnecessary project sprawl,
- keep team memberships current,
- and regularly review permissions and structure.

4. Security and compliance

└ 4.3 Secret and sensitive data management

└ 4.3.1 Azure Key Vault secrets/keys/certs

1. What are the types of objects stored in Azure Key Vault?
 2. How do you create and manage secrets in Azure Key Vault?
 3. What is the difference between keys and secrets in Key Vault?
 4. How do you control access to Key Vault using RBAC vs. access policies?
 5. How do you integrate Azure Key Vault with GitHub Actions or Azure Pipelines?
 6. How are secrets retrieved securely in CI/CD pipelines?
 7. What are best practices for rotating secrets and certificates?
 8. How is soft-delete and purge protection used in Key Vault?
 9. How do managed identities access Azure Key Vault?
 10. How do you audit access and usage of Azure Key Vault?
-

1. What are the types of objects stored in Azure Key Vault?

- Secrets (e.g., passwords, tokens),
 - Keys (RSA/EC keys),
 - and Certificates (with optional auto-renewal).
-

2. How do you create and manage secrets in Azure Key Vault?

Use Azure CLI:

```
az keyvault secret set --vault-name <name> --name <secretName> --value <value>
```

You can also manage via Azure Portal, REST API, or ARM/Bicep templates.

3. What is the difference between keys and secrets in Key Vault?

- **Secrets:** Store plain strings (e.g., passwords).
 - **Keys:** Used for cryptographic operations (encrypt, sign) and can't be exported.
-

4. How do you control access to Key Vault using RBAC vs. access policies?

- **RBAC:** Assign roles like "*Key Vault Secrets User*" at Azure resource level.
 - **Access policies:** Grant specific permissions (e.g., get, list) directly in Key Vault settings. RBAC is recommended for new deployments.
-

5. How do you integrate Azure Key Vault with GitHub Actions or Azure Pipelines?

Use AzureLogin action or service connection to authenticate, then access secrets via:

```
az keyvault secret show in a script step or with built-in AzureKeyVault@2 task.
```

6. How are secrets retrieved securely in CI/CD pipelines?

Use secure service connections and managed identities. Avoid hardcoding secrets; pull them at runtime using secure pipeline tasks (e.g., *AzureKeyVault@2* or *azure/login* + CLI).

7. What are best practices for rotating secrets and certificates?

- Enable expiration dates.
- Use Event Grid with automation (e.g., *Logic Apps*, *Azure Functions*) for rotation.
- For certificates, enable auto-renew with integrated CAs.

8. How is soft-delete and purge protection used in Key Vault?

- **Soft-delete:** Keeps deleted items for a retention period (default 90 days).
 - **Purge protection:** Prevents permanent deletion until retention expires. Both are essential for regulatory compliance.
-

9. How do managed identities access Azure Key Vault?

Assign the identity a role (RBAC) or access policy in *Key Vault* with required permissions. No credentials are needed—Azure handles auth transparently.

10. How do you audit access and usage of Azure Key Vault?

Enable diagnostic settings to send logs to *Log Analytics*, *Storage*, or *Event Hubs*. Monitor events like *SecretGet*, *KeySign* using *Azure Monitor* or *KQL*.

4. Security and compliance

└ 4.3 Secret and sensitive data management

└ 4.3.2 Secrets in GitHub Actions/Azure Pipelines

1. How are secrets stored and accessed in GitHub Actions?
 2. What are environment secrets in GitHub Actions and how do they differ from repository secrets?
 3. How do you securely use secrets in GitHub Actions workflows?
 4. How do you define and use secrets in Azure Pipelines?
 5. What are secure files in Azure Pipelines and when should you use them?
 6. What are common mistakes that expose secrets in CI/CD workflows?
 7. How do you use variable groups to manage secrets across pipelines in Azure DevOps?
 8. What permissions are required to access secrets in GitHub and Azure Pipelines?
 9. How can secrets be injected into runtime containers securely in pipelines?
 10. What are the best practices for managing secret lifecycle in DevOps pipelines?
-

1. How are secrets stored and accessed in GitHub Actions?

Secrets are added under *Settings > Secrets and variables > Actions*. Access them in workflows using `${{ secrets.SECRET_NAME }}`.

2. What are environment secrets in GitHub Actions and how do they differ from repository secrets?

Environment secrets are tied to specific deployment environments and support protection rules.

Repository secrets apply globally to all workflows.

3. How do you securely use secrets in GitHub Actions workflows?

Use `${{ secrets.NAME }}` in workflow YAML. Never echo secrets or log them. Use `secrets.*` only within run blocks or secure inputs.

4. How do you define and use secrets in Azure Pipelines?

Define secrets in variable groups or pipeline variables (`isSecret: true`). Access them in scripts via `$(secretName)` or environment variables.

5. What are secure files in Azure Pipelines and when should you use them?

Secure files are encrypted files (e.g., certificates) uploaded via the UI or API. Use them when pipelines need non-text secrets like PFX or SSH keys.

6. What are common mistakes that expose secrets in CI/CD workflows?

- Echoing secrets to logs
 - Using secrets in non-secure scripts
 - Committing secrets to source control
 - Disabling scrubbers or secret masking
-

7. How do you use variable groups to manage secrets across pipelines in Azure DevOps?

Create a variable group, mark secrets as secret, and link the group to pipelines. Requires linking authorization for YAML usage.

8. What permissions are required to access secrets in GitHub and Azure Pipelines?

- GitHub: Write access to secrets and workflow access permissions
- Azure DevOps: Pipeline or service connection must have access to variable groups/secure files

9. How can secrets be injected into runtime containers securely in pipelines?

Pass them as environment variables or mount secure files.

In GitHub, use env: block; in Azure, use environment: or script variables.

10. What are the best practices for managing secret lifecycle in DevOps pipelines?

- Rotate secrets regularly
- Use Key Vault or GitHub Environments for sensitive scopes
- Avoid hardcoding; retrieve secrets dynamically
- Audit and clean unused secrets periodically

4. Security and compliance

└ 4.3 Secret and sensitive data management

└ 4.3.3 Secure files during deployment

1. What are secure files in Azure Pipelines and what are common use cases?
 2. How do you upload secure files to Azure Pipelines?
 3. How are secure files used in pipeline steps?
 4. What permissions are required to access secure files?
 5. How does Azure Pipelines handle encryption and storage of secure files?
 6. What are best practices for managing certificates or signing keys securely in deployment pipelines?
 7. Can secure files be used in GitHub Actions?
 8. How can you audit access to secure files in Azure DevOps?
 9. What is the difference between pipeline secrets and secure files?
 10. How do you prevent accidental exposure of sensitive files during deployment?
-

1. What are secure files in Azure Pipelines and what are common use cases?

Secure files are encrypted files (e.g., certificates, provisioning profiles, signing keys) stored securely in Azure Pipelines.

Use cases: iOS provisioning profiles, Android keystore files, SSL certs, signing keys.

2. How do you upload secure files to Azure Pipelines?

Navigate to:

Pipelines → Library → Secure files → "+ Secure file" → Upload and save.

Files are encrypted at rest.

3. How are secure files used in pipeline steps?

Use the DownloadSecureFile@1 task to access them during runtime.

Example:

```
- task: DownloadSecureFile@1
  inputs: secureFile: 'cert.pfx'
```

4. What permissions are required to access secure files?

Users must have "Download secure files" permission for the pipeline or release.

Access is restricted via Library security roles.

5. How does Azure Pipelines handle encryption and storage of secure files?

Files are encrypted with AES-256 at rest and decrypted only in the pipeline execution context. Files are temporary and auto-deleted post-job.

6. What are best practices for managing certificates or signing keys securely in deployment pipelines?

- Store certs as secure files
- Restrict access via RBAC
- Use per-environment scopes
- Rotate keys regularly
- Never log or output cert data

7. Can secure files be used in GitHub Actions?

No native secure file equivalent in GitHub. Use base64-encoded secrets or store encrypted files in repo and decrypt during workflow using stored secrets.

8. How can you audit access to secure files in Azure DevOps?

Review audit logs in *Azure DevOps* under *Organization Settings* → *Auditing*. Logs include who uploaded, downloaded, or deleted secure files.

9. What is the difference between pipeline secrets and secure files?

- **Secrets:** key-value strings (e.g., API keys, passwords)
 - **Secure files:** binary files (e.g., certificates)
- Both are encrypted but serve different deployment needs.
-

10. How do you prevent accidental exposure of sensitive files during deployment?

- Use secure file tasks (never inline file content)
- Disable verbose logging
- Never echo secrets or paths
- Restrict debug mode
- Run deployments in isolated environments

4. Security and compliance

└ 4.3 Secret and sensitive data management

└ 4.3.4 Prevent sensitive data leakage in pipelines

1. What are the primary sources of sensitive data leakage in pipelines?
 2. How do you securely manage secrets in Azure Pipelines?
 3. How do you securely manage secrets in GitHub Actions?
 4. What logging configurations help avoid data exposure?
 5. How can environment variables be secured in pipelines?
 6. How do you enforce secret masking in Azure Pipelines?
 7. What are best practices for reviewing pipeline YAML for leaks?
 8. How does GitHub detect and prevent secret exposure automatically?
 9. What tools can scan pipelines for secrets or credentials?
 10. How do you ensure sensitive data is not logged by custom scripts or tools?
-

1. What are the primary sources of sensitive data leakage in pipelines?

- Hardcoded secrets,
 - unmasked output logs,
 - exposed environment variables,
 - misconfigured scripts,
 - or use of unsecured third-party tools.
-

2. How do you securely manage secrets in Azure Pipelines?

Use Library variable groups with secret variables, or link *Azure Key Vault*. Access them via `$(secretName)` and ensure secret masking is enabled.

3. How do you securely manage secrets in GitHub Actions?

Store secrets in *Settings* → *Secrets and variables* → *Actions*. Access using `${{ secrets.MY_SECRET }}`. They're automatically masked in logs.

4. What logging configurations help avoid data exposure?

- Disable debug logging when not needed,
 - avoid echo on secret variables,
 - and never write secrets to stdout or log files.
-

5. How can environment variables be secured in pipelines?

- Use runtime injection from secret stores, never hardcode.
 - In *Azure*, define as secret variables.
 - In *GitHub*, use secrets and avoid exporting them to global env.
-

6. How do you enforce secret masking in Azure Pipelines?

Mark variables as “*Keep this value secret*” when defining. *Azure* auto-masks them in logs if exact string match is found during execution.

7. What are best practices for reviewing pipeline YAML for leaks?

- Scan for hardcoded strings resembling keys/passwords
 - Use templates to centralize secret usage
 - Avoid inline scripts containing credentials
-

8. How does GitHub detect and prevent secret exposure automatically?

GitHub Advanced Security performs **secret scanning**. If a known pattern (e.g., AWS keys) is pushed, it alerts and can revoke the secret via partner integrations.

9. What tools can scan pipelines for secrets or credentials?

- GitHub Advanced Security (Code/secret scanning)
 - Microsoft Defender for DevOps
 - Open-source tools: truffleHog, gitleaks
-

10. How do you ensure sensitive data is not logged by custom scripts or tools?

- Sanitize script output,
- use set +x in bash,
- avoid printing env variables,
- redirect logs,
- and validate tools do not expose variables implicitly.

4. Security and compliance

└ 4.4 Security and compliance scanning

└ 4.4.1 Dependency/code/secret/license scanning

-
1. What tools are commonly used for dependency scanning in Azure DevOps and GitHub?
 2. How does GitHub Dependabot work and what vulnerabilities can it detect?
 3. What is CodeQL and how is it integrated into GitHub Actions?
 4. How do you configure secret scanning in GitHub repositories?
 5. What license compliance features does GitHub provide?
 6. How do you enable and enforce security scanning policies in GitHub Advanced Security?
 7. How can Microsoft Defender for DevOps be integrated for scanning in Azure DevOps?
 8. What are the differences between static code analysis and secret scanning?
 9. How are vulnerabilities tracked and managed after detection in GitHub?
 10. What best practices ensure effective and compliant security/compliance scanning pipelines?
-

1. What tools are commonly used for dependency scanning in Azure DevOps and GitHub?

- *GitHub* uses *Dependabot* and *GitHub Advanced Security*;
 - *Azure DevOps* relies on integrations like *Microsoft Defender for DevOps* and third-party tools like *WhiteSource* or *Snyk*.
-

2. How does GitHub Dependabot work and what vulnerabilities can it detect?

Dependabot scans dependency files (e.g., *package.json*, *requirements.txt*) for known CVEs and suggests automatic PRs with updated versions. It uses the *GitHub Advisory Database*.

3. What is CodeQL and how is it integrated into GitHub Actions?

CodeQL performs static code analysis to find security vulnerabilities. It runs as a *GitHub Action* and supports scheduled or PR-triggered scans across multiple languages.

4. How do you configure secret scanning in GitHub repositories?

Enable it under *repository settings* → *Security* → *Code security and analysis*. *GitHub* scans commits and PRs for credential patterns and alerts repo admins.

5. What license compliance features does GitHub provide?

GitHub scans dependencies to detect license types and highlight potential violations. License metadata is shown in dependency graphs; violations can trigger *Dependabot* alerts.

6. How do you enable and enforce security scanning policies in GitHub Advanced Security?

Enable Advanced Security per repository. Configure required status checks (e.g., *CodeQL*, secret scanning) in branch protection rules to block merging if checks fail.

7. How can Microsoft Defender for DevOps be integrated for scanning in Azure DevOps?

Connect via *Microsoft Defender for Cloud* → *DevOps Security* → *Connect Azure DevOps*. It enables dependency, IaC, and container scanning and centralizes findings in Defender.

8. What are the differences between static code analysis and secret scanning?

Static analysis (e.g., *CodeQL*) finds logic flaws in code structure; secret scanning searches for exposed credentials and tokens in code and commits.

9. How are vulnerabilities tracked and managed after detection in GitHub?

Vulnerabilities appear in the *Security tab* → *Dependabot alerts*. You can view affected versions, upgrade recommendations, and dismiss alerts with a justification.

10. What best practices ensure effective and compliant security/compliance scanning pipelines?

Use CI-integrated scanning (e.g., *CodeQL*, *secret scan*), auto-fix with *Dependabot*, enforce branch protections, monitor security dashboards, and integrate alerting with workflows.

4. Security and compliance

└ 4.4 Security and compliance scanning

└ 4.4.2 Microsoft Defender for DevOps Security

1. What is Microsoft Defender for DevOps and what problems does it solve?
 2. How do you onboard Azure DevOps and GitHub into Microsoft Defender for DevOps?
 3. What types of security risks does Defender for DevOps detect?
 4. Where are Defender for DevOps findings surfaced and managed?
 5. How is remediation workflow handled for vulnerabilities in Defender for DevOps?
 6. How does Defender for DevOps integrate with GitHub Advanced Security?
 7. What permissions are required to connect GitHub or Azure DevOps to Defender for DevOps?
 8. How do you enable continuous monitoring across multiple repositories?
 9. How can you enforce policies based on Defender for DevOps findings?
 10. What best practices ensure effective implementation of Defender for DevOps?
-

1. What is Microsoft Defender for DevOps and what problems does it solve?

It's a cloud-native security tool that integrates with *Azure DevOps* and *GitHub* to detect

- misconfigurations,
- exposed secrets,
- vulnerable dependencies,
- and infrastructure risks

across the DevOps pipeline.

2. How do you onboard Azure DevOps and GitHub into Microsoft Defender for DevOps?

In *Microsoft Defender for Cloud*, go to *DevOps Security* → *Environment Settings* → *Add DevOps platform*. Authenticate and connect your *Azure DevOps* org or *GitHub* repo via OAuth or service connection.

3. What types of security risks does Defender for DevOps detect?

It identifies

- exposed secrets,
 - dependency vulnerabilities,
 - IaC misconfigurations,
 - weak configurations in workflows,
 - and usage of unapproved tools or practices.
-

4. Where are Defender for DevOps findings surfaced and managed?

Findings are centralized in *Microsoft Defender for Cloud* under *DevOps Security* and *Security Recommendations*. Integration with *Azure Security Center* enables correlation with broader cloud risks.

5. How is remediation workflow handled for vulnerabilities in Defender for DevOps?

Defender provides actionable remediation steps and enables exporting findings to *Azure Boards*, *GitHub issues*, or third-party ticketing systems. Findings can be prioritized by severity.

6. How does Defender for DevOps integrate with GitHub Advanced Security?

Defender imports results from *GitHub Advanced Security* (CodeQL, secret scanning, dependency scanning) to enrich risk context and provide a unified dashboard for security posture.

7. What permissions are required to connect GitHub or Azure DevOps to Defender for DevOps?

You need Owner/admin privileges on the *GitHub* org or *Azure DevOps* project. OAuth permissions must include access to metadata, repos, and workflows. *Defender* must be granted permission to scan code.

8. How do you enable continuous monitoring across multiple repositories?

Connect the org level (not just individual repos), enable auto-discovery of new repos, and configure scanning rules and policies to apply by default across all new projects.

9. How can you enforce policies based on Defender for DevOps findings?

Integrate with *Azure Policy* or *GitHub* branch protection rules. Use conditional access policies or block deployments based on unresolved critical issues via pipeline gates.

10. What best practices ensure effective implementation of Defender for DevOps?

Onboard all repos/orgs, enable integration with *GitHub Advanced Security*, triage alerts regularly, automate issue tracking, review pipeline permissions, and monitor via centralized dashboards.

4. Security and compliance

└ 4.4 Security and compliance scanning

└ 4.4.3 Github advanced security

1. What features are included in GitHub Advanced Security (GHAS)?
 2. How do you enable GHAS for a GitHub repository or organization?
 3. What is CodeQL in GHAS and how does it function?
 4. How does GHAS handle secret scanning and what are its capabilities?
 5. How is dependency scanning managed within GHAS?
 6. What is the difference between alerts and security overview in GHAS?
 7. How do you enforce GHAS checks in branch protection rules?
 8. What are the licensing and billing requirements for GHAS?
 9. How does GHAS integrate with CI/CD workflows and GitHub Actions?
 10. What are best practices for managing GHAS alerts and findings?
-

1. What features are included in GitHub Advanced Security (GHAS)?

GHAS includes

- CodeQL (static analysis),
- secret scanning,
- and dependency review.

It enhances repo security with

- alerts,
 - PR checks,
 - and security dashboards.
-

2. How do you enable GHAS for a GitHub repository or organization?

In *repo settings* → *Security & analysis*, enable features like Code scanning and Secret scanning.

Organization admins must assign a GHAS license to the repo.

3. What is CodeQL in GHAS and how does it function?

CodeQL scans source code for vulnerabilities using semantic queries. It's configured via *GitHub Actions* and runs on PRs, pushes, or scheduled intervals.

4. How does GHAS handle secret scanning and what are its capabilities?

Secret scanning detects credentials (e.g., API keys, tokens) in pushes and PRs. Alerts are triggered, and some partner secrets are automatically revoked.

5. How is dependency scanning managed within GHAS?

GHAS shows known CVEs via the dependency graph and triggers *Dependabot* alerts. It identifies vulnerable versions and suggests secure updates.

6. What is the difference between alerts and security overview in GHAS?

Alerts show specific issues (e.g., a secret or CVE). Security overview aggregates all alerts, provides trends, and gives org-wide or repo-wide posture insight.

7. How do you enforce GHAS checks in branch protection rules?

Enable required status checks (e.g., CodeQL, secret scanning) under *Branch protection rules*. This blocks merging PRs with unresolved security issues.

8. What are the licensing and billing requirements for GHAS?

GHAS requires an Enterprise plan with Advanced Security enabled. Billing is per active committer per month for each enabled private repository.

9. How does GHAS integrate with CI/CD workflows and GitHub Actions?

GHAS tools (e.g., CodeQL) run as *GitHub Actions* in the CI pipeline. They can fail builds, comment on PRs, and prevent merging based on scan results.

10. What are best practices for managing GHAS alerts and findings?

Triage alerts promptly, prioritize critical issues, auto-create issues from alerts, integrate with project boards, and use codeowners for review responsibility.

4. Security and compliance

└ 4.4 Security and compliance scanning

└ 4.4.4 CodeQL, Container Scanning

-
1. What is CodeQL and how does it work in GitHub Advanced Security?
 2. How do you configure CodeQL analysis in a GitHub repository?
 3. What languages are supported by CodeQL?
 4. How does CodeQL identify vulnerabilities?
 5. How do you run CodeQL in a containerized environment?
 6. What tools are commonly used for container image scanning?
 7. How can GitHub Actions be used to scan container images?
 8. How do you automate container scanning in CI/CD pipelines?
 9. How are container scan results surfaced and consumed?
 10. What best practices ensure secure use of CodeQL and container scanning in DevOps?
-

1. What is CodeQL and how does it work in GitHub Advanced Security?

CodeQL performs semantic analysis on source code using a query language to detect vulnerabilities. It analyzes code on PRs, pushes, and schedules.

2. How do you configure CodeQL analysis in a GitHub repository?

Add the `github/codeql-action/init` and `analyze` actions in a `.github/workflows/codeql.yml` file. Select target languages and triggers (e.g., push, PR).

3. What languages are supported by CodeQL?

CodeQL supports C/C++, C#, Go, Java, JavaScript/TypeScript, Python, Ruby, and Swift.

4. How does CodeQL identify vulnerabilities?

It uses a query engine to detect security patterns (e.g., SQL injection, XSS). Queries are run against the code database built during analysis.

5. How do you run CodeQL in a containerized environment?

Use GitHub-hosted or self-hosted runners with container support. CodeQL can analyze code inside Docker containers by configuring the `container: option` in workflows.

6. What tools are commonly used for container image scanning?

Microsoft Defender for Cloud, *Trivy*, *Aqua*, and *Anchore* are popular tools. They detect OS/package-level vulnerabilities and policy violations.

7. How can GitHub Actions be used to scan container images?

Use actions like `aquasecurity/trivy-action` or `anchore/scan-action` in a workflow to scan images after build and fail the job on high-severity findings.

8. How do you automate container scanning in CI/CD pipelines?

Integrate scanning steps post-image-build in *GitHub Actions* or *Azure Pipelines*. Scan images before pushing to registry and gate releases on scan results.

9. How are container scan results surfaced and consumed?

Results appear in the *GitHub* Security tab (if supported), job logs, or are exported to external tools. *Defender* shows results in *Azure Security Center*.

10. What best practices ensure secure use of CodeQL and container scanning in DevOps?

- Scan on every PR,
- enforce status checks,
- update scan tools regularly,
- exclude false positives via configuration,
- and act on critical alerts before deploy.

4. Security and compliance

└ 4.4 Security and compliance scanning

└ 4.4.5 Dependabot for OSS scanning

-
1. What is Dependabot and how does it help with open-source security?
 2. How do you enable and configure Dependabot in a GitHub repository?
 3. What file types and ecosystems does Dependabot support?
 4. How does Dependabot detect and report vulnerabilities?
 5. What are Dependabot security updates vs version updates?
 6. How are vulnerabilities prioritized in Dependabot alerts?
 7. How can PRs from Dependabot be managed and approved?
 8. How do you control update frequency and behavior in Dependabot?
 9. Where are Dependabot alerts shown and how are they resolved?
 10. What are best practices for integrating Dependabot in CI/CD?
-

1. What is Dependabot and how does it help with open-source security?

Dependabot scans dependencies for known vulnerabilities and automatically creates pull requests to upgrade to secure versions.

2. How do you enable and configure Dependabot in a GitHub repository?

Create a `.github/dependabot.yml` file defining package ecosystems, directories, and update frequency. Security updates are enabled by default in repo settings.

3. What file types and ecosystems does Dependabot support?

It supports npm, Maven, pip, RubyGems, NuGet, Cargo, Go modules, Docker, and more—based on lockfiles or manifest files in the repo.

4. How does Dependabot detect and report vulnerabilities?

It matches dependency versions in the repo against the GitHub Advisory Database. If a match exists, it creates alerts and optionally PRs.

5. What are Dependabot security updates vs version updates?

- Security updates target known vulnerabilities.
- Version updates are routine upgrades (e.g., minor/patch bumps).

Both are configured via `dependabot.yml`.

6. How are vulnerabilities prioritized in Dependabot alerts?

Alerts are ranked by CVSS severity (low to critical) and show impact details, affected version ranges, and fixed versions.

7. How can PRs from Dependabot be managed and approved?

PRs can be auto-approved using *GitHub Actions* or require manual review. Apply branch protection rules to enforce checks before merge.

8. How do you control update frequency and behavior in Dependabot?

In `dependabot.yml`, set `schedule.interval` (daily/weekly/monthly), and `open-pull-requests-limit` to manage PR volume.

9. Where are Dependabot alerts shown and how are they resolved?

Alerts appear in the *Security > Dependabot* tab. Resolve them by merging the auto-generated PR or manually updating dependencies.

10. What are best practices for integrating Dependabot in CI/CD?

Enable alerts, review PRs promptly, restrict PRs to safe branches, run tests on PRs, and auto-merge low-risk updates after passing checks.

5. Instrumentation

└ 5.1 Monitoring Setup

└ 5.1.1 Configure Azure Monitor, Log Analytics

1. What is Azure Monitor and how does it relate to Log Analytics?
 2. How do you enable and configure Azure Monitor for a resource?
 3. How do you link a Log Analytics workspace to Azure Monitor?
 4. What data types are collected by default in Azure Monitor?
 5. How do you configure Application Insights with Azure Monitor?
 6. How do you configure VM Insights and Container Insights?
 7. How do you query logs using KQL in Log Analytics?
 8. How do you configure data retention and diagnostic settings?
 9. How do you integrate Azure Monitor with Azure DevOps or GitHub?
 10. How do you create alerts from Log Analytics queries?
-

1. What is Azure Monitor and how does it relate to Log Analytics?

- *Azure Monitor* is a platform for collecting, analyzing, and acting on telemetry.
 - *Log Analytics* is its query engine and data platform, storing logs in a workspace that *Azure Monitor* uses for diagnostics and alerting.
-

2. How do you enable and configure Azure Monitor for a resource?

Go to the *Azure resource* → *Monitoring* → *Diagnostic settings* → *Enable diagnostics* → *Select logs and metrics* → *Route to Log Analytics workspace or Event Hub*.

3. How do you link a Log Analytics workspace to Azure Monitor?

In the resource's Diagnostic settings, select "Send to Log Analytics workspace" and choose the subscription and workspace. Logs are then streamed automatically.

4. What data types are collected by default in Azure Monitor?

Metrics (numeric performance data) and logs (activity, diagnostics). Resource types vary: *VM Insights* collects perf counters, while *App Insights* collects requests, dependencies, exceptions.

5. How do you configure Application Insights with Azure Monitor?

Enable *Application Insights* via *Azure Portal* or SDK. It auto-collects telemetry and integrates with *Log Analytics* if linked. Use `ApplicationInsights.config` or code for customization.

6. How do you configure VM Insights and Container Insights?

Install the *Log Analytics* agent or *Azure Monitor Agent (AMA)* on VMs. For containers, enable monitoring in the AKS cluster or attach agents manually to non-AKS clusters.

7. How do you query logs using KQL in Log Analytics?

Open Logs under the workspace or resource → *Use KQL* (e.g., `AzureDiagnostics | where ResourceType == "XYZ"`). Use `project`, `summarize`, `where`, `join` for insights.

8. How do you configure data retention and diagnostic settings?

In *Log Analytics workspace* → *Usage and estimated costs* → *Data retention* → *Set days* (up to 730). Configure Diagnostic settings per resource to control data flow and granularity.

9. How do you integrate Azure Monitor with Azure DevOps or GitHub?

Use *Action Groups* with *Logic Apps* or *Webhooks* to trigger work item creation in *Azure DevOps* or *GitHub Issues* when alerts are fired.

10. How do you create alerts from Log Analytics queries?

Run a KQL query → Click “New Alert Rule” → Define alert logic, threshold, frequency → Attach to an Action Group (email, webhook, Logic App) → Save.

5. Instrumentation

└ 5.1 Monitoring Setup

└ 5.1.2 Telemetry with App Insights, VM/Container/Storage/Network Insights

1. What telemetry data does Application Insights collect by default?
 2. How do you enable Application Insights in an app or service?
 3. How do you instrument custom telemetry with Application Insights SDK?
 4. What is VM Insights and what telemetry does it collect?
 5. How do you enable VM Insights on Azure VMs?
 6. What is Container Insights and what data does it gather?
 7. How is Container Insights configured on AKS?
 8. How do Storage Insights enhance storage monitoring?
 9. What metrics are available from Network Insights?
 10. How can telemetry from these Insights be correlated in Log Analytics?
-

1. What telemetry data does Application Insights collect by default?

- Requests,
 - dependencies,
 - exceptions,
 - traces,
 - page views,
 - performance counters,
 - and availability test results.
-

2. How do you enable Application Insights in an app or service?

Use *Azure Portal* during resource creation, or manually integrate SDK (e.g., .NET, Node.js) and configure instrumentation key or connection string.

3. How do you instrument custom telemetry with Application Insights SDK?

Use SDK methods like `TrackEvent`, `TrackMetric`, `TrackTrace`, `TrackException` to send custom telemetry. Context properties can be added for traceability.

4. What is VM Insights and what telemetry does it collect?

It provides performance metrics (CPU, memory, disk, network), dependency maps, and process info for Azure/Arc VMs.

5. How do you enable VM Insights on Azure VMs?

Enable from the *Azure Monitor blade or VM* → *Insights* → *Enable*. This deploys the monitoring agent and connects to a *Log Analytics* workspace.

6. What is Container Insights and what data does it gather?

It collects container performance (CPU, memory), node health, pod logs, and Kubernetes event logs for AKS or other clusters.

7. How is Container Insights configured on AKS?

Enable monitoring during AKS creation or later from the *Azure Portal* → *Insights*. *Azure Monitor* agent is deployed to collect telemetry.

8. How do Storage Insights enhance storage monitoring?

They provide metrics like

- capacity,
- transactions,
- latency,
- and availability per blob/table/file/queue

via diagnostic settings sent to Log Analytics.

9. What metrics are available from Network Insights?

Metrics include

- throughput,
- packet loss,
- latency,
- SNAT port exhaustion,
- and flow data

from NSGs, Load Balancers, and VPN gateways.

10. How can telemetry from these Insights be correlated in Log Analytics?

All telemetry streams into the same *Log Analytics* workspace and can be queried using KQL. Use joins and time filters to correlate across services.

5. Instrumentation

└ 5.2 Alerting and metrics

└ 5.2.1 GitHub monitoring insights/charts

1. What monitoring insights are available in GitHub repositories?
 2. How do you access and interpret GitHub Insights charts (e.g., Pulse, Traffic)?
 3. What key repository metrics are commonly visualized in GitHub Insights?
 4. How do you enable and configure repository Insights in GitHub?
 5. How can you monitor workflow and pipeline health using GitHub Actions insights?
 6. What are best practices for using charts to identify trends in repository activity?
 7. How do you set up alerts for repository or workflow events in GitHub?
 8. How can you export or integrate GitHub Insights data with other monitoring tools?
 9. What permissions are needed to view and configure Insights in a GitHub repository?
 10. What are the limitations of GitHub Insights for monitoring production-grade DevOps environments?
-

1. What monitoring insights are available in GitHub repositories?

GitHub provides built-in insights including

- Pulse (activity overview),
 - Traffic (clones/views),
 - Contributors (commit stats),
 - Commits,
 - Code Frequency,
 - and Dependency Graph.
-

2. How do you access and interpret GitHub Insights charts (e.g., Pulse, Traffic)?

Go to the “*Insights*” tab of a repository.

- Pulse shows merged PRs, opened/closed issues, and commits over time.
 - Traffic displays unique visitors, page views, clones, and referrers.
-

3. What key repository metrics are commonly visualized in GitHub Insights?

Metrics include

- commits per period,
 - contributor stats,
 - issue and PR activity,
 - code additions/deletions,
 - clone/view counts,
 - and dependency relationships.
-

4. How do you enable and configure repository Insights in GitHub?

Insights are enabled by default. No special configuration is needed; users require at least read access to view most charts.

5. How can you monitor workflow and pipeline health using GitHub Actions insights?

Go to “*Actions*” > “*Insights*”. View run duration, workflow success/failure rates, recent run history, and failure causes.

6. What are best practices for using charts to identify trends in repository activity?

Regularly review commit frequency, issue/PR trends, and contributor activity to identify bottlenecks or spikes.

7. How do you set up alerts for repository or workflow events in GitHub?

Use the “Watch” setting for notifications. Configure Actions workflows to notify via email, chat, or external integrations using workflow steps or *GitHub Apps*.

8. How can you export or integrate GitHub Insights data with other monitoring tools?

Use the *GitHub* API to export data (commits, issues, traffic). Integrate with external tools (e.g., *Azure Monitor*, *Power BI*) using APIs or webhooks.

9. What permissions are needed to view and configure Insights in a GitHub repository?

Read access is sufficient for most Insights. Traffic stats require admin/owner rights. Only admins can manage feature availability.

10. What are the limitations of GitHub Insights for monitoring production-grade DevOps environments?

Insights are repository-scoped, lack advanced custom dashboards, and do not provide real-time or cross-repo observability. Use dedicated monitoring tools for advanced needs.

5. Instrumentation

└ 5.2 Alerting and metrics

└ 5.2.2 Alerts in GitHub Actions and Azure Pipelines

1. What types of alerts can be configured in GitHub Actions and Azure Pipelines?
 2. How do you configure alerts for failed workflows in GitHub Actions?
 3. How do you set up alert notifications in Azure Pipelines?
 4. What are the main channels for receiving alerts from GitHub Actions?
 5. How do you send custom alerts in a GitHub Actions workflow?
 6. How can you integrate Azure Pipelines alerts with external systems (e.g., Teams, Slack)?
 7. What are best practices for actionable alerting in CI/CD pipelines?
 8. How do you suppress or customize notification frequency in GitHub Actions and Azure Pipelines?
 9. What permissions are required to configure alert rules in GitHub Actions and Azure Pipelines?
 10. What limitations exist with built-in alerting in GitHub Actions and Azure Pipelines?
-

1. What types of alerts can be configured in GitHub Actions and Azure Pipelines?

Alerts include

- workflow/job failures,
 - run duration warnings,
 - environment deployment issues,
 - and custom status checks.
-

2. How do you configure alerts for failed workflows in GitHub Actions?

By default, users receive notifications for workflow failures. Customize alerts using workflow steps (e.g., send email, *Slack*) or third-party *Actions* on failure.

3. How do you set up alert notifications in Azure Pipelines?

Use *Project Settings* → *Notifications*. Create subscriptions for pipeline run events (e.g., failures, approvals), set filters, and select recipients.

4. What are the main channels for receiving alerts from GitHub Actions?

Notifications can be received via

- email,
 - *GitHub* web,
 - mobile app,
 - or external services (e.g., *Slack*, *Teams*) via custom workflow steps.
-

5. How do you send custom alerts in a GitHub Actions workflow?

Add steps to use *Actions* or scripts that trigger webhooks, emails, or chat notifications upon job failure or completion (e.g., actions/notify, curl to API).

6. How can you integrate Azure Pipelines alerts with external systems (e.g., Teams, Slack)?

Use Service Hooks or add marketplace extensions (e.g., *Slack*, *Teams* connector) to send alerts on pipeline events to external channels.

7. What are best practices for actionable alerting in CI/CD pipelines?

- Alert only on actionable events (e.g., failures, blocking approvals),
 - avoid excessive noise,
 - include context (run links, error details),
 - and direct alerts to responsible teams.
-

8. How do you suppress or customize notification frequency in GitHub Actions and Azure Pipelines?

Configure personal and project notification settings to mute, group, or restrict alerts to certain event types or severities.

9. What permissions are required to configure alert rules in GitHub Actions and Azure Pipelines?

- Repository write/admin access is needed for workflow modification in *GitHub Actions*.
 - In *Azure Pipelines*, project admin or manage notifications permission is required.
-

10. What limitations exist with built-in alerting in GitHub Actions and Azure Pipelines?

Built-in alerting is basic: lacks advanced filtering, grouping, or cross-project aggregation. Use external monitoring or notification platforms for advanced scenarios.

5. Instrumentation

└ 5.2 Alerting and metrics

└ 5.3.1 Inspect: CPU, memory, disk, network

-
1. What tools are available in Azure to inspect CPU, memory, disk, and network metrics?
 2. How do you enable and configure monitoring for infrastructure metrics in Azure?
 3. Where are infrastructure metrics (CPU, memory, disk, network) visualized in the Azure portal?
 4. How do you set up alerts for CPU, memory, disk, or network thresholds in Azure Monitor?
 5. What is the difference between Azure Metrics and Azure Logs for infrastructure monitoring?
 6. How can Application Insights be used for infrastructure telemetry?
 7. What are best practices for monitoring high CPU usage in Azure VMs?
 8. How do you collect and analyze performance counters for Azure resources?
 9. What are the recommended retention and sampling settings for metrics?
 10. How do you diagnose and troubleshoot resource bottlenecks using collected telemetry?
-

1. What tools are available in Azure to inspect CPU, memory, disk, and network metrics?

Azure Monitor and *Azure Log Analytics* provide core infrastructure metrics. *VM Insights* and *Container Insights* give detailed metrics for VMs and containers.

2. How do you enable and configure monitoring for infrastructure metrics in Azure?

Enable monitoring by installing the *Azure Monitor Agent* or *Log Analytics* agent on your resources. Register your VM or container to a *Log Analytics* workspace and enable Insights as required.

3. Where are infrastructure metrics (CPU, memory, disk, network) visualized in the Azure portal?

Metrics are visualized in *Azure Monitor* under “Metrics” or via “Insights” tabs on VM/Container resources. Dashboards can aggregate multiple metrics for quick visibility.

4. How do you set up alerts for CPU, memory, disk, or network thresholds in Azure Monitor?

Create an Alert Rule in *Azure Monitor*. Define a condition (e.g., CPU > 80%), set the evaluation period, and assign an action group for notifications (email, SMS, webhook, etc.).

5. What is the difference between Azure Metrics and Azure Logs for infrastructure monitoring?

Metrics provide near real-time numeric time series (e.g., CPU percentage), optimized for performance. Logs provide richer, queryable telemetry (e.g., event details), useful for diagnostics and historical analysis.

6. How can Application Insights be used for infrastructure telemetry?

Application Insights primarily collects app-level telemetry, but can be extended to include infrastructure metrics by connecting to *Azure Monitor* or configuring custom telemetry ingestion.

7. What are best practices for monitoring high CPU usage in Azure VMs?

Set alerts for sustained high CPU usage, review resource utilization in *Metrics Explorer*, investigate running processes, and scale up/out as required.

8. How do you collect and analyze performance counters for Azure resources?

Enable collection of performance counters using the Log Analytics agent. Use *Log Analytics* workspace to query and analyze the data (e.g., Perf table for Windows/Linux metrics).

9. What are the recommended retention and sampling settings for metrics?

Use default retention (typically 93 days for metrics, 30 days for logs, extendable as needed). Set sampling for logs and custom metrics to balance cost and granularity.

10. How do you diagnose and troubleshoot resource bottlenecks using collected telemetry?

- Correlate spikes in CPU, memory, disk, or network metrics with application and OS logs.
- Use Log Analytics queries to identify patterns and root causes.
- Review recommendations in *Azure Advisor* and *Insights* for optimization actions.

5. Instrumentation

└ 5.2 Alerting and metrics

└ 5.3.2 Analyze app usage/performance metrics

-
1. What Azure tools are used to analyze application usage and performance metrics?
 2. How do you enable application performance monitoring in Azure?
 3. What types of application usage metrics can be collected in Azure?
 4. How do you visualize and interpret performance metrics in Application Insights?
 5. How do you configure custom metrics and events in Application Insights?
 6. What is the difference between availability, performance, and usage metrics?
 7. How do you set up alerts based on application performance thresholds?
 8. How can you use Kusto Query Language (KQL) to analyze usage and performance data?
 9. What best practices should be followed when monitoring distributed applications?
 10. How do you diagnose and resolve application performance bottlenecks using Azure telemetry?
-

1. What Azure tools are used to analyze application usage and performance metrics?

Azure Application Insights and *Azure Monitor* are the primary tools. They provide telemetry on usage, performance, failures, and dependencies.

2. How do you enable application performance monitoring in Azure?

Install and configure the *Application Insights SDK* or use the *Azure Monitor agent*. Enable monitoring from the *Azure portal* or through instrumentation in application code.

3. What types of application usage metrics can be collected in Azure?

You can collect

- request counts,
 - response times,
 - failed requests,
 - user sessions,
 - page views,
 - custom events,
 - and dependency calls.
-

4. How do you visualize and interpret performance metrics in Application Insights?

Use the *Application Insights* “Metrics” and “Performance” blades for charts and tables. Review response time distributions, failure trends, and drill into specific requests or dependencies.

5. How do you configure custom metrics and events in Application Insights?

Track custom metrics or events using the *Application Insights SDK*.

Use methods like

- `TrackEvent`,
- `TrackMetric`,
- or `TrackException`

in your application code.

6. What is the difference between availability, performance, and usage metrics?

- Availability metrics show if an app is up and responding.
 - Performance metrics measure speed (response times, resource usage).
 - Usage metrics show how the app is used (sessions, users, page views).
-

7. How do you set up alerts based on application performance thresholds?

In *Application Insights* or *Azure Monitor*, create an alert rule, define the target metric (e.g., response time > 1s), and assign an *Action Group* for notifications.

8. How can you use Kusto Query Language (KQL) to analyze usage and performance data?

Use *Log Analytics* or *Application Insights* “Logs” to write KQL queries. Query tables like requests, dependencies, traces, and visualize results or export data.

9. What best practices should be followed when monitoring distributed applications?

- Instrument all services,
 - use distributed tracing,
 - correlate telemetry via operation IDs,
 - set up centralized dashboards
 - and alerts across services.
-

11. How do you diagnose and resolve application performance bottlenecks using Azure telemetry?

- Analyze slow requests and dependencies in *Application Insights*,
- review end-to-end transaction views,
- use failure and performance charts,
- and drill into traces to find root causes.

Optimize code, resources, or dependencies based on findings.

5. Instrumentation

└ 5.2 Alerting and metrics

└ 5.3.3 Distributed tracing with App Insights

-
1. What is distributed tracing and why is it critical in microservices or cloud-native architectures?
 2. How does Application Insights implement distributed tracing?
 3. What are key telemetry types collected by App Insights for distributed tracing?
 4. How do you instrument .NET/Java/Node.js applications for distributed tracing with App Insights?
 5. What is an operation ID and how does it correlate telemetry across services?
 6. How are dependencies and requests visualized in the App Insights Application Map?
 7. How can you query distributed traces in Application Insights using Kusto Query Language (KQL)?
 8. How do you set up automatic vs. manual correlation for custom telemetry in distributed tracing?
 9. What are best practices to minimize trace sampling impact and maximize troubleshooting value?
 10. How do you integrate distributed tracing from App Insights with Azure Monitor or third-party observability tools?
-

1. What is distributed tracing and why is it critical in microservices or cloud-native architectures?

Distributed tracing is tracking a request as it flows across multiple services or components. It is critical in microservices/cloud-native environments to identify bottlenecks, diagnose failures, and understand dependencies across distributed systems.

2. How does Application Insights implement distributed tracing?

Application Insights uses correlation IDs injected into telemetry (requests, dependencies, exceptions) to automatically track and link operations across components and services.

3. What are key telemetry types collected by App Insights for distributed tracing?

Key telemetry types:

- Request telemetry (incoming HTTP requests),
 - Dependency telemetry (calls to databases, HTTP, services),
 - Exception telemetry,
 - Trace telemetry (logs),
 - and Custom telemetry (events, metrics).
-

4. How do you instrument .NET/Java/Node.js applications for distributed tracing with App Insights?

Use *Application Insights* SDK or agents.

- .NET: Install the *NuGet* package or enable with *Application Insights* Agent.
 - Java: Use *Application Insights* Java agent.
 - Node.js: Use the npm package. Configuration auto-instruments most frameworks.
-

5. What is an operation ID and how does it correlate telemetry across services?

An operation ID is a unique identifier assigned to each end-to-end request. It propagates across services via headers, allowing *App Insights* to correlate telemetry events for a single operation.

6. How are dependencies and requests visualized in the App Insights Application Map?

Application Map displays detected components and dependencies as a graph, showing request flow, health, response times, and failure hotspots, enabling quick drill-down into distributed traces.

7. How can you query distributed traces in Application Insights using Kusto Query Language (KQL)?

Use the *traces*, *requests*, and *dependencies* tables in *Log Analytics*. Query by *operation_Id* to retrieve and analyze related events across services for a given distributed trace.

8. How do you set up automatic vs. manual correlation for custom telemetry in distributed tracing?

Automatic correlation is handled by the SDK/agent for supported frameworks. For custom telemetry, set the *operation_Id* and *parent_Id* explicitly when tracking events to maintain trace linkage.

9. What are best practices to minimize trace sampling impact and maximize troubleshooting value?

Use adaptive sampling to balance telemetry volume and cost. Always capture telemetry for errors/exceptions. Tune sampling rates as needed to ensure critical traces are retained for troubleshooting.

10. How do you integrate distributed tracing from App Insights with Azure Monitor or third-party observability tools?

Enable integration by configuring Diagnostic settings to export logs/metrics to *Azure Monitor* or *Log Analytics*. Use *OpenTelemetry* exporters or API integrations for third-party tools.

5. Instrumentation

└ 5.2 Alerting and metrics

└ 5.3.4 Kusto Query Language (KQL) basics

-
1. What is Kusto Query Language (KQL) and where is it used in Azure DevOps monitoring?
 2. What are the main KQL data tables relevant for Application Insights?
 3. How do you write a basic query to retrieve request telemetry in KQL?
 4. What is the syntax for filtering, projecting, and ordering data in KQL?
 5. How do you aggregate and summarize data using KQL?
 6. How can you join multiple tables in KQL for advanced insights?
 7. What functions are available in KQL to parse and extract fields from logs?
 8. How do you visualize query results in Azure Monitor using KQL?
 9. What are common pitfalls and best practices for writing efficient KQL queries?
 10. How do you save and share KQL queries in Azure Monitor or Application Insights?
-

1. What is Kusto Query Language (KQL) and where is it used in Azure DevOps monitoring?

KQL is a read-only query language for analyzing large volumes of structured, semi-structured, and unstructured data stored in Azure Data Explorer, Log Analytics, and Application Insights.

2. What are the main KQL data tables relevant for Application Insights?

Common tables: requests, dependencies, exceptions, traces, customEvents, and availabilityResults.

3. How do you write a basic query to retrieve request telemetry in KQL?

```
requests  
| take 10
```

4. What is the syntax for filtering, projecting, and ordering data in KQL?

Use where to filter, project to select columns, and order by to sort. Example:

```
requests  
| where resultCode == "500"  
| project timestamp, name, url  
| order by timestamp desc
```

5. How do you aggregate and summarize data using KQL?

Use summarize for aggregation. Example:

```
requests  
| summarize count() by resultCode
```

6. How can you join multiple tables in KQL for advanced insights?

Use the join operator. Example:

```
requests  
| join kind=inner (dependencies) on operation_Id
```

7. What functions are available in KQL to parse and extract fields from logs?

Functions: parse, extract, split, substring, tostring. Example:

```
traces  
| extend errorType = extract("Error: (.*)", 1, message)
```

8. How do you visualize query results in Azure Monitor using KQL?

Run the KQL query and select visualization options (time chart, pie, table) from the UI toolbar.

9. What are common pitfalls and best practices for writing efficient KQL queries?

- Pitfalls: Large scans, missing filters, unnecessary joins.
 - Best practices:
 - Filter early,
 - project only needed fields,
 - use summarization,
 - and test queries for performance.
-

10. How do you save and share KQL queries in Azure Monitor or Application Insights?

Use "Save" or "Pin to dashboard" in the query editor. Share query links or save as a shared workbook for collaboration.